Computational Control

Lecture Handout

June 26, 2026

Contents

1 Dynamic Programming and LQR **5**
1.1 Introduction . . . 5
1.2 Dynamic Programming . . . 6
1.3 Optimal LQR . . . 7
1.4 Dynamic Programming for LQR . . . 7
1.4.1 Infinite-Horizon LQR . . . 9

2 Model Predictive Control **13**
2.1 Introduction . . . 13
2.2 Implementation of MPC . . . 17
2.2.1 Explicit MPC . . . 17
2.3 Stability . . . 20
2.4 Disturbance rejection . . . 25
2.4.1 Zero steady-state offset and incremental MPC . . . 26
2.4.2 Disturbance rejection during transients . . . 27
2.4.3 Advanced disturbance models . . . 28
2.4.4 Robust satisfaction of constraints . . . 29
2.4.5 Prior information on disturbances . . . 30
2.4.6 Open-loop robust MPC . . . 31
2.4.7 Why open-loop robust MPC can be too conservative . . . 32
2.4.8 Robust MPC over policies . . . 33
2.4.9 Nested feedback for robust MPC . . . 34

3 Economic MPC **36**
3.1 From setpoint tracking to nonzero steady states . . . 36
3.1.1 Computation of a feasible steady-state target . . . 36
3.1.2 Computation of the steady-state target . . . 36
3.2 Motivation for economic MPC . . . 37
3.2.1 Steady-state economic optimization plus tracking MPC . . . 37
3.3 Economic MPC formulation . . . 38
3.3.1 Dynamic economic optimization . . . 38
3.3.2 Infinite horizon, finite horizon, and terminal constraints . . . 39
3.4 Illustrative example . . . 40
3.4.1 Computation of the optimal steady state . . . 40
3.4.2 Tracking MPC versus economic MPC . . . 41
3.5 Closed-loop analysis . . . 42
3.5.1 Tracking MPC: Lyapunov decrease . . . 42
3.5.2 Economic MPC: why the value function is not a Lyapunov function . . . 42
3.5.3 Asymptotic average performance . . . 43

4 Feedback optimization **45**
4.1 Problem setting and architecture . . . 45
4.1.1 Optimal steady-state regulation . . . 45
4.1.2 Fast-stable plant . . . 46
4.1.3 “Certainty-equivalence” design . . . 47
4.1.4 Interconnecting plant and algorithms . . . 49
4.1.5 Online feedback optimization . . . 50
4.2 First-order methods for feedback optimization . . . 51
4.2.1 First-order optimization algorithms . . . 51

4.2.2 Gradient descent with penalty / barrier . . . 51
4.2.3 Unconstrained gradient descent . . . 53
4.2.4 Gradient descent with penalty . . . 54
4.2.5 Projected gradient descent . . . 56
4.2.6 Projected gradient flow via repeated quadratic programming . . . 58
4.3 Duality-based methods . . . 60
4.3.1 Dual problem . . . 60
4.3.2 Primal–dual iterations . . . 61
4.3.3 Dual ascent for feedback optimization . . . 61
4.3.4 An old friend: dual ascent and PI control . . . 63
4.4 Design insights . . . 64
4.4.1 Design guidelines and tradeoffs . . . 64
4.5 Extremum seeking . . . 66
4.5.1 Extremum seeking . . . 66
4.5.2 Extremum-seeking architecture . . . 67
4.5.3 Why extremum seeking behaves like gradient descent . . . 68
4.5.4 Applications of extremum seeking . . . 70

5 System Identification 71
5.1 Controllability, observability, and minimal realizations . . . 73
5.2 Realization from data and the Ho–Kalman construction . . . 75
5.3 SVD-based Ho–Kalman realization . . . 77

6 Data-driven LQR 80
6.1 LQR parameterization . . . 80
6.2 SDP formulation of the LQR . . . 81
6.3 From model-based to data-driven LQR . . . 84
6.4 Indirect data-driven control . . . 85
6.4.1 System identification from input-state data . . . 85
6.4.2 Indirect and certainty-equivalent LQR . . . 86
6.5 Direct data-driven LQR via trajectory parameterization . . . 87
6.5.1 Non-uniqueness of the trajectory parameterization . . . 88
6.5.2 Equivalence between direct and indirect certainty-equivalent LQR . . . 89
6.5.3 Certainty-equivalence-promoting regularization . . . 90
6.5.4 Robustness-promoting regularization for trajectory parameterization . . . 91
6.5.5 Summary and limitations of direct trajectory parameterization . . . 92
6.6 Direct data-driven LQR via sample covariance parameterization . . . 93
6.6.1 Robustness-promoting regularization for covariance parameterization . . . 94
6.7 Comparison of data-driven LQR methods . . . 95

7 Data-Driven Predictive Control 98
7.1 Lifted input–output representation . . . 98
7.2 Free response and observability . . . 99
7.2.1 Example: scalar system . . . 99
7.3 Known input sequence and state reconstruction . . . 100
7.4 Behavioral description of finite-horizon trajectories . . . 101
7.4.1 Behavioral viewpoint . . . 102
7.4.2 Why the horizon $L = n + 1$ is the first predictive horizon . . . 102
7.4.3 Multi-step prediction as trajectory completion . . . 103
7.4.4 Basis representation of the trajectory space . . . 104
7.4.5 Example: rank of the trajectory matrix . . . 106
7.5 From trajectory bases to Hankel-based prediction . . . 106

7.5.1 Reusing a single experiment through overlapping windows . . . 106
7.5.2 Hankel matrices as data-driven trajectory bases . . . 107
7.5.3 Data-driven prediction with past and future blocks . . . 108
7.6 From data-driven prediction to data-enabled predictive control . . . 108
7.6.1 From prediction to control . . . 108
7.6.2 The data-enabled predictive control formulation . . . 109
7.6.3 Comparison with model-based MPC . . . 110
7.7 Practical issues: excitation, MIMO extension, and noise . . . 110
7.7.1 Persistence of excitation . . . 110
7.7.2 Extension from SISO to MIMO . . . 111
7.7.3 Effect of noise on the trajectory representation . . . 112
7.7.4 Regularization and slack variables . . . 113
7.8 Limitations and final perspective . . . 115
7.8.1 Main limitations . . . 115
7.8.2 When data-driven predictive control is attractive . . . 116

8 Markov Decision Processes 117
8.1 Motivation: stochastic decision problems . . . 117
8.2 Finite Markov Decision Processes . . . 117
8.2.1 Policies . . . 118
8.3 Finite-horizon dynamic programming . . . 119
8.3.1 Finite-horizon sequential decision problems . . . 119
8.3.2 Dynamic programming recursion for a fixed policy . . . 119
8.3.3 Bellman's optimality principle and backward induction . . . 120
8.4 Infinite-horizon discounted MDPs . . . 121
8.4.1 Stationary problems and discounted cost . . . 121
8.4.2 Discount factor . . . 122
8.4.3 Closed-loop MDP and stationary distribution . . . 122
8.4.4 Closed-loop queue example . . . 123
8.5 Policy evaluation and Bellman optimality . . . 124
8.5.1 Infinite-horizon value of a policy . . . 124
8.5.2 Computational complexity of policy evaluation . . . 124
8.5.3 Queue example for policy evaluation . . . 125
8.5.4 Bellman principle for infinite-horizon MDPs . . . 125
8.6 Iterative computation of the optimal policy . . . 126
8.6.1 Greedy policy improvement . . . 126
8.6.2 Policy improvement theorem . . . 127
8.6.3 Policy iteration algorithm . . . 128
8.6.4 Value iteration . . . 129
8.6.5 Value iteration algorithm . . . 129

9 Monte Carlo Learning 131
9.1 From model-based dynamic programming to learning . . . 131
9.2 The Q-function . . . 132
9.3 Bellman equations in Q-form . . . 132
9.4 Bellman optimality in Q-form . . . 133
9.5 Policy iteration with Q-functions . . . 133
9.6 Experimental policy evaluation . . . 134
9.7 Exploration and exploitation . . . 135
9.8 Scalability and parametrization of Q . . . 135
9.9 Linear parametrization . . . 136
9.10 Projected Bellman equation . . . 137

9.11 Matrix form of the projected Bellman equation . . . 137
9.12 Model-free computation of the projected Bellman equation . . . 138
9.13 Empirical interpretation of the weighted matrices . . . 140
9.14 Monte Carlo learning with linear approximation . . . 141
9.14.1 Policy evaluation . . . 141
9.14.2 Greedy policy improvement . . . 141
9.15 Key points . . . 142

10 Reinforcement Learning **143**
10.1 From model information to learning from data . . . 143
10.2 Monte Carlo policy evaluation . . . 143
10.3 Temporal-difference error . . . 144
10.4 Stochastic approximation . . . 145
10.5 TD-learning and SARSA . . . 146
10.5.1 Interleaving policy evaluation and improvement . . . 146
10.6 Q-learning . . . 147
10.7 SARSA versus Q-learning . . . 148
10.8 Q-learning as stochastic gradient descent . . . 148
10.9 Q-learning with linear approximation . . . 149
10.10 General approximators . . . 150
10.11 Is it really model-free? . . . 150
10.12 Policy gradient methods . . . 151

A Convex optimization **152**
A.1 Convex sets and convex functions . . . 152
A.2 Derivative-based characterizations of convexity . . . 154
A.3 Convex optimization problems . . . 154
A.4 First-order optimality conditions . . . 155
A.5 KKT conditions . . . 155
A.6 Lagrangian viewpoint . . . 156
A.7 Connection with MPC and control . . . 156

1 Dynamic Programming and LQR

1.1 Introduction

In general we formulate the optimization control task as

$$\min_{\mathbf{u}\in\mathcal{U}} J(\mathbf{u}) \tag{1}$$

where $\mathcal{U} = \tilde{\mathcal{U}} \times \{0, 1, \dots, \mathrm{T}\}$ is the set of the feasible inputs and $J(\mathbf{u})$ is generally the sum of some *stage costs* and a terminal cost/constraint.
The issue is that generally 1 is an intractable optimization problem. This is due to several factors:

- the dimension of $\mathbf{u}$ is too big;
- the cost function requires an accurate model of the system;
- the optimization problem is **non-convex**;
- In presence of **disturbances** and **uncertainity** the open-loop nature of the task makes it useless

In this context, Dynamic Programming (DP) is a powerfull tool whenever we can decompose the problem. In particular when the problem allows for a **Markovian representation**:

Key assumptions: Markovian representation

The following hypothesis are crucial in order for DP to be meaningfull:

1. There exists a **Markovian State**[a] that evolves according to some dynamics
$$x_{t+1} = f_t(x_t, u_t)^{[b]}$$
2. The initial condition x_0 is known.
3. The cost is additive over time, i.e.
$$J = \sum_t g_t(x_t, u_t)$$

[a] i.e. $X_{t+1} \perp X_{1:t-1} | X_t$
[b] discrete time dynamical system representation. Notice f_t can change with time

In this context a powerfull result is given by **Bellman's principle**. Considering the following setup:

$$\begin{aligned} \min_{\mathbf{u},\mathbf{x}} \quad & \sum_{t=0}^{T} g_t(x_t, u_t) \\ \text{subject to} \quad & x_{t+1} = f_t(x_t, u_t) \\ & x_0 = X_0 \\ & x_t \in \mathcal{X}_t \ \forall t \\ & u_t \in \mathcal{U}_t \ \forall t. \end{aligned}$$

a very powerfull result on which relies DP is **Bellman's optimality principle**.

Bellman's optimality principle

An optimal policy has the property that whatever the initial state and the initial decisions are, the remaining decisions must consititute an optimal policy with regard to the state resulting from the first decision.

In a intuitive way we can say that any tail of an optimal trajectory is optimal too.

$$\begin{aligned} V_0(X_0) &= \min_{u_0} \left\{ g_0(X_0, u_0) + \min_{\substack{x_1,\dots,x_T \\ u_1,\dots,u_T}} \sum_{t=1}^{T} g_t(x_t, u_t) \;\text{ s.t. } \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_1 = f_0(X_0, u_0) \end{cases} \right\} \\ &= \min_{u_0} \{ g_0(X_0, u_0) + V_1(x_1) \} \\ &= \min_{u_0} \{ g_0(X_0, u_0) + V_1(f_0(x_0, u_0)) \} \end{aligned}$$

We can see that the problems are nested, therefore we need to proceed in a **backward** way, starting from the last time step and going backward in time. In particular we need to proceed in the following order

1. solve the subproblem parametrized by x_1;
2. compute the value function V_1;
3. solve the outer problem
$$\min_{u_0} \{ g(X_0, u_0) + V_1(f_0(X_0, u_0)) \}$$

1.2 Dynamic Programming

We can now write the general DP algorithm, which is a backward procedure to compute the value function and the optimal policy. We start from the last time step and we proceed backward in time until we reach the initial time step.
At the last stage (T) the subproblem is trivial, since there are no more decisions to be taken, therefore we have

$$V_T(x_T) = g_T(x_T)$$

At time $T-1$ we compute the value function as

$$\min_{u_{T-1}} \{ g_{T-1}(x_{T-1}, u_{T-1}) + V_T(f_{T-1}(x_{T-1}, u_{T-1})) \}$$

In particular notice that in the formulation above we know the value of $V_T(x_T)$, therefore we can compute the value function at time $T-1$ and so on until we reach the initial time step. So at this moment we have an optimal policy for the time step $T-1$, that means that we know how to compute the optimal control action at time $T-1$ given the state at time $T-1$. We can repeat this procedure until we reach the initial time step, therefore we have an optimal policy for all time steps.
At time $T-2$ we compute $V_{T-2}(x_{T-2})$ as

$$\min_{u_{T-2}} \{ g_{T-2}(x_{T-2}, u_{T-2}) + V_{T-1}(f_{T-2}(x_{T-2}, u_{T-2})) \}$$

Notice that this value function $V_{T-2}(x_{T-2})$ is a function of the state x_{T-2}, therefore we have to compute it $\forall x_{T-2} \in \mathcal{X}_{T-2}$, which is generally a very hard task.

Problem Decomposition

We have seen that the optimal control problem is decomposed into **stage problems** that we can solve via **backward induction** :

$$V_t(x_t) = \min_{u_t}\{g_t(x_t, u_t) + V_{t+1}(f_t(x_t, u_t))\} \tag{2}$$

And $V_T(x_T) = g_T(x_T)$

A reasonable question at this point would be, in which cases is this stage problem formulation simple to solve ? We can list three scenarios:

- Decision space $\tilde{\mathcal{U}}$ is convex, or maybe finite.
- If g_t is convex in u_t.
- If V_{t+1} evaluated at the future step f_t is convex in u_t.

In practice, a very common setup in which the assumptions above are satisfied is when we have a **quadratic cost** g_t and a **linear dynamics** f_t, which is the so called **LQR problem**. Another way is also to use approximations of the value function, in which case maybe not all the assumptions above are satisfied, but we can still solve the stage problem.

1.3 Optimal LQR

The LQR problem is a special case of the optimal control problem, in which we have a linear dynamics and a quadratic cost. In particular we have:

- Markovian update and linear dynamics

$$x_{t+1} = A_t x_t + B_t u_t$$

- Quadratic cost function:

$$\sum_{t=0}^{T-1} \left(x_t^\top \, Q_t \, x_t + u_t^\top \, R_t \, u_t \right) + x_T^\top \, S \, x_T \quad \text{with } Q, S \succeq 0 \, , \, R \succ 0$$

- Value function (i.e. minimum cost to go starting from x at time t) :

$$V_t(x) = \min_{u_t, \ldots, u_{T-1}} \sum_{k=t}^{T-1} \left(x_k^\top \, Q_k \, x_k + u_k^\top \, R_k \, u_k \right) + x_T^\top \, S \, x_T$$

$$\begin{aligned} \text{subject to } & x_{k+1} = A_k x_k + B_k u_k \quad \forall k = t, \ldots, T-1 \\ & x_t = x \end{aligned}$$

We say that the optimal cost is $V_0(x_0)$.

1.4 Dynamic Programming for LQR

Let us now specialize the dynamic programming recursion to the Linear Quadratic Regulator (LQR) problem. Recall that the value function represents the optimal *cost-to-go*, i.e. the minimum cost achievable from a given state when acting optimally from that point onward.
A key observation in the LQR setting is that the terminal cost is quadratic in the state. Indeed, from the problem formulation we have

$$V_T(x) = x^\top S x,$$

which is a convex quadratic function since $S \succeq 0$.
The main idea of the LQR derivation is to show that the value function $V_t(x)$ is also quadratic and has the form of $V_t(x) = x^\top P_t\, x$ for some symmetric positive semidefinite matrix P_t.
We will then show that the matrices P_t can then be computed recursively by working *backward in time* starting from the terminal time T. Eventually the the optimal control input u_t will be computed as a solution of a convex quadratic optimization problem, which admits a closed-form solution.

Proof. To derive the recursion, consider the dynamic programming stage problem at time t where the value function is

$$V_t(x) = \min_u \Big(x^\top Q\, x + u^\top R\, u + V_{t+1}(Ax + Bu) \Big).$$

We now proceed by induction, assuming that $V_{t+1}(x) = x^\top P_{t+1} x$ for some positive semidefinite matrix P_{t+1}. Then $V_t(x)$ can be written as

$$\begin{aligned} V_t(x) &= x^\top Q x + \min_u \Big(u^\top R u + (Ax + Bu)^\top P_{t+1}(Ax + Bu) \Big) \\ &= x^\top Q x + \min_u \Big(u^\top R u + x^\top A^\top P_{t+1} A x + 2u^\top B^\top P_{t+1} A x + u^\top B^\top P_{t+1} B u \Big) \\ &= x^\top\, Q\, x + \min_u \Big(u^\top (R + B^\top P_{t+1} B) u + x^\top A^\top P_{t+1} A x + 2u^\top B^\top P_{t+1} A x \Big). \end{aligned}$$

We set the gradient of the expression inside the minimization with respect to u to zero:

$$(R + B^\top P_{t+1} B) u + B^\top P_{t+1} A x = 0$$

Yielding the optimal control input

$$\boxed{u^* = -(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x}$$

But now we need to check that $V_t(x)$ is indeed quadratic in X. That means $V_t(x) = x^\top P_t x$ for some P_t and we have to find how to compute P_t. We can do that by plugging the optimal control input u^* back into the expression for $V_t(x)$:

$$\begin{aligned} V_t(x) &= x^\top Q x + (u^*)^\top R u^* + (Ax + Bu^*)^\top P_{t+1}(Ax + Bu^*) \\ &= x^\top Q x + (u^*)^\top R u^* + x^\top A^\top P_{t+1} A x + 2(u^*)^\top B^\top P_{t+1} A x + (u^*)^\top B^\top P_{t+1} B u^* \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + (u^*)^\top (R + B^\top P_{t+1} B) u^* + 2(u^*)^\top B^\top P_{t+1} A x \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + (u^*)^\top \big((R + B^\top P_{t+1} B) u^* + 2B^\top P_{t+1} A x \big) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x \\ &\quad -\big((R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x \big)^\top \big(-(R + B^\top P_{t+1} B)(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x + 2B^\top P_{t+1} A x \big) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x - \big((R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x \big)^\top \big(B^\top P_{t+1} A x \big) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x - x^\top A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x \\ &= x^\top \Big(Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \Big) x \\ &= x^\top P_t x \end{aligned}$$

To complete the proof one should check that P_t is positive semidefinite. Here we homit the proof.

In cocnlusion we have been able to derive a closed-form expression for the optimal control input u^* for the LQR problem.Be aware that this solution is not valid anymore if the problem becomes constrained. □

We summerize here the main results:

Optimal LQR solution

Backward induction

$$u_t = \underbrace{-(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A}_{\Gamma_t} \, x_t$$

where

$$P_t = Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \quad \text{and } P_T = S$$

Forward integration from x_0:

$$x_{t+1} = Ax_t + Bu_t$$

So clearly one could compute this whole open-loop sequence $u_0, \dots, u_{T-1}$ in a offline computation. But what if the optimal control input $\mathbf{u}$ takes us to a trajectory different from the one we compouted ? In this case we can use the optimal policy Γ_t to compute the optimal control input at each time step, given the current state x_t. This is the main advantage of having a closed-form solution for the optimal control input, since we can use it in a feedback way.

We can do a quick comparison of the computational cost of using these equations in a feedback way vs computing the whole open-loop sequence. By fixing m as the size of the states and n as the size of the control input, we have the following computational costs:

- **Offline computation**
  - $n \times n$ matrix multiplications to compute P_t for $t = T-1, \dots, 0$
  - $m \times m$ matrix inversions to compute Γ_t and P_t for $t = T-1, \dots, 0$
  - T iterations to compute P_t and Γ_t for $t = T-1, \dots, 0$
- **Online computation**
  - Storage of T $m \times n$ matrices Γ_t for $t = T-1, \dots, 0$
  - $m \times n$ matrix multiplications to compute u_t at each time step, given x_t

1.4.1 Infinite-Horizon LQR

In many practical control applications the system is required to operate over a very long time horizon. Examples include regulation problems where the controller must stabilize a system indefinitely, or tracking tasks where disturbances may occur continuously.

For these reasons it is natural to study the limit case where the horizon tends to infinity. The resulting problem is known as the **infinite-horizon Linear Quadratic Regulator (LQR)**.

Consider the linear time-invariant system

$$x_{t+1} = Ax_t + Bu_t, \qquad x_0 \text{ given}$$

and the infinite-horizon quadratic cost

$$\min_{u_0, u_1, \dots} \sum_{t=0}^{\infty} \left(x_t^\top Q x_t + u_t^\top R u_t \right), \qquad Q \succeq 0, \; R \succ 0.$$

Compared to the finite-horizon case, the cost now accumulates indefinitely. Therefore a first important question concerns the **feasibility** of the problem.

Feasibility

If the pair (A, B) is **stabilizable**, then there exists a control input sequence that yields a finite value of the infinite-horizon cost.

Indeed, if the system is stabilizable there exists a feedback matrix K such that the closed-loop matrix $A + BK$ has all eigenvalues inside the unit circle. Consider the linear feedback policy

$$u_t = Kx_t.$$

The state then evolves according to

$$x_t = (A + BK)^t x_0.$$

Substituting this into the cost yields

$$V(x_0) = \sum_{t=0}^{\infty} \left(x_t^\top Q x_t + x_t^\top K^\top R K x_t \right).$$

Using the closed-loop dynamics we obtain

$$V(x_0) = x_0^\top \left(\sum_{t=0}^{\infty} (A + BK)^{t\top} (Q + K^\top R K)(A + BK)^t \right) x_0.$$

Since $A + BK$ is stable, the above summation behaves like a *matrix geometric series* and therefore converges. Consequently the cost is finite.

But how do we compute the optimal policy for the infinite-horizon LQR problem ? The answer is that we can still apply the dynamic programming recursion, but now the value function does not depend on time. The solution is given by the **Discrete Algebraic Riccati Equation (DARE)**, which is a fixed-point equation for the matrix P that defines the value function.

Algebraic Riccati Equation

The iteration

$$P_{t-1} = Q + A^\top P_t A - A^\top P_t B (R + B^\top P_t B)^{-1} B^\top P_t A \qquad P_0 = 0$$

converges, for $t \to -\infty$, to a unique positive definite solution $P_\infty \succ 0$ of the algebraic Riccati equation

$$P = Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A$$

We can show that then the optimal feedback control in the infinite-horizon case is given by

$$u_t = \underbrace{-(R + B^\top P B)^{-1} B^\top P A}_{F_\infty} x_t$$

and thus minimizes the infinite horizon cost and yields $V(X_0) = X_0^\top P X_0$.
We now show again a comparison of the computational cost of using the infinite-horizon solution in a feedback way vs computing the whole open-loop sequence. By fixing m as the size of the states and n as the size of the control input, we have the following computational costs:

- **Offline computation**

  - $n \times n$ matrix multiplications to compute P
  - $m \times m$ matrix inversions to compute F_∞
  - T iterations to compute P and F_∞ for $t = T-1, \dots, 0$
  - "Infinite iterations" to compute P for $t \to -\infty$[1]

- **Online computation**
  - Storage of a **single** $m \times n$ matrix F_∞. So less memory is required compared to the finite-horizon case, since we do not need to store T matrices Γ_t.
  - $m \times n$ matrix multiplications to compute u_t at each time step, given x_t

A natural question at this point is whether the optimal infinite-horizon feedback law is also *stabilizing*. Indeed, even if the infinite-horizon cost is well defined and the Riccati equation admits a solution, one would still like to know whether the resulting closed-loop dynamics

$$x_{t+1} = (A + BF_\infty)x_t$$

are asymptotically stable.

This is a meaningful question because the quadratic cost does *not* necessarily penalize all state directions. In particular, if some unstable state component is not "seen" by the cost, then the optimizer may have no incentive to stabilize it.

Illustrative example Consider the system

$$x_{t+1} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} x_t + u_t, \qquad Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad R = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Here both open-loop modes are unstable, since the eigenvalues of A are 2 and 3. However, the stage cost penalizes only the first state component x_1, while the second component x_2 does not appear in the state cost. Therefore, from the viewpoint of the objective function, an unstable behavior in the second state may go completely unnoticed unless it is indirectly forced to be controlled through the dynamics.

This already suggests an important principle:

> Unstable modes must be visible in the cost function, otherwise the optimal controller may fail to stabilize them.

To make this statement precise, write $Q = C^\top C$. Then the cost penalizes exactly the directions that are observable through the pair (C, A).

Stability of the optimal infinite-horizon LQR controller

Let $Q = C^\top C$. The optimal infinite-horizon feedback F_∞ stabilizes the system if and only if

1. the pair (A, B) is stabilizable;
2. the pair (C, A) has no unobservable unstable modes.

Equivalently, all unstable modes of A must be detectable through the state penalty Q.

The second condition is often stated by saying that the pair $(A, Q^{1/2})$ must be **detectable**. Detectability is weaker than observability: it does not require all modes to be observable, but only the unstable ones.

[1] P_∞ can be computed by solving the Algebraic Riccati Equation

Interpretation This theorem has a clear practical meaning. The matrix R penalizes the control effort, while Q determines which state directions the controller tries to regulate. If an unstable mode is not penalized by Q, then the optimization problem may prefer not to spend control effort on that mode, since doing so does not improve the objective. As a consequence, the optimal controller may exist but fail to stabilize the closed-loop system.

Hence, in practice:

- (A, B) stabilizable ensures that unstable modes can in principle be controlled;
- detectability of $(A, Q^{1/2})$ ensures that every unstable mode matters in the cost.

When both conditions hold, the stabilizing solution P of the algebraic Riccati equation yields the optimal feedback

$$u_t = F_\infty x_t, \qquad F_\infty = -(R + B^\top P B)^{-1} B^\top P A,$$

and the closed-loop matrix $A + BF_\infty$ has all eigenvalues inside the unit disk.

2 Model Predictive Control

2.1 Introduction

In the previous chapter we have seen an overview of the LQR problem and how to solve it using dynamic programming. In that formulation, we assumed that there were no constraints on the state and control input. However, in many real-world applications, constraints on the state and input must be explicitly taken into account.
In the presence of constraints, the problem cannot, in general, be solved in closed form via backward induction as in the unconstrained LQR case, since the value function is no longer quadratic.The resulting optimization problem becomes:

Constrained LQ finite horizon problem

$$\begin{aligned}\min_{u_0,\dots,u_{T-1},x_0,\dots,x_T} \quad & \sum_{t=0}^{T-1} \left(x_t^\top Q x_t + u_t^\top R u_t \right) + x_T^\top S x_T \\ \text{s.t.} \quad & x_{t+1} = A x_t + B u_t, \quad t = 0, \dots, T-1 \\ & x_0 = x_0 \\ & x_t \in \mathcal{X}_t \\ & u_t \in \mathcal{U}_t\end{aligned}$$

Usually, the constraint sets are convex, e.g., polytopes.

One could solve this optimization problem in a **one-shot** fashion (i.e., open-loop strategy), obtaining an optimal input sequence $u_0, \dots, u_{T-1}$ for the given initial state x_0, instead of a state-feedback policy as in the unconstrained case (where $u_t = \Gamma_t x_t$).
However, if disturbances or model mismatch cause the system to deviate from the predicted trajectory, the previously computed control sequence is no longer optimal. As a result, the system will follow a different trajectory than expected (see Figures 1 and 2).

To address this issue, the optimal control problem must be re-solved using the updated state information. Since we assume a **Markovian system**, the current state contains all the information needed to compute an optimal control sequence for the future. This implies that the optimization problem must be solved repeatedly, at each time step, within one sampling interval.

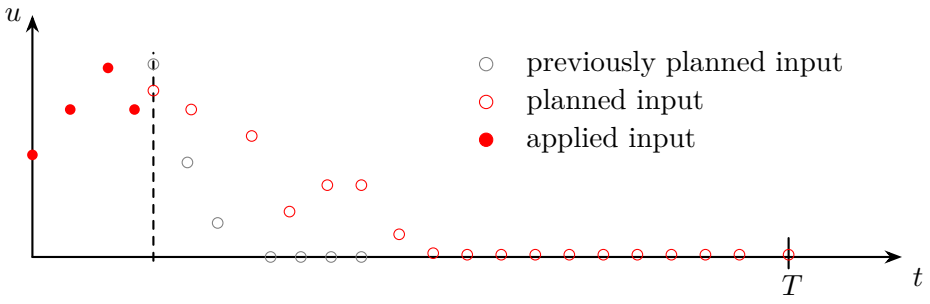

Figure 1: Receding-horizon update of the input sequence: previously planned inputs are shifted, a new optimal input sequence is computed based on the current state.

The idea that emerges from this discussion is to solve the optimization problem in a **receding horizon** fashion. At each time step, we solve a finite-horizon optimal control problem of length T, and apply only the first control input of the optimal sequence (see Figure 3).
At the next time step, the new state is measured, and the optimization problem is solved again

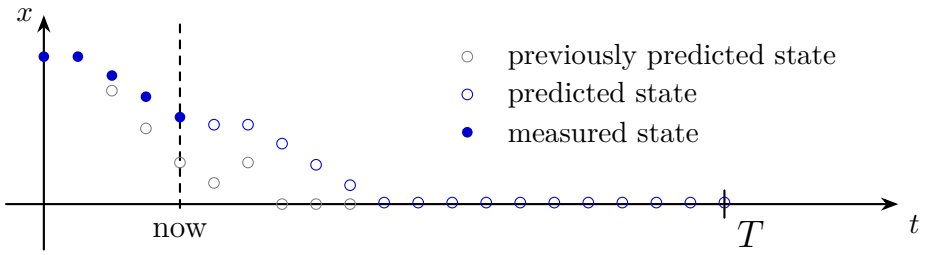

Figure 2: Receding-horizon update of the state trajectory: previously predicted states are corrected using the measured state at the current time, and a new optimal state trajectory is predicted over the horizon.

with the updated initial condition. If the system is time-invariant, the optimization problem remains the same at each time step, except for the initial state. The corresponding predicted state evolution over the horizon is illustrated in Figure 4.

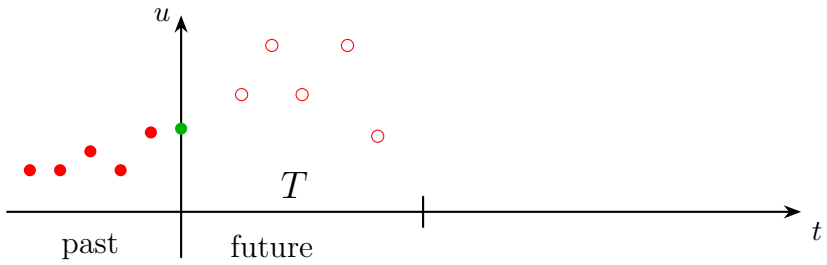

Figure 3: Finite-horizon input sequence in receding horizon control: at the current time, an optimal sequence of future inputs over a horizon of length T is computed, while past inputs are fixed.

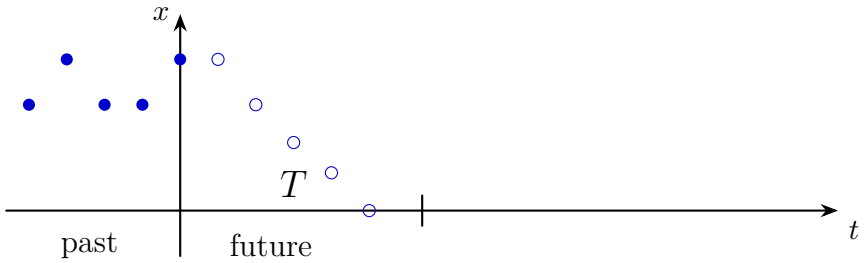

Figure 4: Finite-horizon state prediction in receding horizon control: starting from the current state, future states are predicted over a horizon of length T using the planned input sequence.

Thus the following scheme defines the receding horizon control strategy, also known as **Model Predictive Control (MPC)**:

1. At time t, measure the current state x_t.
2. Solve the finite-horizon optimal control problem with initial state x_t to obtain the optimal input sequence $u_t, u_{t+1}, \ldots, u_{t+T-1}$.
3. Apply the first control input u_t to the system.
4. Increment time $t \leftarrow t + 1$ and repeat from step 1.

The resulting **open-loop optimal control problem** corresponds to a *parametric optimization problem* parametrized in the initial state x.

Parametric optimization problem

$u^*(x)$ is determined by[a]

$$\begin{aligned} \min_{\mathbf{u},\mathbf{x}} \quad & \sum_{k=0}^{T-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) + x_T^\top S x_T \\ \text{s.t.} \quad & x_{k+1} = A x_k + B u_k \\ & x_0 = x \\ & x_k \in \mathcal{X}_k \\ & u_k \in \mathcal{U}_k \end{aligned}$$

[a] Notation: k is the index for the optimization problem, while t is the index for the time steps of the system.

The parametric nature of the problem immediately raises the question of whether the resulting control strategy is *feedforward* or *feedback*. Although the optimization problem computes an open-loop input sequence, the receding horizon implementation **introduces feedback implicitly**. Indeed, at each time step the optimization problem is solved using the current state x_t, and only the first input of the optimal sequence is applied. Therefore, the implemented control law can be written as

$$u_t = u^*(x_t),$$

which defines a **state-feedback control law**. One can have a visual intuition of this feedback structure by looking at the block diagram in Figure 5.

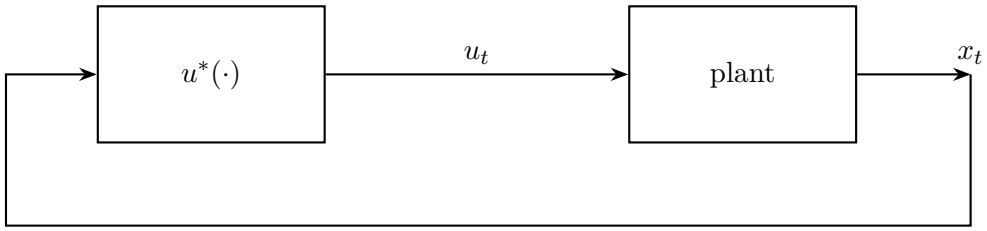

Figure 5: Block diagram of the MPC control scheme

We can establish some properties about this optimization problem:

- Under **continuity** assumptions on the cost and the dynamics, the minimum cost is a continuous function of the initial state x.
- Under **compactness** assumptions on the constraints, the parametric optimization problem admits a solution for all x in the feasible set.
- The map $x \mapsto u^*(x)$ is not necessarily continuous for a general cost function

In the case of linear dynamics, quadratic cost, and convex constraints, the optimization problem is a **convex quadratic program (QP)** parametrized in the initial state x. In this case, the map $x \mapsto u^*(x)$ is piecewise affine and continuous. However, in general, the control law induced by MPC can be nonlinear and non-smooth.

MPC control law

As a consequence, MPC can be interpreted as a **memory-less, nonlinear, time-invariant feedback control law**. The nonlinearity arises from the presence of constraints, even when the system dynamics are linear and the cost is quadratic.

A natural question is whether MPC can achieve *perfect tracking*, i.e., zero steady-state error. In general, this is not guaranteed without additional design choices (e.g., terminal ingredients or disturbance modeling), since the controller repeatedly solves a finite-horizon problem and does not explicitly enforce asymptotic properties beyond the prediction horizon. To better understand the relation between MPC and the finite-horizon optimal control problem, consider the following example:

Example

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \sum_{t=0}^{T-1} a^t |u_t| + |x_T|^2, \quad |a| < 1 \\ \text{s.t.} \quad & x_{t+1} = x_t + u_t. \end{aligned}$$

Two different questions can be asked:

1. What is the optimal open-loop solution $(\mathbf{x}^*, \mathbf{u}^*)$?
2. What is the MPC feedback law $x \mapsto u^*(x)$ obtained by solving the problem in a receding horizon fashion?

To solve this we observe that the cost is not quadratic and that the cost of applying inputs is discounted over time. Furthermore the state is penalized only at the end of the horizon.
Thus the optimal solution is to simply wait for the last time step and then apply $u_T = -x_T$, which yields a cost of $|x_0|^2$.
In contrast, the MPC feedback law is given by the control law $x \mapsto u^*(x) = 0$. This is because at each time step the optimization problem is solved with a shifted horizon, and the optimal input sequence is always to wait until the end of the horizon and then apply $u_{t+T} = -x_{t+T}$. Therefore, at each time step, the first input of the optimal sequence is zero.
A key observation is that the **optimal trajectory** associated with the finite-horizon problem is, in general, **never realized in closed loop**, even in the absence of disturbances or model mismatch. This is because at each time step the optimization problem is re-solved with a shifted horizon, leading to a different optimal sequence than the one computed previously. ■

Consider now the **unconstrained linear-quadratic** problem:

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \sum_{t=0}^{T-1} \left(x_t^\top Q x_t + u_t^\top R u_t \right) + x_T^\top S x_T, \quad Q, S \succeq 0,\ R \succ 0 \\ \text{s.t.} \quad & x_{t+1} = A x_t + B u_t. \end{aligned}$$

Finite-time LQR yields a time-varying linear feedback law

$$\begin{aligned} u_t &= \Gamma_t x_t \\ \Gamma_t &= -(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \end{aligned}$$

where the matrices P_t are computed backward in time. The control gain explicitly depends on time through P_{t+1}.
In contrast, **MPC** repeatedly solves the same finite-horizon problem at each time step. As a result, the applied control law is

$$u_t = \Gamma_0 x_t,$$

where Γ_0 is the gain associated with the first step of the horizon. Hence, MPC induces a **linear time-invariant** feedback law.
More explicitly, at time $t = 0$ both approaches yield the same control input:

$$u_0^{\text{MPC}} = -(R + B^\top P_1 B)^{-1} B^\top P_1 A x_0 = u_0^{\text{LQR}}.$$

However, at the next time step a discrepancy appears. Finite-time LQR applies

$$u_1^{\text{LQR}} = -(R + B^\top P_2 B)^{-1} B^\top P_2 A x_1,$$

while MPC recomputes the problem and applies again

$$u_1^{\text{MPC}} = -(R + B^\top P_1 B)^{-1} B^\top P_1 A x_1.$$

Therefore, even in the unconstrained case, **MPC does not reproduce the finite-horizon optimal control sequence beyond the first step**.

2.2 Implementation of MPC

The implementation of MPC requires solving an optimization problem at each time step. We show two main approaches to implement MPC:

1. Compute the **function** $u^*(\cdot)$ **offline**, and then evaluate it online at each time step.
   - It is extremely complex to solve with exception of very few cases
   - Abundant offline computational resources are required
   - Large memory is required to store the function $u^*(\cdot)$
2. Compute the **value** of the function $u^*(\cdot)$ **online** at each time step by solving the optimization problem.
   - It is often a tractable optimization problem.
   - It has limited time to perform computation online.

2.2.1 Explicit MPC

Explicit MPC corresponds to the **first implementation strategy**: instead of solving the optimization problem online at each sampling instant, we try to compute the feedback law $x \mapsto u^*(x)$ **offline**.
We have already seen one important case. In the **unconstrained linear MPC** case, the optimizer is simply the first element of the finite-horizon LQR solution. Therefore, the applied input can be written as

$$u_t = u^*(x_t) = \Gamma_0 x_t,$$

so the control law is linear.
The natural question is what happens when **constraints are introduced**. Consider the standard constrained linear MPC problem

Linear MPC with linear constraints

$$\begin{aligned}
\min_{\mathbf{u},\mathbf{x}} \quad & \sum_{k=0}^{T-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) + x_T^\top S x_T \\
\text{s.t.} \quad & x_{k+1} = A x_k + B u_k, \qquad k = 0, \dots, T-1, \\
& x_0 = x, \\
& x_k \in \mathcal{X}, \qquad \mathcal{X} = \{x \in \mathbb{R}^n : Cx \le d\}, \\
& u_k \in \mathcal{U}, \qquad \mathcal{U} = \{u \in \mathbb{R}^m : Eu \le f\},
\end{aligned}$$

with $Q, S \succeq 0, \qquad R \succ 0.$
This is the most common MPC formulation: linear dynamics, quadratic cost, and polyhedral state/input constraints.

This problem is a **convex quadratic program** parametrized by the initial state x. In general, the optimizer is no longer globally linear. However, due to the quadratic cost and linear constraints, it still has a very specific structure: the explicit MPC law turns out to be **piecewise affine**. The idea is best understood through a very simple example.

Example Consider a scalar integrator with horizon $T = 1$:

$$\begin{aligned} \min_{u_0, x_1} \quad & u_0^2 + x_1^2 \\ \text{s.t.} \quad & x_1 = x_0 + u_0, \\ & x_1 \leq 1. \end{aligned}$$

This example is useful because:

- it is a quadratic problem with a linear constraint,
- it is parametrized by the current state x_0,
- the equality constraint allows us to eliminate the variable x_1,
- the problem is convex, hence the KKT conditions are necessary and sufficient.

Using the model equation $x_1 = x_0 + u_0$, we can eliminate x_1 and rewrite the problem as

$$\min_{u_0} f(u_0) \qquad \text{s.t.} \qquad g(u_0) \leq 0,$$

where

$$\begin{aligned} f(u_0) &= u_0^2 + (x_0 + u_0)^2, \\ g(u_0) &= x_0 + u_0 - 1. \end{aligned}$$

The derivatives are

$$\begin{aligned} \nabla f(u_0) &= 4u_0 + 2x_0, \\ \nabla g(u_0) &= 1. \end{aligned}$$

KKT conditions for one inequality constraint

For a problem of the form

$$\min_{x} f(x) \qquad \text{s.t.} \qquad g(x) \leq 0,$$

the KKT conditions are

$$\begin{aligned} \nabla f(x) + \mu \nabla g(x) = 0, \qquad & \mu \geq 0, \\ g(x) \leq 0, & \\ \mu\, g(x) = 0. & \end{aligned}$$

The last relation is the *complementary slackness* condition.

Applying the KKT conditions to the problem gives

$$\begin{aligned} 4u_0 + 2x_0 + \mu = 0, \qquad & \mu \geq 0, \\ x_0 + u_0 - 1 \leq 0, & \\ \mu(x_0 + u_0 - 1) = 0. & \end{aligned}$$

The complementary slackness condition implies that two cases are possible.

Case 1: inactive constraint. Assume first that the inequality is inactive, so

$$\mu = 0.$$

Then stationarity becomes

$$4u_0 + 2x_0 = 0,$$

hence

$$u_0 = -\frac{x_0}{2}.$$

To check whether this candidate is feasible, we substitute it into the constraint:

$$x_0 + u_0 - 1 = x_0 - \frac{x_0}{2} - 1 = \frac{x_0}{2} - 1.$$

Therefore feasibility requires

$$\frac{x_0}{2} - 1 \leq 0 \qquad \Longleftrightarrow \qquad x_0 \leq 2.$$

So, for all initial states satisfying $x_0 \leq 2$, the optimal input is

$$u_0^* = -\frac{x_0}{2}.$$

Case 2: active constraint. Assume now that the inequality is active, so

$$x_0 + u_0 - 1 = 0.$$

Then

$$u_0 = 1 - x_0.$$

Substituting this into the stationarity condition,

$$4u_0 + 2x_0 + \mu = 0,$$

we obtain

$$\mu = -4u_0 - 2x_0 = -4(1 - x_0) - 2x_0 = 2x_0 - 4.$$

Since $\mu \geq 0$, this case is valid only if

$$2x_0 - 4 \geq 0 \qquad \Longleftrightarrow \qquad x_0 \geq 2.$$

Hence, for all $x_0 \geq 2$, the optimal input is

$$u_0^* = 1 - x_0.$$

Combining the two cases, the explicit solution is

Explicit solution of the toy MPC problem

$$u_0^*(x_0) = \begin{cases} -\dfrac{x_0}{2}, & x_0 \leq 2, \\ 1 - x_0, & x_0 \geq 2. \end{cases}$$

This example shows the main structural property of explicit MPC:

- the control law is **continuous**,
- the control law is **piecewise affine**,
- each affine expression is valid in a region where the same set of constraints is active.

The reason is the following. Once the active constraints are known, the KKT conditions reduce to a linear system in the decision variables and the multipliers. Solving that linear system gives an optimizer that depends *affinely* on the parameter x. When the active set changes, a different affine expression is obtained. Therefore, the state space is partitioned into regions, and on each region the controller is affine.

Although the explicit solution is conceptually appealing, it becomes difficult to use in large problems. In practice:

- there may be tens or hundreds of constraints,
- the number of regions may become very large,
- the offline computation required to determine all regions can be very long,
- even online, one must determine in which region the current state lies before evaluating the affine law.

As a consequence, explicit MPC is mainly **attractive when the system dimension is small and the sampling time is so short that solving a quadratic program online would be too expensive**. For larger systems, the most common approach is instead to solve the MPC optimization problem online at each sampling instant.

2.3 Stability

A useful way to think about MPC is as a *feedback design tool*. Once the prediction horizon T and the weights Q, R, S are chosen, the receding-horizon implementation induces a control law of the form

$$u_t = u^*(x_t).$$

Even though the optimization problem computes an open-loop input sequence, only the first input is applied and the problem is solved again at the next time step. Therefore, the implemented controller is a **static, time-invariant, nonlinear feedback law**. The nonlinearity does not necessarily come from the dynamics; it already appears because of the repeated constrained optimization.
From this perspective, the main question is no longer only how to solve the optimization problem, but **whether the resulting closed-loop system is stable**. There are two conceptually different ways to approach this question. The first is an *analysis* point of view: one tries to compute the control law $u^*(\cdot)$ explicitly and then study the stability of the closed-loop dynamics. In general, this is difficult, because the MPC law can be piecewise affine or more generally nonlinear and non-smooth. The second is a *design* point of view: instead of computing the feedback law explicitly, one looks for **conditions on the design parameters T, Q, R, S that guarantee closed-loop stability directly**. This is the more useful viewpoint in practice.

To study stability, the standard tool is Lyapunov analysis. Consider a discrete-time system

$$x_{t+1} = f(x_t),$$

and suppose that the origin $x = 0$ is an equilibrium, that is, $f(0) = 0$.

Stable equilibrium

The equilibrium $x = 0$ is said to be **stable** if

$$\forall \varepsilon > 0,\ \exists \delta > 0 \text{ such that } \|x_0\| < \delta \implies \|x_t\| < \varepsilon \quad \forall t \geq 0.$$

In words, if the initial condition is sufficiently close to the equilibrium, then the entire trajectory remains close to it for all future times.

Stability alone only guarantees that trajectories do not move far away from the equilibrium. A **stronger notion is asymptotic stability**, which additionally requires convergence to the equilibrium.

Asymptotically stable equilibrium

The equilibrium $x = 0$ is **asymptotically stable** if it is stable and

$$\lim_{t\to\infty} \|x_t\| = 0.$$

Hence, trajectories starting sufficiently close to the origin remain close to it and eventually converge to it.

The classical criterion to prove asymptotic stability is based on a Lyapunov function.

Lyapunov theorem for discrete-time systems

Assume there exists a real-valued function $W : \mathbb{R}^n \to \mathbb{R}$ such that

$$\begin{aligned} W(0) &= 0, \\ W(x) &> 0 \qquad \forall x \neq 0, \\ W(f(x)) &< W(x) \qquad \forall x \neq 0. \end{aligned}$$

Then the equilibrium $x = 0$ is asymptotically stable.

The idea is simple: W plays the role of an **energy-like quantity**. It is zero only at the equilibrium, positive everywhere else, and strictly decreases along the closed-loop trajectories. Therefore, the system is forced to move toward the origin.

This Lyapunov viewpoint explains why stability is relatively transparent for *infinite-horizon* MPC. Suppose that, at time t, the controller computes an input sequence that minimizes the infinite-horizon cost

$$V_t^\infty(x, \mathbf{u}) = \sum_{k=t}^{\infty} g_k(x_k, u_k),$$

subject to the system dynamics. If the minimum is attained, it is natural to define the optimal value function

$$W(x) = \min_{\mathbf{u}} V_t^\infty(x, \mathbf{u}).$$

Now comes the key observation. By Bellman's principle of optimality, if an input sequence is optimal from time t, then its tail is optimal from time $t + 1$ onward. In other words, after applying the first optimal input $u^*(x_t)$ and reaching the next state x_{t+1}, the remaining part of the optimal sequence is still optimal for the optimization problem starting at $t + 1$. As a consequence, along the closed-loop trajectory one obtains

$$W(x_{t+1}) = W(x_t) - g_t(x_t, u^*(x_t)).$$

Therefore, if the stage cost satisfies suitable positivity properties, for example if

$$g_t(x_t, u_t) > 0 \qquad \text{for all } x_t \neq 0,$$

then

$$W(x_{t+1}) < W(x_t) \qquad \text{for all } x_t \neq 0.$$

This means that the **optimal value function is a natural Lyapunov candidate** for the closed-loop system. Under reasonable assumptions, it follows that the origin is asymptotically

stable.

This argument is very elegant, but it also hides some subtleties. First, an infinite-horizon MPC problem is infinite-dimensional, so from a computational viewpoint it is not directly implementable as stated. Second, one is implicitly assuming that the global minimum exists and is attained. Third, the stage cost must satisfy suitable properties so that the value function is positive definite and decreases strictly along the closed loop. These requirements are analogous in spirit to the assumptions encountered in infinite-horizon LQR.

The situation changes significantly for *finite-horizon* MPC. In that case, the same Lyapunov argument does not apply automatically. At time t, the controller minimizes a cost over the interval $[t, t+T]$. At time $t+1$, however, the optimization problem is not simply the tail of the previous one: the first stage cost has disappeared, but a new term has entered at the end of the horizon. Therefore, the controller at time $t+1$ is minimizing a *different* objective. As a result, the optimizer changes, and the **finite-horizon value function is not automatically decreasing along the closed-loop trajectory**.
This is the fundamental reason why closed-loop stability is immediate for infinite-horizon optimal control, but not for finite-horizon receding-horizon MPC. **For finite-horizon MPC, stability must be enforced through additional design choices**. This will be the starting point for the next part of the discussion.

A classical way to **recover a Lyapunov argument** for finite-horizon MPC is to modify the optimization problem by imposing a **terminal constraint**. The simplest choice is the *zero terminal constraint*, namely

$$x_{t+T} = 0.$$

The corresponding finite-horizon optimal control problem becomes

Finite-horizon MPC with zero terminal constraint

Given the current state x_t, define

$$\begin{aligned} W_T(x_t) = \min_{\mathbf{x},\mathbf{u}} \quad & \sum_{k=t}^{t+T-1} g_k(x_k, u_k) \\ \text{s.t.} \quad & x_{k+1} = f(x_k, u_k), \qquad k = t, \dots, t+T-1, \\ & x_t \text{ given}, \\ & x_{t+T} = 0. \end{aligned}$$

The intuition is very simple. By forcing the predicted state to reach the origin at the end of the horizon, we make the finite-horizon problem look more similar to the infinite-horizon one. In particular, once the terminal state is the equilibrium, the tail of the optimal trajectory can be completed in a natural way by keeping the system at the origin with zero input, provided that

$$f(0,0) = 0.$$

This terminal condition restores the key mechanism that was missing in the plain finite-horizon case. Indeed, suppose that at time t the optimal solution of the MPC problem is

$$\{\tilde{u}_t, \tilde{u}_{t+1}, \dots, \tilde{u}_{t+T-1}\},$$

with associated state trajectory

$$\{\tilde{x}_t, \tilde{x}_{t+1}, \dots, \tilde{x}_{t+T}\},$$

where by construction

$$\tilde{x}_{t+T} = 0.$$

After applying the first control input $\tilde{u}_t$, the system reaches the state

$$x_{t+1} = \tilde{x}_{t+1}.$$

At time $t+1$, a natural feasible candidate input sequence is then obtained by taking the *tail* of the previous optimizer and appending the zero input:

$$\{\tilde{u}_{t+1}, \tilde{u}_{t+2}, \ldots, \tilde{u}_{t+T-1}, 0\}.$$

This is feasible because the previous optimal trajectory already satisfies $\tilde{x}_{t+T} = 0$, and the appended input $u = 0$ keeps the system at the equilibrium.
Therefore, the optimal cost at time $t+1$ can be upper bounded by the cost of this feasible candidate. This gives

$$\begin{aligned} W_T(x_{t+1}) &\leq \sum_{k=t+1}^{t+T} g_k(x_k, u_k) \\ &= \sum_{k=t}^{t+T-1} g_k(x_k, u_k) - g_t(x_t, u_t) + g_{t+T}(x_{t+T}, u_{t+T}). \end{aligned}$$

Since $x_{t+T} = 0$ and we append $u_{t+T} = 0$, the last term vanishes under the standard assumption that the stage cost is zero at the equilibrium:

$$g_{t+T}(0, 0) = 0.$$

Hence,

$$W_T(x_{t+1}) \leq W_T(x_t) - g_t(x_t, u_t).$$

If, in addition, the stage cost is positive away from the origin, then for every $x_t \neq 0$ we obtain

$$W_T(x_{t+1}) < W_T(x_t).$$

This shows that the optimal value function W_T decreases along the closed-loop trajectories, and therefore it becomes a Lyapunov candidate for the closed-loop system.

Asymptotic stability with zero terminal constraint

Under the usual assumptions of feasibility, attainment of the minimum, $f(0, 0) = 0$, and positivity of the stage cost, the equilibrium $x = 0$ is asymptotically stable for the closed-loop system generated by finite-horizon MPC with terminal constraint

$$x_{t+T} = 0.$$

The proof is essentially the shifted-sequence argument above. The terminal constraint ensures that the optimal trajectory at time t can be transformed into a feasible candidate at time $t+1$ simply by removing the first input and appending the zero input at the end. This gives a monotonic decrease of the value function, exactly as required by Lyapunov theory. We report here a more formal proof for completeness.

Proof. Let

$$W(x_t) = \min_{\mathbf{u},\mathbf{x}} \sum_{k=t}^{t+T-1} g(x_k, u_k)$$

subject to

$$x_{k+1} = f(x_k, u_k), \qquad x_t \text{ given}, \qquad x_{t+T} = 0.$$

We want to compare the optimal cost at time $t+1$ with the one at time t. By definition,

$$\begin{aligned} W(x_{t+1}) &= \min \sum_{k=t+1}^{t+T} g(x_k, u_k) \\ &= \min \left(\sum_{k=t}^{t+T-1} g(x_k, u_k) - g(x_t, u_t) + g(x_{t+T}, u_{t+T}) \right). \end{aligned}$$

By definition of the minimum, this quantity is less than or equal to the same expression evaluated at any feasible input sequence. In particular, let

$$\tilde{\mathbf{u}} = \arg\min_{\mathbf{u},\mathbf{x}} \sum_{k=t}^{t+T-1} g(x_k, u_k)$$

be the optimizer of the MPC problem at time t, with associated state trajectory

$$\tilde{x}_{t+1}, \tilde{x}_{t+2}, \ldots, \tilde{x}_{t+T}, \qquad \tilde{x}_{t+T} = 0.$$

Now construct a feasible candidate input sequence for the problem at time $t+1$ by taking the tail of the previous optimal sequence and appending the input

$$u_{t+T} = 0.$$

Since $\tilde{x}_{t+T} = 0$ and $f(0,0) = 0$, this implies

$$x_{t+T+1} = 0,$$

so the terminal constraint is satisfied also at time $t+1$.
Evaluating the cost at this feasible candidate yields

$$\begin{aligned} W(x_{t+1}) &\le \sum_{k=t+1}^{t+T} g(\tilde{x}_k, \tilde{u}_k) \\ &= \sum_{k=t}^{t+T-1} g(\tilde{x}_k, \tilde{u}_k) - g(\tilde{x}_t, \tilde{u}_t) + g(\tilde{x}_{t+T}, u_{t+T}). \end{aligned}$$

Because $\tilde{x}_{t+T} = 0$ and $u_{t+T} = 0$, and assuming $g(0,0) = 0$, we obtain

$$\begin{aligned} W(x_{t+1}) &\le \sum_{k=t}^{t+T-1} g(\tilde{x}_k, \tilde{u}_k) - g(\tilde{x}_t, \tilde{u}_t) \\ &= W(x_t) - g(\tilde{x}_t, \tilde{u}_t) \\ &\le W(x_t). \end{aligned}$$

Hence the optimal value function is non-increasing along the closed-loop trajectories. If moreover

$$g(x, u) > 0 \qquad \forall x \neq 0,$$

then

$$W(x_{t+1}) < W(x_t) \qquad \forall x_t \neq 0,$$

so W is a Lyapunov function for the closed-loop system. Therefore, the equilibrium $x = 0$ is asymptotically stable. □

The zero terminal constraint is conceptually very appealing because it gives a clean and direct stability argument. However, it can also be **quite restrictive**. Requiring the predicted state to reach the origin exactly in T steps may be impossible for some initial conditions, especially when the horizon is short or constraints are tight. In other words, **stability can be obtained, but possibly at the price of a reduced feasible set**.

The zero terminal constraint is only one possible way to guarantee stability of finite-horizon MPC. More generally, several families of sufficient conditions can be used:

- choosing the prediction horizon T large enough;
- imposing a zero terminal constraint;
- imposing a suitable **terminal region** constraint for the state;
- combining terminal ingredients, such as terminal constraints and terminal penalties.

The common idea behind all these approaches is to recover, in one form or another, a Lyapunov-like decrease of the optimal cost along the closed-loop trajectories. The precise technical construction changes from one method to another, but the objective is always the same: to ensure that the receding-horizon controller does not merely optimize over the next T steps, but also induces a long-term stabilizing behavior.

When discussing MPC stability results, a few important subtleties should always be kept in mind.
First, throughout the analysis we implicitly assume that the MPC optimization problem is **feasible**. This is a nontrivial assumption. In particular, with a zero terminal constraint the problem may easily become infeasible, because not every state can be driven exactly to the origin in T steps while satisfying the constraints.
Second, **future feasibility may depend on the control action selected at the current time**. Thus, it is not enough to have feasibility only at the initial time: one would like feasibility to be preserved as the horizon recedes. This issue is often referred to as *recursive feasibility*, and it is closely connected to the choice of terminal ingredients.
Third, all these arguments assume that the **global minimum** of the optimization problem is found. This is automatic in the convex quadratic MPC problems discussed so far, but it becomes more delicate in more general nonlinear or nonconvex settings.
For these reasons, stability of MPC is not only a matter of optimization, but also of careful controller design. The horizon length, the stage cost, and the terminal ingredients must be selected jointly so that the resulting receding-horizon law is both feasible and stabilizing.

2.4 Disturbance rejection

So far, the MPC design has been discussed mainly in nominal conditions, that is, under the assumption that the model is exact and that no unknown perturbation acts on the system. In practice, however, disturbances are unavoidable. Depending on the application, the controller may be required to achieve different levels of robustness.

Disturbance rejection specifications

In an MPC problem, robustness against disturbances may refer to different objectives:

1. **rejection at steady state**, that is, **zero offset**;
2. **rejection during the transient**, namely a good response while the disturbance is acting and being compensated;
3. **guaranteed constraint satisfaction** despite disturbances.

These are related but distinct goals. A controller may remove the steady-state error and still react poorly during the transient. Conversely, a controller may have a reasonable transient behavior but fail to guarantee that state and input constraints are respected under uncertainty. In this subsection we focus on the first two aspects: zero steady-state offset and transient disturbance rejection.

2.4.1 Zero steady-state offset and incremental MPC

It is useful to remember that MPC is, in the end, a nonlinear static feedback law of the form

$$u_t = u^*(x_t).$$

This observation immediately suggests a **possible weakness**. If the plant requires a *nonzero* constant input in order to keep the state at the desired equilibrium, then a standard MPC formulation centered at the origin may fail to achieve

$$x_t \to 0.$$

In other words, a constant disturbance or a model mismatch may produce a **steady-state offset**. This issue is especially visible in the presence of:

- input disturbances,
- process noise,
- multiplicative uncertainty.

Moreover, trying to compensate this effect by choosing very small input weights may lead to poor numerical conditioning of the optimization problem. A standard remedy is to reformulate the controller in **incremental form**. Instead of using the input u_t directly as the manipulated variable, one introduces the input increment

$$\Delta u_t$$

and imposes the update law

$$u_{t+1} = u_t + \Delta u_t.$$

This is equivalent to augmenting the plant with an input integrator. If the original nominal model is

$$x_{t+1} = Ax_t + Bu_t,$$

then the augmented model becomes

$$\begin{bmatrix} x_{t+1} \\ u_{t+1} \end{bmatrix} = \begin{bmatrix} A & B \\ 0 & I \end{bmatrix} \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} 0 \\ I \end{bmatrix} \Delta u_t.$$

The optimization problem is then written in terms of Δu_t rather than u_t. This has two important consequences. First, the controller acquires an **integral action**, which is exactly what is needed to remove steady-state offsets caused by constant disturbances. Second, the input penalty in the stage cost now penalizes changes in the control signal, that is, it penalizes Δu_t rather than the absolute value of u_t. As a consequence, the controller naturally prefers smooth input profiles. Hence, the incremental formulation can be interpreted as an **integral nonlinear feedback law**: the feedback structure is still given by MPC, but the plant seen by the optimizer includes an integrator on the input channel. This is the basic mechanism used in MPC to obtain zero steady-state offset against constant disturbances.

2.4.2 Disturbance rejection during transients

Removing the steady-state error is often not enough. In many applications, the controller should also react well *while* the disturbance is affecting the system. A common strategy in MPC is based on two steps:

1. estimate the disturbance using the available measurements and the plant model;
2. correct the steady-state target using the estimated disturbance.

The idea is the following. The nominal prediction model alone cannot explain the measured plant behavior when a disturbance is present. Therefore, an additional disturbance variable is introduced and estimated online. The estimate is then used to shift the equilibrium that the MPC controller should track.
Unless better prior information is available, a very common choice is to model the disturbance as **constant**:

$$d_{t+1} = d_t, \qquad d_t \in \mathbb{R}^{n_d}, \qquad n_d \leq n.$$

This means that the unknown perturbation is assumed to vary slowly compared with the system dynamics, so that over the time scale of interest it can be regarded as approximately constant.

Step 1: Disturbance estimation Assume that the plant is described by the model

$$x_{t+1} = Ax_t + Bu_t + B_d d_t,$$

where d_t is an unknown disturbance entering through the matrix B_d. Since d_t is not measured, it must be estimated from the observed evolution of the state.
A simple and effective choice is a Luenberger-type disturbance observer. One first predicts the next state using the current state, the applied input, and the current disturbance estimate:

$$\hat{x}_{t+1} = Ax_t + Bu_t + B_d \hat{d}_t.$$

Then the disturbance estimate is corrected using the prediction error:

$$\hat{d}_{t+1} = \hat{d}_t + L(x_{t+1} - \hat{x}_{t+1}),$$

where L is an observer gain chosen so that

$$I - LB_d$$

is stable.
The logic is very intuitive. If the measured next state differs from the nominal prediction, the mismatch is interpreted as the effect of an unmodeled disturbance. The observer then updates $\hat{d}_t$ in order to make future predictions consistent with the measurements. In this way, the estimate gradually converges to the constant disturbance that is acting on the plant.

Step 2: correction of the steady-state target Once the disturbance has been estimated, the next step is to compute which steady state should be tracked by the controller. If the disturbance were absent, the desired equilibrium for a regulation problem would usually be the origin. However, when a constant disturbance is present, the true equilibrium generally shifts. Therefore, instead of forcing MPC to regulate the state to zero, we first compute a **disturbance-consistent steady-state target** (x_s, u_s).
Under the constant disturbance model, the current estimate $\hat{d}_t$ is our best approximation of the steady-state disturbance. The corresponding steady-state target can be obtained by solving

Steady-state target selection

$$\begin{aligned} \min_{x_s,u_s} \quad & \|x_s\|_{Q_s}^2 + \|u_s\|_{R_s}^2 \\ \text{s.t.} \quad & \begin{bmatrix} I - A & -B \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = B_d \hat{d}_t, \\ & x_s \in \mathcal{X}, \\ & u_s \in \mathcal{U}. \end{aligned}$$

The equality constraint is simply the steady-state balance equation for the disturbed system. Indeed, at equilibrium we must have

$$x_s = Ax_s + Bu_s + B_d \hat{d}_t,$$

which is equivalent to the constraint above. The optimization criterion is used to select, among all admissible equilibria compatible with the disturbance estimate, the one that is preferred according to the weights Q_s and R_s.
Once (x_s, u_s) has been computed, the MPC controller is no longer asked to regulate the plant to the nominal origin. Instead, it regulates the *deviations*

$$\tilde{x}_t = x_t - x_s, \qquad \tilde{u}_t = u_t - u_s$$

toward zero. In this way, the controller tracks the correct equilibrium associated with the current disturbance estimate. As the estimate $\hat{d}_t$ evolves, the target (x_s, u_s) is updated accordingly. This is the basic mechanism that improves disturbance rejection during transients.

2.4.3 Advanced disturbance models

The constant-disturbance model is often a good default choice, but it is not the only possibility. A more general principle is summarized by the following statement.

Good regulator theorem

"Every good regulator of a system must be a model of that system."

The message is that a controller can reject well only those classes of disturbances that are represented, explicitly or implicitly, in its internal model. Therefore, if better prior knowledge about the disturbance is available, that knowledge should be used.

For instance, the disturbance may be:

- constant,
- ramp-like,
- periodic.

This is precisely the viewpoint of the **internal model principle**: the closed-loop system can reject disturbances belonging to the class that has been modeled in the controller design.
If the disturbance has its own linear dynamics

$$d_{t+1} = A_d d_t,$$

then the plant can be augmented as

$$\begin{bmatrix} x_{t+1} \\ \hat{d}_{t+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & A_d \end{bmatrix} \begin{bmatrix} x_t \\ \hat{d}_t \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_t.$$

In this form, the disturbance is treated as an additional dynamical state. This model can then be used in two ways:

- inside the disturbance estimator;
- directly inside the MPC prediction model, instead of merely shifting the steady-state target.

This framework includes the constant-disturbance model as a special case, obtained by taking

$$A_d = I.$$

More generally, the choice of A_d determines which class of disturbances the controller is able to reject. In the spirit of the internal model principle, one integrator is associated with rejection of step-like disturbances, while more complex models are needed for ramps or periodic signals.

2.4.4 Robust satisfaction of constraints

The disturbance-rejection mechanisms discussed so far are mainly concerned with improving tracking and reducing steady-state offsets. However, in constrained MPC there is an additional, and often more critical, question:

Can we guarantee that the state and input constraints are satisfied despite disturbances?

This question is central in safety-critical applications. For instance, in an irrigation system, the controller may decide how much water to apply to a field based on a prediction of future weather conditions. The relevant disturbances include temperature, solar radiation, rainfall, and evapotranspiration. The constraints express that the soil moisture should not be too low (under-watering) and not too high (over-watering). A nominal MPC controller may perform well for the predicted weather, but it may violate the constraints if the actual rainfall or temperature differs from the forecast. Robust MPC aims to explicitly account for this uncertainty.

There are several possible meanings of "constraint satisfaction under uncertainty". The most important ones are the following.

Different robustness specifications

Consider a disturbed system and constraints of the form

$$x_k \in \mathcal{X}, \qquad u_k \in \mathcal{U}.$$

Depending on the available information on the disturbance, one may require:

1. **Guaranteed constraint satisfaction**

   $$x_k(w) \in \mathcal{X}, \qquad u_k(w) \in \mathcal{U} \qquad \forall w \in \mathcal{W}.$$

   The constraints must hold for every admissible disturbance realization.

2. **Constraint satisfaction with high probability**

   $$\mathbb{P}(x_k \in \mathcal{X},\ u_k \in \mathcal{U}) \geq 1 - \varepsilon.$$

   Small probabilities of violation are allowed.

3. **Scenario or sample-based satisfaction**

   $$x_k^{(s)} \in \mathcal{X}, \qquad u_k^{(s)} \in \mathcal{U}, \qquad s = 1, \ldots, N,$$

   where the constraints are imposed only on a finite number of disturbance samples.

4. **Bounded expected violation**

$$\mathbb{E}\left[\max\{0, h(x_k, u_k)\}\right] \leq \eta,$$

where the constraint is written as $h(x, u) \leq 0$.

5. **Risk-sensitive satisfaction, for instance through CVaR**

$$\mathrm{CVaR}_{1-\varepsilon}\left(h(x_k, u_k)\right) \leq 0.$$

This controls not only the probability of violation, but also the size of the violation in the tail of the distribution.

The strongest requirement is the first one: constraints must hold for all admissible disturbances. This leads to **robust MPC**. The other formulations are less conservative and are useful when a probability distribution of the disturbance is available, or when occasional small violations are acceptable.

2.4.5 Prior information on disturbances

We now assume that some prior information on the disturbance is available. Consider the discrete-time system

$$x_{t+1} = f(x_t, u_t, w_t),$$

where w_t is an unknown disturbance. In the linear additive case this becomes

$$x_{t+1} = Ax_t + Bu_t + Dw_t.$$

Different types of prior information can be used.

Finite disturbance set. One possibility is that the disturbance can take only finitely many values:

$$w_t \in \mathcal{D} := \{w^0, w^1, \ldots, w^p\}.$$

This situation arises, for example, when the uncertainty is represented by a finite ensemble of forecast scenarios.

Convex disturbance set. If the finite set is interpreted as a set of extreme cases, one can define the admissible disturbance set as the convex hull

$$w_t \in \mathrm{co}\{w^0, w^1, \ldots, w^p\}.$$

For linear dynamics and convex constraints, this is particularly convenient. Since the predicted state depends affinely on the disturbance, and linear inequalities are convex constraints, it is often enough to check the constraints on the extreme disturbance scenarios.

Probabilistic disturbance model. If a distribution is available, one can write

$$w_t \sim \mathbb{W},$$

where $\mathbb{W}$ denotes the probability law of the disturbance. In this case the controller may optimize expected performance, impose chance constraints, or use risk measures such as CVaR.

A disturbance sequence over a horizon T is denoted by

$$\mathbf{w} = (w_0, w_1, \ldots, w_{T-1}).$$

If the disturbances are selected from a finite set, then an ensemble of possible disturbance sequences is a set

$$\mathbb{W}_T \subseteq \mathcal{D}^T.$$

Each element $\mathbf{w} \in \mathbb{W}_T$ represents one possible future scenario.

2.4.6 Open-loop robust MPC

Let us first consider the most direct robust MPC formulation. We use the linear additive model

$$x_{k+1} = Ax_k + Bu_k + Dw_k.$$

At the current state x, we predict one state trajectory for each possible disturbance scenario. The important point is that the input sequence

$$u_0, u_1, \ldots, u_{T-1}$$

is the same for all disturbance scenarios. Hence, inside the optimization problem the future inputs are open-loop: they are planned now and are not allowed to depend on which disturbance will actually occur in the future.

Let $\bar{\mathbf{w}}$ be a nominal disturbance sequence, for example the zero disturbance or the mean forecast. A common robust MPC formulation is the following.

Open-loop robust MPC with nominal cost

Given the current state x, solve

$$\begin{aligned}
\min_{\{u_k\},\{x_k^{\mathbf{w}}\}} \quad & \sum_{k=0}^{T-1} g_k\left(x_k^{\bar{\mathbf{w}}}, u_k\right) + g_T\left(x_T^{\bar{\mathbf{w}}}\right) \\
\text{s.t.} \quad & x_{k+1}^{\mathbf{w}} = Ax_k^{\mathbf{w}} + Bu_k + Dw_k, \qquad k = 0, \ldots, T-1, \quad \mathbf{w} \in \mathbb{W}_T, \\
& x_0^{\mathbf{w}} = x, \qquad \mathbf{w} \in \mathbb{W}_T, \\
& x_k^{\mathbf{w}} \in \mathcal{X}, \qquad k = 0, \ldots, T, \quad \mathbf{w} \in \mathbb{W}_T, \\
& u_k \in \mathcal{U}, \qquad k = 0, \ldots, T-1.
\end{aligned}$$

The MPC input applied to the plant is the first element of the optimizer:

$$u_t = u_0^*(x).$$

This formulation separates performance and safety. The objective may be evaluated only on the nominal trajectory, while the constraints are imposed on all admissible disturbance scenarios. In this way, the controller tries to behave well for the nominal forecast, but it keeps all possible disturbed trajectories inside the admissible set.

The same idea can be made more conservative by optimizing the worst-case cost:

$$\min_{\{u_k\}} \max_{\mathbf{w} \in \mathbb{W}_T} \left(\sum_{k=0}^{T-1} g_k(x_k^{\mathbf{w}}, u_k) + g_T(x_T^{\mathbf{w}}) \right),$$

subject again to robust constraints. This protects also the performance against the worst disturbance, not only the constraints.

The advantage of open-loop robust MPC is that it is conceptually simple. The disadvantage is that it can be extremely conservative. The reason is that the same future input sequence must work for every possible future disturbance realization.

2.4.7 Why open-loop robust MPC can be too conservative

The conservatism of open-loop robust MPC can be seen from a very simple scalar example:

$$x_{t+1} = x_t + u_t + w_t, \qquad |w_t| \le 0.5.$$

Consider the stage cost

$$g(x, u) = x^2 + u^2,$$

the state constraint

$$|x_t| \le 1,$$

and no input constraint. We take a finite prediction horizon and ask that the predicted state constraint be satisfied for all admissible disturbances.

By iterating the dynamics, the state at the end of the prediction horizon can be written as

$$x_5 = x_0 + \sum_{k=1}^{4} u_k + \sum_{k=1}^{4} w_k.$$

Therefore, the terminal constraint $|x_5| \le 1$ requires

$$-1 \le x_0 + \sum_{k=1}^{4} u_k + \sum_{k=1}^{4} w_k \le 1.$$

Now consider two extreme disturbance realizations.

All disturbances positive. If

$$w_k = 0.5, \qquad k = 1, \dots, 4,$$

then

$$\sum_{k=1}^{4} w_k = 2.$$

The terminal constraint implies

$$-1 \le x_0 + \sum_{k=1}^{4} u_k + 2 \le 1,$$

or equivalently

$$-x_0 - 3 \le \sum_{k=1}^{4} u_k \le -x_0 - 1.$$

All disturbances negative. If

$$w_k = -0.5, \qquad k = 1, \dots, 4,$$

then

$$\sum_{k=1}^{4} w_k = -2.$$

The terminal constraint implies

$$-1 \le x_0 + \sum_{k=1}^{4} u_k - 2 \le 1,$$

or equivalently

$$-x_0 + 1 \leq \sum_{k=1}^{4} u_k \leq -x_0 + 3.$$

Thus, the same open-loop input sequence must satisfy

$$-x_0 - 3 \leq \sum_{k=1}^{4} u_k \leq -x_0 - 1$$

and

$$-x_0 + 1 \leq \sum_{k=1}^{4} u_k \leq -x_0 + 3$$

at the same time. These two intervals are disjoint. Hence, no open-loop input sequence can guarantee the terminal state constraint for all admissible disturbance realizations.

This conclusion is surprising at first, because a very simple feedback controller is feasible. Indeed, consider

$$u_t = -x_t.$$

Then the closed-loop dynamics become

$$x_{t+1} = w_t.$$

Since $|w_t| \leq 0.5$, it follows that

$$|x_{t+1}| \leq 0.5 < 1.$$

Therefore, the state constraint is satisfied for every admissible disturbance.

The lesson is the following.

Open-loop robust feasibility versus feedback feasibility

An open-loop robust MPC problem may be infeasible even when a feasible robust feedback controller exists.
The issue is not the lack of a stabilizing control law. The issue is that the robust optimization problem forces one single future input sequence to work for all possible future disturbance realizations.

This also clarifies a subtle point about MPC. MPC is a feedback controller because at every sampling time it measures the current state and recomputes the first input:

$$u_t = u^*(x_t).$$

However, the optimization problem solved at time t usually optimizes over an open-loop sequence

$$u_0, \ldots, u_{T-1}.$$

Thus, within the predicted future, the inputs are not allowed to adapt to future measurements. Robustness against all disturbance sequences can then become overly conservative.

2.4.8 Robust MPC over policies

The natural way to reduce this conservatism is to optimize not over input sequences, but over input policies:

$$u_k = \pi_k(x_k), \qquad k = 0, \ldots, T-1.$$

The corresponding robust optimal control problem is

$$\begin{aligned}\min_{\pi_0,\dots,\pi_{T-1}} \quad & \mathcal{J}(\pi_0,\dots,\pi_{T-1};x)\\ \text{s.t.} \quad & x_{k+1}^{\mathbf{w}} = Ax_k^{\mathbf{w}} + B\pi_k\left(x_k^{\mathbf{w}}\right) + Dw_k, \qquad \mathbf{w}\in\mathbb{W}_T,\\ & x_0^{\mathbf{w}} = x, \qquad \mathbf{w}\in\mathbb{W}_T,\\ & x_k^{\mathbf{w}}\in\mathcal{X}, \qquad k=0,\dots,T, \quad \mathbf{w}\in\mathbb{W}_T,\\ & \pi_k\left(x_k^{\mathbf{w}}\right)\in\mathcal{U}, \qquad k=0,\dots,T-1, \quad \mathbf{w}\in\mathbb{W}_T.\end{aligned}$$

This formulation is much less conservative, because different future states may lead to different future inputs. In the scalar example above, the policy

$$\pi(x) = -x$$

is feasible, whereas no open-loop sequence is feasible.

For finite disturbance sets, the policy optimization viewpoint can be represented through a **scenario tree**. Each node corresponds to a possible state reached after a particular disturbance history. The control action may depend on the node, hence on the information available up to that time. However, the controller must satisfy a non-anticipativity condition: two scenarios that have the same disturbance history up to time k must use the same input at time k.

The main drawback is computational complexity. If there are $p+1$ possible disturbance values at each time step, then the number of possible scenarios over a horizon T grows like

$$(p+1)^T.$$

Therefore, exact robust MPC over policies quickly becomes intractable.

2.4.9 Nested feedback for robust MPC

A practical compromise is to keep the robust MPC problem relatively simple, but to add a pre-designed feedback controller inside the prediction model. This idea is often described as wrapping the plant around a stabilizing controller.

Consider again the linear additive system

$$x_{t+1} = Ax_t + Bu_t + Dw_t.$$

Instead of optimizing directly over u_t, we parametrize the input as

$$u_t = v_t + Lx_t,$$

where L is a fixed feedback gain and v_t is the new decision variable optimized by MPC. Substituting into the dynamics gives

$$x_{t+1} = Ax_t + B(v_t + Lx_t) + Dw_t = (A+BL)x_t + Bv_t + Dw_t.$$

Thus the robust MPC problem is solved for the pre-stabilized system

$$x_{t+1} = (A+BL)x_t + Bv_t + Dw_t.$$

Nested robust MPC parametrization

Choose a feedback gain L such that $A+BL$ is stable. For discrete-time systems this means that $A+BL$ is Schur, i.e., all eigenvalues lie strictly inside the unit disk. For continuous-time systems the analogous condition is that $A+BL$ is Hurwitz.

Then solve a robust MPC problem in the variables $v_0, \dots, v_{T-1}$:

$$\begin{aligned} \min_{\{v_k\},\{x_k^{\mathbf{w}}\}} \quad & \sum_{k=0}^{T-1} g_k\left(x_k^{\bar{\mathbf{w}}}, v_k + Lx_k^{\bar{\mathbf{w}}}\right) + g_T\left(x_T^{\bar{\mathbf{w}}}\right) \\ \text{s.t.} \quad & x_{k+1}^{\mathbf{w}} = (A+BL)x_k^{\mathbf{w}} + Bv_k + Dw_k, \qquad \mathbf{w} \in \mathbb{W}_T, \\ & x_0^{\mathbf{w}} = x, \qquad \mathbf{w} \in \mathbb{W}_T, \\ & x_k^{\mathbf{w}} \in \mathcal{X}, \qquad k = 0, \dots, T, \quad \mathbf{w} \in \mathbb{W}_T, \\ & v_k + Lx_k^{\mathbf{w}} \in \mathcal{U}, \qquad k = 0, \dots, T-1, \quad \mathbf{w} \in \mathbb{W}_T. \end{aligned}$$

The control input applied to the plant is

$$u_t = v_0^*(x) + Lx_t.$$

The role of the two terms is different:

- The sequence $v_0, \dots, v_{T-1}$ is optimized by MPC and steers the nominal trajectory.
- The feedback term Lx_t reacts immediately to the current state and attenuates the effect of disturbances.
- The gain L is chosen offline using any suitable method, for example pole placement, LQR, or another robust control design.

This architecture reduces conservatism because disturbances are no longer left to accumulate open-loop over the prediction horizon. The feedback gain L continuously pulls the disturbed trajectories back toward the nominal one. Geometrically, the disturbed trajectories form a tube around the nominal trajectory, and the feedback gain controls the width of this tube.

This is the basic idea behind **tube-based robust MPC**. The MPC layer plans a nominal trajectory, while the ancillary feedback controller keeps the real trajectory inside a robust tube around it. The constraints are then tightened so that, even after accounting for the tube width, the true state and input remain feasible.

It is important to observe that this parametrization is useful only in the presence of disturbances or uncertainty. If $w_t = 0$ for all t, then there is no advantage in writing

$$u_t = v_t + Lx_t.$$

Indeed, for a nominal trajectory, the optimized variable v_t can simply absorb the term Lx_t. The benefit appears when different disturbance realizations produce different states: the term Lx_t then provides an automatic feedback correction without requiring the MPC optimizer to choose a different open-loop sequence for every possible future disturbance.

3 Economic MPC

3.1 From setpoint tracking to nonzero steady states

In the previous chapters we have always implicitly assumed that we are regulating the steady state $x_S = 0$, $u_S = 0$.
However, in many applications we are interested in regulating to a nonzero setpoint, or even tracking a time-varying reference trajectory. In particular, we typically want the plant to operate at a desired working point $x_S \neq 0$, and we can allow for a constant input u_S that compensates for constant disturbances.

3.1.1 Computation of a feasible steady-state target

If a desired equilibrium point (x_S, u_S) is known, it satisfies

$$x_S = Ax_S + Bu_S.$$

In this case, MPC can be used to stabilize the system around this point by introducing the shifted variables

$$\tilde{x}_k = x_k - x_S, \qquad \tilde{u}_k = u_k - u_S.$$

Substituting into the system dynamics $x_{k+1} = Ax_k + Bu_k$, we obtain

$$\tilde{x}_{k+1} = A\tilde{x}_k + B\tilde{u}_k,$$

which has exactly the same form as the original system. Therefore, in the new coordinates, the regulation problem becomes a standard stabilization problem around the origin.
The finite-horizon MPC cost can then be written as

$$V(x_0) = \min_{\tilde{\mathbf{x}},\tilde{\mathbf{u}}} \sum_{k=0}^{K-1} \left(\tilde{x}_k^\top Q\tilde{x}_k + \tilde{u}_k^\top R\tilde{u}_k\right) + \tilde{x}_K^\top S\tilde{x}_K.$$

For linear systems, this change of variables is **inconsequential**, since it does not modify the structure of the optimal control problem: the dynamics remain linear, the cost remains quadratic, and the constraints (if any) remain affine. As a result, regulating the system to (x_S, u_S) is equivalent to regulating the shifted system to the origin.

3.1.2 Computation of the steady-state target

But how do we decide the steady state (x_S, u_S) to which we want to regulate? In some cases, this may be given by the problem specification, e.g., if we want to track a reference trajectory. In other cases, it can be derived from first principles, e.g., by solving the steady-state equations of the system.
In other cases, specifications indicate a desidered steady state $(x_{\text{spec}}, u_{\text{spec}})$, but the system cannot be stabilized around it. In this case, we can use MPC to find the closest stabilizable point to the desired one, by solving the following optimization problem:

$$\begin{aligned} \min_{x_S,u_S} \quad & \|x_S - x_{\text{spec}}\|_{Q_S}^2 + \|u_S - u_{\text{spec}}\|_{R_S}^2 \\ \text{subject to} \quad & \begin{bmatrix} I - A & -B \end{bmatrix} \begin{bmatrix} x_S \\ u_S \end{bmatrix} = 0 \\ & (x_S, u_S) \in \mathcal{X} \times \mathcal{U}, \end{aligned}$$

where $\mathcal{X}$ and $\mathcal{U}$ are the state and input constraint sets, respectively. The condition $\begin{bmatrix} I - A & -B \end{bmatrix} \begin{bmatrix} x_S \\ u_S \end{bmatrix} = 0$ ensures that the chosen point is an equilibrium of the system, while the cost function penalizes deviations from the desired specifications.
This optimization problem is typically **solved offline** and serves to compute a feasible and stabilizable steady-state target (x_S, u_S). The resulting pair is then used as the reference for the MPC regulator, which is responsible for driving the system to this point while satisfying constraints.

3.2 Motivation for economic MPC

Many modern control problems present **complex performance metrics** that cannot be easily captured by a quadratic cost function. For example, in chemical processes, the economic performance of the system may depend on nonlinear functions of the state and input variables, such as production rates, energy consumption, or profit margins. In such cases, we can use **economic MPC**, which allows for more general cost functions that directly reflect the economic objectives of the system.

Thus, let $\ell(x, u)$ be a general stage cost function that captures the economic performance of the system like economic losses, energy use , cost of material, etc. The economic MPC problem assumes that

- $\ell(x, u)$ is a **continuos** function;
- $\ell(x, u)$ is **lower bounded** in the domain

While remember that in MPC we were assuming that

- $g(x_S, u_S) = 0$ (i.e., the system is at equilibrium at the desired setpoint);
- $g(x, u) \geq 0$
- $g(x, u)$ is radially unbounded, continuos, zero at the origin, and strictly increasing.
- e.g. $g(x, u) = x^\top Q x + u^\top R u$.

3.2.1 Steady-state economic optimization plus tracking MPC

One possible approach is to **separate the economic objective from the dynamic control problem**. In particular, the economic cost is handled at the steady-state level, while MPC is used to steer the system toward the corresponding optimal operating point.
Given a general stage cost $\ell(x, u)$, we first define the **steady-state optimization problem**

$$\begin{aligned} \min_{x_S, u_S} \quad & \ell(x_S, u_S) \\ \text{subject to} \quad & x_S = f(x_S, u_S), \\ & (x_S, u_S) \in \mathcal{X} \times \mathcal{U}. \end{aligned}$$

This problem **determines the economically optimal equilibrium of the system**.

The resulting pair (x_S, u_S) is then used as a reference for the MPC controller, which is designed to track this steady state using a standard quadratic cost. In this way, the economic objective is indirectly enforced through steady-state optimization, while the MPC ensures closed-loop stability and constraint satisfaction.

This approach is typically implemented by solving the steady-state optimization problem offline, or at a slower time scale than the MPC loop. However, this separation introduces some limitations. The closed-loop trajectory is generally **suboptimal from an economic point of**

view, since the stage cost $\ell(x, u)$ is not minimized during transients. Moreover, the controller cannot exploit transient behavior to improve performance, and its response to disturbances may be economically inefficient. On the other hand, the resulting MPC problem remains a standard linear-quadratic program, which is computationally efficient and well understood.

3.3 Economic MPC formulation

3.3.1 Dynamic economic optimization

An alternative approach is to **incorporate the economic objective** directly into the dynamic optimization problem. This leads to the formulation of **economic MPC**, in which the stage cost $\ell(x, u)$ is optimized along the entire predicted trajectory:

$$V(x) = \min_{\mathbf{x},\mathbf{u}} \sum_{k=0}^{K-1} \ell(x_k, u_k)$$

subject to

$$x_{k+1} = f(x_k, u_k), \quad x_0 = x, \quad x_k \in \mathcal{X}, \quad u_k \in \mathcal{U}.$$

In contrast to the previous approach, there is **no separation** between steady-state optimization and tracking. The controller directly optimizes the economic performance over the prediction horizon, resulting in a *single time-scale* scheme in which both transient and steady-state behavior are determined simultaneously.

As a consequence, the controller computes control actions that generate an economically optimal trajectory, rather than simply driving the system toward a precomputed steady state. In particular, the optimal operation may not correspond to a fixed equilibrium, and the controller may intentionally exploit transient behavior if this improves the overall performance.
This increased flexibility comes at the price of a **more challenging optimization problem**. Since the cost $\ell(x, u)$ is in general nonlinear and not necessarily convex, the resulting optimal control problem may be non-convex and harder to solve in real time. Moreover, standard quadratic Lyapunov arguments used in tracking MPC no longer apply directly, and additional conditions are required to guarantee stability.

On the other hand, economic MPC is particularly well suited for nonlinear systems and applications in which performance is naturally described by non-quadratic criteria, such as energy efficiency or economic profit. In these cases, directly optimizing $\ell(x, u)$ allows the controller to fully exploit the system dynamics and constraints to achieve improved closed-loop performance.

A **key issue** with economic MPC arises from the fact that the stage cost $\ell(x, u)$ is not designed to enforce convergence to a steady state. In particular, it may happen that there exist pairs (x, u) that are not equilibria of the system, i.e.,

$$x \neq f(x, u),$$

for which

$$\ell(x, u) < \ell(x_S, u_S),$$

where (x_S, u_S) is an optimal steady state.
This situation is not pathological; on the contrary, it is quite common in practice. Since $\ell(x, u)$ reflects instantaneous economic performance, **it may be advantageous for the system to operate away from equilibrium**, for instance by exploiting transient dynamics or time-varying

operating conditions.

As a consequence, convergence to a steady state is no longer guaranteed nor even desired in general. The optimal closed-loop behavior may correspond to trajectories that do not settle at an equilibrium, such as periodic or more complex operating regimes.
This highlights a **fundamental difference** with standard tracking MPC: convergence to a steady state is not part of the control specification. Instead, the objective is to optimize economic performance over time, possibly at the expense of steady-state operation.

However, this increased flexibility comes with **important challenges**. First, operating away from equilibrium can lead to behaviors that are harder to analyze and predict. In particular, stability and boundedness of the closed-loop trajectories are not automatically guaranteed. Second, without additional conditions or constraints, the controller may generate trajectories that are economically optimal in the short term but undesirable from a safety or robustness perspective.

3.3.2 Infinite horizon, finite horizon, and terminal constraints

A fundamental design choice in economic MPC is the length of the prediction horizon. In principle, the most natural formulation is the infinite-horizon problem

$$\sum_{k=0}^{\infty} \ell(x_k, u_k),$$

which directly captures the long-term economic performance of the system.
The infinite-horizon formulation has a clear economic interpretation, as it reflects the cumulative cost over an indefinite future. This is particularly meaningful in continuous operation settings, where there is no natural terminal time and performance is evaluated over the long run. Moreover, constraints are enforced over the entire trajectory, which ensures boundedness of the closed-loop system by construction.
However, the infinite-horizon problem is infinite-dimensional and therefore computationally intractable in practice. For this reason, it cannot be solved directly in real time.

As a result, practical implementations rely on a finite-horizon approximation of the form

$$\sum_{k=0}^{K-1} \ell(x_k, u_k).$$

This leads to a tractable optimization problem that can be solved online within the MPC framework.
In standard tracking MPC, a finite horizon provides a good approximation of the infinite-horizon problem, since the system is driven toward an equilibrium and the contribution of the tail beyond the horizon can be neglected or approximated. In economic MPC, this argument no longer holds. Since the optimal behavior may not converge to a steady state, the portion of the trajectory beyond the horizon can have a significant impact on performance.

This mismatch introduces several potential issues. First, the controller may favor trajectories that yield low cost within the prediction horizon but lead to poor behavior afterward, as future consequences are not accounted for. In particular, the system may move toward regions from which recovery is difficult or expensive, resulting in undesirable or even unstable behavior.
Second, feasibility problems may arise at the end of the horizon. Due to state and input constraints, not all states reached within the horizon can be extended to admissible infinite

trajectories. This can lead to situations in which the optimization problem becomes infeasible at future time steps.

A common approach to mitigate these issues is to augment the finite-horizon problem with a terminal constraint. In particular, one enforces that the predicted trajectory reaches a feasible steady state at the end of the horizon, i.e.,

$$x_K = x_S, \qquad u_K = u_S,$$

where (x_S, u_S) is an admissible equilibrium of the system.
This modification preserves the computational tractability of the finite-horizon formulation, while introducing a mechanism to guarantee that the predicted trajectory can be extended beyond the horizon in a feasible way. In other words, by constraining the terminal state to an equilibrium, one ensures that there exists a feasible infinite-horizon continuation, thereby recovering *recursive feasibility.*

From this perspective, the terminal constraint provides an explicit characterization of what constitutes a feasible infinite-horizon trajectory: a trajectory is admissible if it can be steered to a steady state that satisfies the system dynamics and constraints.
However, the introduction of a terminal constraint also raises important questions regarding the resulting closed-loop behavior. In particular, it is no longer immediate whether the closed-loop system is stable, or whether the resulting trajectory achieves optimal economic performance. The terminal constraint enforces convergence to a steady state, which may restrict the ability of the controller to exploit economically favorable transient or non-equilibrium operation.

Finally, it is important to emphasize that, as in any MPC scheme, the closed-loop system does not follow the predicted trajectory exactly. At each time step, only the first control input is applied,

$$u_k = u^*(x_k),$$

and the optimization problem is solved again at the next step with updated state information. As a result, the actual closed-loop trajectory is generated by the repeated solution of the finite-horizon problem, and not by the open-loop prediction computed at a single time instant.

3.4 Illustrative example

To better illustrate the difference between standard tracking MPC and economic MPC, consider the following linear system

$$x_{k+1} = Ax_k + Bu_k, \qquad A = \begin{bmatrix} \frac{1}{2} & 1 \\ 0 & \frac{3}{4} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

with input constraint

$$u \in \mathcal{U} = [-1, 1],$$

and economic stage cost

$$\ell(x, u) = q^\top x + ru, \qquad q = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, \quad r = -10.$$

3.4.1 Computation of the optimal steady state

The first step consists in computing the optimal steady state by solving

$$\begin{aligned} \min_{x_S, u_S} \quad & \ell(x_S, u_S) \\ \text{subject to} \quad & x_S = Ax_S + Bu_S, \\ & u_S \in \mathcal{U}. \end{aligned}$$

To find the steady states we need to solve

$$(I - A)x_S = Bu_S,$$

which gives

$$\begin{bmatrix} \frac{1}{2} & -1 \\ 0 & \frac{1}{4} \end{bmatrix} \begin{bmatrix} x_{S,1} \\ x_{S,2} \end{bmatrix} = \begin{bmatrix} 0 \\ u_S \end{bmatrix}$$

Since the cost is linear and the constraints define a polyhedral set, this is a linear program, and the optimum is attained at an extreme point of the feasible set. Thus we only need to evaluate the cost at the two vertices of the input constraint, which gives

$$\text{if } u_S = -1, \quad x_S = \begin{bmatrix} -4 \\ -8 \end{bmatrix}, \quad \ell(x_S, u_S) = 18,$$

$$\text{if } u_S = 1, \quad x_S = \begin{bmatrix} 4 \\ 8 \end{bmatrix}, \quad \ell(x_S, u_S) = -18.$$

The solution (x_S^*, u_S^*) represents the economically optimal equilibrium.

3.4.2 Tracking MPC versus economic MPC

Once (x_S^*, u_S^*) is determined, two fundamentally different control strategies can be constructed.

In the standard MPC tracking approach, the optimal steady state is used as a reference. Defining the shifted variables

$$\tilde{x}_k = x_k - x_S^*, \qquad \tilde{u}_k = u_k - u_S^*,$$

the controller minimizes a quadratic tracking cost of the form

$$g(\tilde{x}, \tilde{u}) = \tilde{x}^\top Q \tilde{x} + \tilde{u}^\top R \tilde{u},$$

driving the system to (x_S^*, u_S^*) as fast as possible. A terminal constraint $\tilde{x}_K = 0$ (equivalently $x_K = x_S^*$) is typically imposed to ensure stability. The resulting optimization problem is a quadratic program.

In contrast, economic MPC directly uses the economic cost in the receding horizon problem:

$$\min_{\mathbf{x},\mathbf{u}} \sum_{k=0}^{K-1} \ell(x_k, u_k)$$

subject to

$$x_{k+1} = Ax_k + Bu_k, \qquad x_k \in \mathcal{X}, \quad u_k \in \mathcal{U},$$

together with a terminal constraint $x_K = x_S^*$ to guarantee feasibility. In this case, since both the dynamics and the cost are linear, the resulting problem is a linear program.

The key difference between the two approaches lies in the objective being optimized. Tracking MPC enforces convergence to the economically optimal steady state and prioritizes stability and regulation performance, but does not optimize the economic cost during transients. Economic MPC, on the other hand, optimizes the economic objective along the entire trajectory, potentially exploiting transient behavior to improve performance.

From a computational perspective, tracking MPC leads to a quadratic program, which is typically well-conditioned and easier to solve reliably in real time. Economic MPC may lead to linear or nonlinear programs depending on the form of $\ell(x, u)$, and can be more challenging, although in this particular example it reduces to a linear program.

3.5 Closed-loop analysis

Now we want to analyze the closed loop behavior. In particular we want to understand if the closed loop stability is guaranteed and if the economic cost is minimez.

3.5.1 Tracking MPC: Lyapunov decrease

In standard tracking MPC, the terminal constraint plays a crucial role in proving asymptotic stability. Indeed, when the cost is built from a positive definite tracking stage cost

$$g(\tilde{x}, \tilde{u}),$$

with

$$g(0,0) = 0, \qquad g(\tilde{x}, \tilde{u}) > 0 \quad \text{for } \tilde{x} \neq 0,$$

the finite-horizon optimal value function can be used as a Lyapunov function for the closed-loop system. The key argument is the same shifted-sequence argument already discussed in the previous chapter. If the optimal predicted trajectory satisfies the terminal constraint

$$\tilde{x}_K = 0,$$

then, after applying the first optimal input, a feasible candidate sequence at the next time step is obtained by shifting the previously optimal sequence and appending the equilibrium input $\tilde{u} = 0$. Since the appended terminal stage cost is zero, one obtains

$$W(f(\tilde{x}, u_0^*(\tilde{x}))) \leq W(\tilde{x}) - g(\tilde{x}, u_0^*(\tilde{x})) \leq W(\tilde{x}),$$

which shows that W decreases along the closed-loop trajectories and is therefore a valid Lyapunov function.

3.5.2 Economic MPC: why the value function is not a Lyapunov function

The natural question is whether the same idea can be applied to economic MPC. Suppose that we consider the finite-horizon economic MPC problem with terminal constraint

$$x_K = x_S,$$

where (x_S, u_S) is an admissible steady state. The same shifted-sequence construction remains feasible: after applying the first optimal control input, one can shift the optimal predicted trajectory and append the steady-state input u_S at the end of the horizon. Therefore, feasibility is preserved. However, the cost argument changes in a fundamental way.
Indeed, for economic MPC the stage cost is $\ell(x, u)$, which is not positive definite with respect to the steady state and is not, in general, minimized at (x_S, u_S). As a consequence, the shifted-sequence argument only yields

$$W(f(x, u_0^*(x))) \leq W(x) - \ell(x, u_0^*(x)) + \ell(x_S, u_S).$$

This inequality is not sufficient to conclude that W decreases. In fact, it may happen that

$$\ell(x_S, u_S) > \ell(x, u_0^*(x)),$$

so the right-hand side can be larger than $W(x)$. Therefore, unlike in tracking MPC, the optimal finite-horizon value function is *not* automatically a Lyapunov function for the closed-loop system.

This observation has an important conceptual consequence. In economic MPC, asymptotic stability of the steady state is not guaranteed by the terminal constraint alone. The terminal

constraint guarantees recursive feasibility, but not necessarily convergence to the equilibrium. This is perfectly consistent with the economic interpretation of the problem: the controller is asked to optimize economic performance, not to minimize the distance from the steady state. If operating away from equilibrium is economically more favorable, then the closed-loop system may prefer persistent transient behavior or even non-equilibrium operating regimes.

For this reason, asymptotic convergence to the steady state is not always the most relevant objective in economic MPC. In some applications, one may actually achieve a lower long-run cost by not converging to an equilibrium. For nonlinear systems, it is even possible that economically preferable periodic trajectories exist, so that oscillatory operation can outperform steady-state operation. Hence, from the viewpoint of economic MPC, the relevant performance question is often not whether the state converges to x_S, but whether the closed-loop system achieves good *average* economic performance over time.

3.5.3 Asymptotic average performance

This leads to a weaker, but very meaningful, notion of closed-loop performance. Consider the closed-loop trajectory generated by economic MPC with terminal constraint x_S. Under suitable assumptions, one can prove the following asymptotic average performance bound:

$$\limsup_{t\to\infty} \frac{1}{t} \sum_{k=0}^{t-1} \ell(x_k, u_k) \leq \ell(x_S, u_S).$$

This result states that the asymptotic average economic performance of the closed loop is no worse than the cost of operating permanently at the chosen steady state. In other words, even if the trajectory does not converge to (x_S, u_S), its long-run average performance is at least as good as the steady-state economic benchmark.
The proof again relies on the terminal constraint and on the shifted-sequence argument. From feasibility of the shifted trajectory, one obtains

$$W(f(x, u_0^*(x))) \leq W(x) - \ell(x, u_0^*(x)) + \ell(x_S, u_S).$$

Summing this inequality along the closed-loop trajectory for $k = 0, \ldots, t-1$ yields

$$\sum_{k=0}^{t-1} \ell(x_k, u_k) \leq t\, \ell(x_S, u_S) + W(x_0) - W(x_t).$$

Dividing by t gives

$$\frac{1}{t} \sum_{k=0}^{t-1} \ell(x_k, u_k) \leq \ell(x_S, u_S) + \frac{W(x_0) - W(x_t)}{t}.$$

If the value function remains bounded along the closed loop, the last term vanishes as $t \to \infty$, from which the average performance bound follows.

It is important to stress the meaning of this result. The function W is not a Lyapunov function here, because it is not required to decrease at every time step. Instead, it acts as a storage-like quantity that allows us to compare the accumulated closed-loop cost with the steady-state benchmark. Thus, the conclusion is not asymptotic stability, but an *asymptotic average cost guarantee.*

We can now summarize the main message of this discussion. In economic MPC, a terminal constraint is essential to guarantee recursive feasibility of the receding-horizon scheme. However,

unlike in tracking MPC, the terminal constraint alone is not enough to guarantee asymptotic stability of the optimal steady state. The closed-loop system may generate economically efficient trajectories that do not converge to an equilibrium, and this is not necessarily undesirable. In fact, even when the trajectory eventually converges to the steady state, economic MPC may still outperform standard tracking MPC because it optimizes the transient behavior as well. Over long operating periods, and especially in the presence of recurrent disturbances, this transient economic advantage can accumulate and lead to significantly improved overall performance.

Additional assumptions can be introduced to recover asymptotic stability of the steady state in economic MPC. These conditions are stronger than those used in standard tracking MPC and are not automatic consequences of the basic formulation. Their role is precisely to connect economic optimality with a suitable notion of dissipativity or storage of the system, so that the closed-loop behavior can again be related to equilibrium convergence. In the absence of such additional assumptions, the most natural guarantee remains the asymptotic average performance property discussed above.

4 Feedback optimization

4.1 Problem setting and architecture

4.1.1 Optimal steady-state regulation

In many automation problems, the control objective is not only to stabilize the plant, but to drive it toward an *optimal operating point*. This operating point is typically characterized by steady-state specifications rather than by transient performance requirements.

The **setting** we have in mind is the following. The plant is nonlinear, but it is either already stable or has been pre-stabilized by a lower-level controller. Therefore, the main control task is no longer to stabilize an unstable system from scratch, but to **determine and maintain the best admissible steady state**. This is particularly relevant when the plant must operate continuously around an equilibrium and performance is mainly determined by the chosen operating point.

A second important feature is that, in many applications, an accurate dynamic model is not available. Even when a rough dynamical description exists, it may be too uncertain to support a reliable model-based dynamic optimization over a prediction horizon. On the other hand, steady-state relations are often easier to obtain from physical insight, experiments, maps, or data. For this reason, one may prefer to formulate the control objective directly in terms of steady-state optimization.
The steady-state specifications are usually expressed in terms of optimality criteria. Typical examples are:

- maximizing yield;
- minimizing economic cost;
- reducing CO_2 emissions.

Hence, instead of assigning a set-point a priori, one wants to *compute* the best steady state according to a given performance index.

This optimization is rarely unconstrained. In practice, several manipulated inputs must be selected simultaneously, and each of them is subject to physical or operational limits. Moreover, the desired operating point must satisfy several steady-state constraints, such as safety limits, balance equations, admissibility conditions, or exact power and mass balances. Therefore, the target selection problem is naturally a **constrained steady-state optimization problem**.

Another crucial aspect is that the optimal steady state is generally not fixed once and for all. It depends on exogenous factors that affect the plant equilibrium. These may include external disturbances, slowly varying demand, ambient conditions, or unknown plant parameters. As these quantities change, the economically or operationally optimal steady state changes as well. The controller must then adapt the plant operation so as to regulate the system around the new optimal equilibrium.

Two representative application domains help clarify the nature of the problem.
In *grid frequency control*, one must decide how to activate available reserves, that is, how much generation should ramp up or down. The external demand is time-varying and acts as an exogenous signal. At the same time, one seeks low operating cost while satisfying the exact steady-state power balance. Thus, the main question is how to choose the steady-state allocation of generation in an optimal and constraint-satisfying way.
In *process systems*, for instance in compressor networks, the mapping from the manipulated variables to the steady-state operating condition can be highly nonlinear and difficult to model

dynamically. There may be several tunable parameters and set-points, together with efficiency objectives, safety constraints, and actuator limitations. Again, the core task is to find and maintain the best admissible steady state.

Difference from Economic MPC. At first sight, this viewpoint may seem close to Economic MPC, since both approaches use economic or performance-oriented criteria rather than purely quadratic tracking objectives. The difference is conceptual and important.
In Economic MPC, the economic stage cost is optimized *along the whole predicted trajectory*. Therefore, transient behavior is part of the optimization problem: the controller may intentionally exploit transients if this improves the overall economic performance. For this reason, Economic MPC requires a dynamic model and solves a finite-horizon dynamic optimization problem online.
In optimal steady-state regulation, instead, the main objective is to optimize the *equilibrium* itself. The transient is not the primary object of optimization. One assumes that the plant is stable or pre-stabilized, and one uses feedback mainly to steer the system toward the optimal admissible steady state and to keep it there despite changes in disturbances or parameters. In this sense, feedback optimization is closer to a real-time steady-state target adaptation layer than to full dynamic economic optimization.

Therefore, the two approaches differ in what is being optimized:

- **Economic MPC:** optimize the economic performance of the entire transient trajectory;
- **Optimal steady-state regulation / feedback optimization:** optimize the steady-state operating point and regulate the plant around it.

This distinction is especially relevant when the dynamic model is inaccurate but the steady-state behavior is sufficiently known. In that case, feedback optimization remains meaningful, while Economic MPC may be difficult to formulate reliably.

4.1.2 Fast-stable plant

We now introduce the basic setting for feedback optimization in the case of a *fast and stable* plant. The idea is that the plant dynamics are sufficiently fast, and already stable, so that from the viewpoint of the upper optimization layer the dominant object is the steady-state input–output relation.

We assume a continuous-time, linear time-invariant plant of the form

$$\dot{x} = Ax + Bu + Ew, \qquad y = Cx + Du,$$

where x is the state, u is the manipulated input, w is an exogenous variable or disturbance, and y is the controlled output or performance variable of interest.

The key assumption is that the plant is *exponentially stable*. This means that, for constant inputs u and constant disturbances w, the state converges exponentially fast to a unique equilibrium. Therefore, **after transients have died out, the output is determined only by the constant values of u and w**. This motivates the introduction of the *steady-state map*

$$y = Hu + Lw.$$

In the general nonlinear case, the same idea is expressed by a steady-state relation of the form

$$y = h(u; w),$$

where h is a possibly nonlinear map that associates to each constant input–disturbance pair the corresponding steady-state output.

Let us now derive the matrices H and L in the linear case. At steady state we must have $\dot{x} = 0$, hence

$$0 = Ax + Bu + Ew.$$

If A is invertible, we can solve for the equilibrium state:

$$x = -A^{-1}Bu - A^{-1}Ew.$$

Substituting this expression into the output equation gives

$$y = Cx + Du = C\left(-A^{-1}Bu - A^{-1}Ew\right) + Du.$$

Rearranging the terms yields

$$y = \left(-CA^{-1}B + D\right)u - CA^{-1}Ew.$$

Therefore, the steady-state gains are

$$H = -CA^{-1}B + D, \qquad L = -CA^{-1}E.$$

So H is the steady-state gain from the manipulated input u to the output y, while L is the steady-state gain from the exogenous signal w to the output y.

This also answers the question about the conditions on A. In order to define the steady-state map in this form, A must be invertible. In particular, if the plant is exponentially stable, then all eigenvalues of A have strictly negative real part. Hence 0 cannot be an eigenvalue of A, and therefore A is automatically nonsingular. So invertibility of A is not an additional restriction here: it already follows from exponential stability.

The conclusion is that, **for a fast and exponentially stable plant**, the dynamic relation between u and y can be replaced at steady state by the static map

$$y = Hu + Lw,$$

which is the object that will be used in the feedback optimization layer.

4.1.3 "Certainty-equivalence" design

Given the steady-state map

$$y = h(u; w),$$

a natural first idea is to formulate the steady-state requirements directly as a static optimization problem. Let $\Phi(u, y)$ denote the performance index to be minimized, let $\mathcal{U}$ be the set of admissible inputs, and let $\mathcal{Y}$ be the set of admissible steady-state outputs. The steady-state specifications can then be written as

$$\begin{aligned} \min_{u,y} \quad & \Phi(u, y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u; w). \end{aligned}$$

This formulation expresses exactly the control objective discussed above: choose the input u so that the plant settles at a feasible steady state y, while optimizing the desired economic or operational criterion.

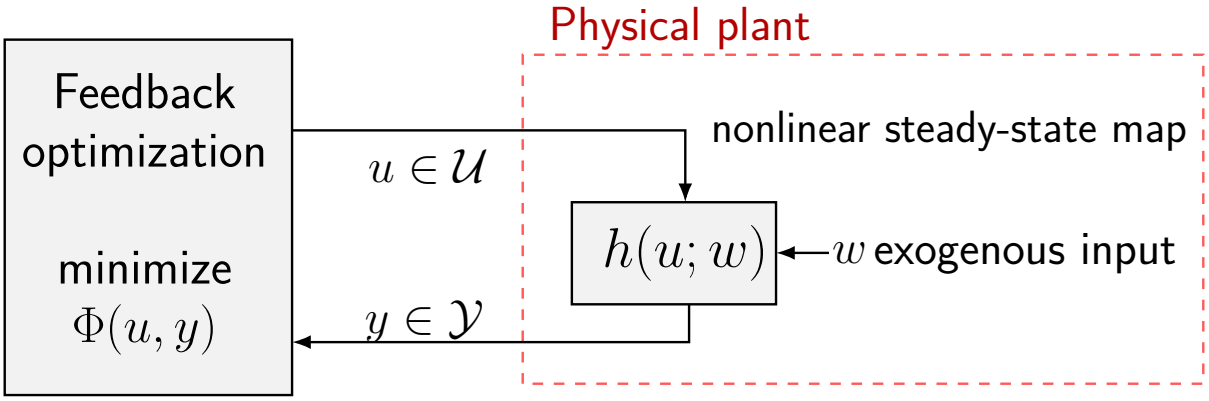

Figure 6: Certainty-equivalence view of steady-state optimization. The optimizer computes the input $u \in \mathcal{U}$ using the steady-state relation $y = h(u; w)$ so as to minimize the performance index $\Phi(u, y)$ while satisfying the steady-state constraints $y \in \mathcal{Y}$.

If the exogenous quantity w were known and the map h were exactly available, then one could eliminate the variable y and solve the reduced problem

$$u^\star = \arg\min_{u \in \mathcal{U}} \tilde{\Phi}(u) \qquad \text{with} \qquad \tilde{\Phi}(u) := \Phi\big(u, h(u; w)\big),$$

subject to

$$h(u; w) \in \mathcal{Y}.$$

Once the optimizer $u^\star$ has been computed, one simply applies it to the plant. This is the standard *certainty-equivalence* or *feedforward* approach: one behaves as if the uncertain quantities were known with certainty and as if the plant steady-state model were exact.

Conceptually, this is appealing because it reduces the problem to a classical nonlinear optimization task. However, this approach has two important weaknesses.
First, it requires accurate knowledge of the exogenous input w. In practice, w may represent an unknown disturbance, a slowly varying demand, or an uncertain operating condition. If the value used in the optimization differs from the true one, then the computed input $u^\star$ is generally not the true optimizer for the physical plant. As a consequence, the achieved steady state may be suboptimal or may even violate the output constraints.

Second, the approach requires an accurate steady-state model h. If the map used in the optimization does not match the actual plant steady-state relation, then the predicted output

$$y = h(u; w)$$

is not the one that will actually be produced by the plant. Hence, even when the optimization problem is solved exactly, the resulting input may fail to achieve the desired steady-state specifications on the real system.

So the usual feedforward problem is clear: solving the static optimization problem is not enough when the disturbance is uncertain and the model is imperfect. The optimizer is optimal only for the *model-based problem*, not necessarily for the physical plant. This is precisely the motivation for introducing feedback into the optimization loop.

A natural question then arises: *how can we design feedback controllers that yield the desired autonomous closed-loop behavior?* The idea developed in feedback optimization is to combine two ingredients:

- an algebraic input/output steady-state map

$$y = h(u; w),$$

  valid when w is constant or slowly changing;

- classical iterative nonlinear optimization algorithms.

Rather than solving the steady-state optimization problem once and applying the resulting input in open loop, we construct a controller that updates the input iteratively on the basis of measured plant outputs. In this way, the optimization algorithm and the physical plant become interconnected and the optimizer is embedded into the feedback loop. The resulting closed-loop system should drive the plant toward a steady state that satisfies the constraints and is optimal for the *actual* plant, not only for the nominal model.

4.1.4 Interconnecting plant and algorithms

This viewpoint can be understood by comparing two different implementations.

In the first implementation, one keeps the optimization algorithm and the plant separate. The algorithm interacts only with a model of the steady-state map and solves

$$\begin{aligned} \min_{u,y} \quad & \Phi(u, y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u; w). \end{aligned}$$

The model returns the optimizer $u^\star$, and this value is then applied to the plant. This is exactly the feedforward architecture discussed above. Its weakness is that the optimization loop closes around the *model*, while the real plant appears only at the end, as a system to which the precomputed command is sent. Therefore, any mismatch between model and plant directly affects the achieved performance.

The alternative is to interconnect the optimization algorithm with the physical plant itself. In this case, the algorithm produces an input u, the plant responds with an output y, and this measured output is fed back to the algorithm. The optimization method is no longer driven only by a nominal model prediction, but by the actual plant behavior. As a consequence, the closed-loop system formed by

$$\text{optimization algorithm} + \text{plant}$$

becomes the main object of analysis and design.

This is the key conceptual step of feedback optimization. The optimization algorithm is no longer viewed as an offline numerical routine that computes a set-point. Instead, it is implemented dynamically and interconnected with the plant in feedback. The plant acts as part of the optimization loop, and the measured output provides the information needed to compensate unknown disturbances and model mismatch.

In summary, the transition is the following:

- in feedforward optimization, the algorithm solves the steady-state problem on a model and sends the resulting optimizer to the plant;
- in feedback optimization, the algorithm is directly interconnected with the plant, and the plant measurements are used online to steer the iterative process toward the desired optimum.

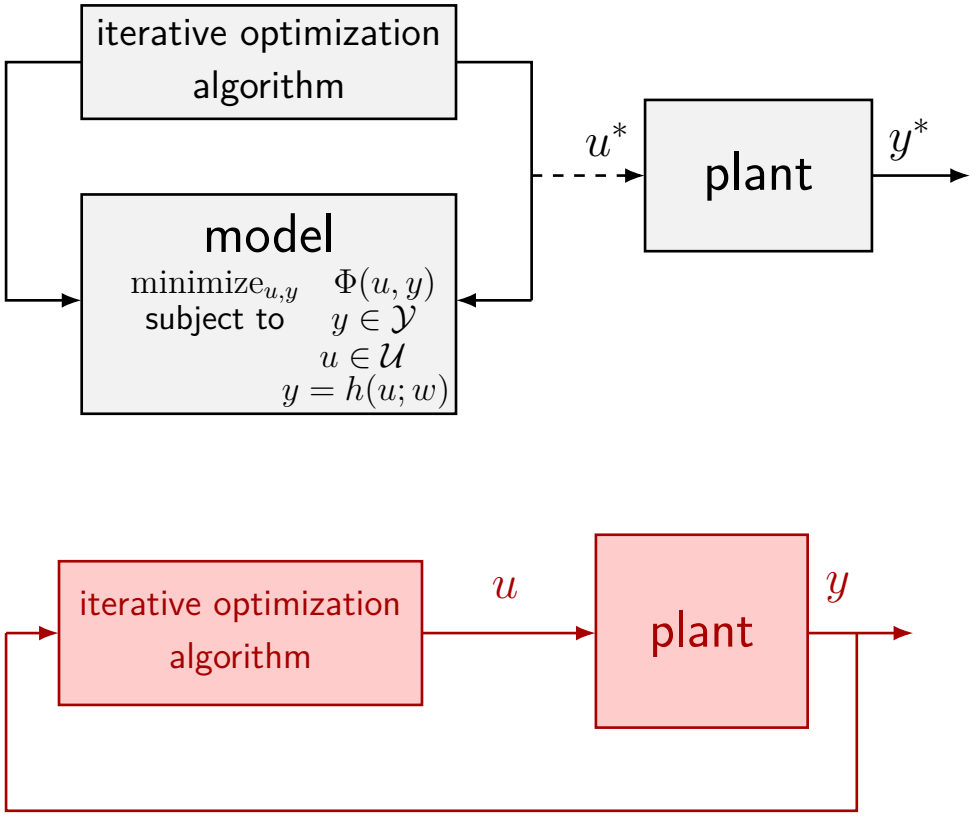

Figure 7: Comparison between model-based feedforward optimization and feedback optimization. In the upper scheme, the iterative optimization algorithm is closed around the model and produces the command $u^\star$ that is then sent to the plant. In the lower scheme, the optimization algorithm is directly interconnected with the physical plant through the measured output y, thereby forming an autonomous feedback loop.

This interconnection is what allows one to move beyond certainty-equivalence design and obtain an autonomous closed-loop behavior with improved robustness to uncertainty in w and in the steady-state map h.

4.1.5 Online feedback optimization

The previous discussion motivates a shift from static feedforward optimization to *online feedback optimization*. The idea is to implement an optimization algorithm as a dynamical system interacting directly with the plant, so that the plant measurements continuously correct the optimization process. In this way, the input is not computed once and for all from an uncertain model, but is updated online on the basis of the actual closed-loop behavior.

The steady-state optimization problem of interest is

$$\begin{aligned} \min_{u,y} \quad & \Phi(u,y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u;w). \end{aligned}$$

Using the steady-state relation, this problem can be equivalently written only in terms of the

decision variable u as

$$\begin{aligned} \min_{u \in \mathcal{U}} \quad & \Phi\big(u, h(u; w)\big) \\ \text{s.t.} \quad & h(u; w) \in \mathcal{Y}. \end{aligned}$$

So the plant enters the optimization problem through the algebraic steady-state map $y = h(u; w)$, while the constraints may act both directly on the input and indirectly on the output.

Once the problem has been written in this form, many classical iterative methods become available. In particular, the most relevant class for feedback optimization is that of *first-order optimization algorithms*, that is, methods based on gradient information or generalized descent directions. Typical examples are:

- gradient descent combined with penalty or barrier terms;
- projected gradient methods;
- primal–dual dynamics or primal–dual flows.

The reason these methods are attractive is that they admit iterative implementations that can naturally be interconnected with the plant. Instead of solving the optimization problem to completion at each step, one performs repeated correction steps, and the plant output provides the information needed to drive the iterations toward the desired optimum.

4.2 First-order methods for feedback optimization

4.2.1 First-order optimization algorithms

Let us now look more closely at the class of first-order methods. Consider again the constrained steady-state problem

$$\begin{aligned} \min_{u,y} \quad & \Phi(u, y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u; w). \end{aligned}$$

Equivalently, after eliminating y through the steady-state map, we obtain

$$\begin{aligned} \min_{u \in \mathcal{U}} \quad & \Phi\big(u, h(u; w)\big) \\ \text{s.t.} \quad & h(u; w) \in \mathcal{Y}. \end{aligned}$$

This equivalent formulation is useful because it shows that the optimization variable can be taken to be only u, while the output constraint is enforced through the plant steady-state relation.

At this point, the problem has the structure of a nonlinear constrained optimization problem, and one may attack it with standard first-order techniques. The main issue is how to handle the constraints. A first possibility is to absorb the output constraint into the cost by adding a suitable extra term. This leads to gradient methods with *penalty* or *barrier* functions.

4.2.2 Gradient descent with penalty / barrier

Consider the constrained problem

$$\begin{aligned} \min_{u,y} \quad & \Phi(u, y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u; w). \end{aligned}$$

We focus here on the output constraint $y \in \mathcal{Y}$. A standard way to handle it is to transform the constrained problem into an unconstrained or less constrained one by modifying the cost function. Two classical constructions are the penalty approach and the barrier approach.

Penalty function. In the penalty approach, one introduces an additional term $p(y)$ in the objective and solves

$$\min_{u,y} \ \Phi(u, y) + p(y).$$

The function $p(y)$ is chosen so that it is small or zero when the constraint is satisfied, and increases when the output violates the admissible set $\mathcal{Y}$. In this way, constraint violations are discouraged, but not made impossible. Small violations may still occur if they lead to a lower total cost.

This approach is often attractive because it leads to a smooth unconstrained optimization problem, which is easy to handle with gradient-based methods. However, the price to pay is that feasibility is not enforced exactly: one only penalizes infeasibility. Therefore, the obtained solution may slightly violate the constraint, depending on the shape and weight of the penalty.

(a) Penalty function. (b) Barrier function.

Figure 8: Qualitative comparison between penalty and barrier functions used to enforce the output constraint $y \in \mathcal{Y}$. A penalty function remains finite outside the feasible set and discourages violations without excluding them completely. A barrier function instead becomes unbounded at the boundary of the feasible set and is undefined outside it, thereby forcing the iterates to remain strictly feasible.

Barrier function. In the barrier approach, one instead introduces a function $b(y)$ and solves

$$\min_{u,y} \ \Phi(u, y) + b(y).$$

Now the function $b(y)$ is defined only inside the feasible set and tends to $+\infty$ as y approaches the boundary of $\mathcal{Y}$. As a consequence, the optimizer is forced to remain in the interior of the feasible region.

This gives a more conservative behavior than the penalty approach. Since the barrier becomes very large near the boundary, the algorithm tends to stay away from it. Moreover, the formulation is not even defined outside the feasible set, so one must initialize and operate the algorithm inside the admissible region. The benefit is that feasibility is preserved by construction, at least in the ideal continuous-time or exact-iteration picture.

Comparison. The two approaches reflect two different ways of dealing with constraints:

- with a *penalty function*, small violations are acceptable, since infeasibility is merely discouraged;

- with a *barrier function*, one is conservative and never reaches the boundary, because leaving the feasible region is forbidden by the formulation itself.

These ideas are particularly useful in feedback optimization because they turn constrained optimization into a form that can be handled by simple first-order iterative laws. The optimization algorithm then updates the input u by descending along the modified objective, while the plant provides the output y entering the cost and the constraint-handling terms.

4.2.3 Unconstrained gradient descent

Before applying gradient methods to the feedback optimization problem, it is useful to recall the basic convergence property of gradient descent in the unconstrained case. Consider the problem

$$\min_x \ \phi(x),$$

and assume that ϕ is twice continuously differentiable and has bounded second derivative, namely

$$\nabla^2\phi(x) \preceq MI \qquad \forall x,$$

for some constant $M > 0$. This means that the gradient of ϕ is Lipschitz continuous with Lipschitz constant bounded by M.
The gradient descent iteration is

$$x_{k+1} = x_k - \alpha\nabla\phi(x_k),$$

where $\alpha > 0$ is the stepsize. Under the assumption above, this iteration converges to a local minimum of ϕ provided that

$$\alpha < \frac{2}{M}.$$

The reason is seen by expanding ϕ around the current iterate x_k. By Taylor's formula,

$$\phi(x_{k+1}) = \phi(x_k) + \nabla\phi(x_k)^\top(x_{k+1} - x_k) + \frac{1}{2}(x_{k+1} - x_k)^\top\nabla^2\phi(z)(x_{k+1} - x_k),$$

where z is a point on the segment joining x_k and x_{k+1}. Substituting the gradient descent update

$$x_{k+1} - x_k = -\alpha\nabla\phi(x_k),$$

we obtain

$$\phi(x_{k+1}) = \phi(x_k) - \alpha\|\nabla\phi(x_k)\|^2 + \frac{\alpha^2}{2}\nabla\phi(x_k)^\top\nabla^2\phi(z)\nabla\phi(x_k).$$

Using the bound $\nabla^2\phi(z) \preceq MI$, the last term can be bounded as

$$\nabla\phi(x_k)^\top\nabla^2\phi(z)\nabla\phi(x_k) \le M\|\nabla\phi(x_k)\|^2,$$

and therefore

$$\phi(x_{k+1}) \le \phi(x_k) - \alpha\|\nabla\phi(x_k)\|^2 + \frac{\alpha^2 M}{2}\|\nabla\phi(x_k)\|^2.$$

Equivalently,

$$\phi(x_{k+1}) \le \phi(x_k) - \alpha\left(1 - \frac{\alpha M}{2}\right)\|\nabla\phi(x_k)\|^2.$$

Hence, if

$$\alpha < \frac{2}{M},$$

the coefficient in parentheses is positive, and so

$$\phi(x_{k+1}) < \phi(x_k) \qquad \text{whenever } \nabla\phi(x_k) \neq 0.$$

So the sequence $\phi(x_k)$ is decreasing along the iterations until the gradient vanishes. This is the key fact behind convergence of gradient descent: the cost decreases monotonically, and the iterates are driven toward the set of stationary points

$$\{x : \nabla\phi(x) = 0\}.$$

Under suitable additional assumptions, this implies convergence to a local minimum.

4.2.4 Gradient descent with penalty

We now apply this idea to the steady-state optimization problem in the presence of output constraints handled through a penalty term. Starting from

$$\begin{aligned} \min_{u,y} \quad & \Phi(u,y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u;w), \end{aligned}$$

we incorporate the constraint $y \in \mathcal{Y}$ into the objective through a penalty function $p(y)$ and obtain the problem

$$\min_{u\in\mathcal{U}} \ \Phi\big(u, h(u;w)\big) + p\big(h(u;w)\big).$$

Here the admissible input set $\mathcal{U}$ is kept explicitly, while the output constraint is treated by penalization.
To derive a gradient algorithm, define the penalized objective

$$J(u) := \Phi\big(u, h(u;w)\big) + p\big(h(u;w)\big).$$

Then the gradient descent iteration is

$$u_{k+1} = u_k - \alpha\nabla J(u_k).$$

Using the chain rule, the gradient of J is

$$\nabla J(u) = \nabla_u\Phi(u, h(u;w)) + \nabla_u h(u;w)^\top \nabla_y\Phi(u, h(u;w)) + \nabla_u h(u;w)^\top \nabla_y p(h(u;w))$$

. Therefore, the gradient descent algorithm becomes

$$u_{k+1} = u_k - \alpha\Big(\nabla_u\Phi(u_k, h(u_k;w)) + \nabla_u h(u_k;w)^\top \nabla_y\Phi(u_k, h(u_k;w)) + \nabla_u h(u_k;w)^\top \nabla_y p(h(u_k;w))\Big).$$

This is the ideal algorithm one would write if the steady-state map h were exactly known. However, in feedback optimization we do not want to evaluate the cost only on the basis of the model output $h(u_k;w)$; rather, we want to use the *measured* plant output. Denoting by y_k the measured output corresponding to the current input u_k, the update law becomes

$$u_{k+1} = u_k - \alpha\Big(\nabla_u\Phi(u_k, y_k) + \nabla_u h(u_k;w)^\top \nabla_y\Phi(u_k, y_k) + \nabla_u h(u_k;w)^\top \nabla_y p(y_k)\Big).$$

This is the *gradient descent controller*: the optimization step is corrected using the actual output measurement y_k, while the Jacobian $\nabla_u h(u_k;w)$ is still taken from the model. The input is finally constrained to lie in $\mathcal{U}$, for instance by saturation or projection.
The structure of the controller is worth interpreting carefully:

- $\nabla_u\Phi(u_k, y_k)$ measures the direct effect of the input on the cost;
- $\nabla_y\Phi(u_k, y_k)$ measures how the cost changes with the output;

- $\nabla_y p(y_k)$ measures how strongly the penalty reacts to output constraint violations;
- $\nabla_u h(u_k; w)$ maps output sensitivities back to input directions.

So the matrix

$$\nabla_u h(u; w)^\top$$

plays a crucial role: it tells us how a change in input affects the steady-state output, and therefore how to translate output-based optimization information into an input update.

Steady-state sensitivities. The quantity

$$\nabla_u h(u; w)^\top$$

is the transpose of the Jacobian of the steady-state map with respect to the input. It is the local steady-state input–output sensitivity matrix. If the output dimension is p and the input dimension is m, then

$$\nabla_u h(u; w) \in \mathbb{R}^{p \times m},$$

and its (i, j)-th entry is

$$\frac{\partial h_i(u; w)}{\partial u_j}.$$

This derivative tells us how the i-th steady-state output changes when the j-th input is perturbed. For a linear fast-stable plant, where

$$y = Hu + Lw,$$

the steady-state sensitivity is simply constant:

$$\nabla_u h(u; w) = H.$$

So in the linear case the controller uses the steady-state gain matrix $H^\top$.

In nonlinear plants, instead, the sensitivity depends on the current operating point. Some intuitive examples are the following:

- in a compressor or process unit, the derivative describes how the steady-state flow, pressure, or efficiency changes when a valve opening or actuator set-point is modified;
- in power systems, it represents how steady-state frequency, power flow, or voltage-related quantities react to a change in generation dispatch;
- in thermal systems, it measures how the equilibrium temperature changes when heater power or coolant flow is varied.

Thus, the Jacobian $\nabla_u h(u; w)$ is exactly the information needed to connect an optimization algorithm to the physical plant at steady state.

The key point is that the controller does not require a full dynamic model of the plant. It only needs a model, estimate, or approximation of the steady-state sensitivities. This is much weaker than full model-based control and is one of the main reasons why feedback optimization is attractive in applications where the transient dynamics are uncertain but the steady-state behavior is sufficiently structured.

4.2.5 Projected gradient descent

Penalty and barrier methods handle the output constraint indirectly by modifying the objective function. A different possibility is to enforce the constraints explicitly through projection. Starting from

$$\begin{aligned} \min_{u,y} \quad & \Phi(u,y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u;w), \end{aligned}$$

and eliminating the variable y through the steady-state relation, we obtain

$$\min_{u} \ \Phi\big(u, h(u;w)\big)$$

subject to

$$u \in \mathcal{U}, \qquad h(u;w) \in \mathcal{Y}.$$

It is convenient to collect both feasibility requirements into a single admissible set for the input:

$$\tilde{\mathcal{U}} := \mathcal{U} \cap h^{-1}(\mathcal{Y};w),$$

where

$$h^{-1}(\mathcal{Y};w) := \{u : \ h(u;w) \in \mathcal{Y}\}.$$

Then the optimization problem can be rewritten as

$$\min_{u \in \tilde{\mathcal{U}}} \ \Phi\big(u, h(u;w)\big).$$

This suggests the use of projected gradient descent: one first takes a descent step for the cost, and then projects the result back onto the feasible set $\tilde{\mathcal{U}}$. Formally, if

$$J(u) := \Phi\big(u, h(u;w)\big),$$

the projected gradient iteration is

$$u_{k+1} = \Pi_{\tilde{\mathcal{U}}}\Big(u_k - \alpha \nabla J(u_k)\Big),$$

where $\Pi_{\tilde{\mathcal{U}}}$ denotes the Euclidean projection onto $\tilde{\mathcal{U}}$.
The appealing feature of this method is that, unlike a penalty formulation, it tries to maintain feasibility explicitly. The difficulty, however, is hidden in the projection step. While projection onto the input set $\mathcal{U}$ may be easy, the set

$$h^{-1}(\mathcal{Y};w)$$

is generally complicated. Even when $\mathcal{Y}$ is a simple set, its inverse image through a nonlinear steady-state map may be highly nonconvex and may have a complicated geometry. As a consequence, projecting onto

$$\tilde{\mathcal{U}} = \mathcal{U} \cap h^{-1}(\mathcal{Y};w)$$

can be numerically difficult and conceptually unpleasant. This is exactly the issue highlighted by figure 9: a simple admissible set in the output space may correspond, in the input space, to a distorted and difficult set.

To make this approach practical, one would like a good approximation of the set $h^{-1}(\mathcal{Y};w)$ that satisfies two properties:

- it should be easy to project onto, ideally a polytope or another simple convex set;
- it should be sufficiently accurate, especially near the boundary of $\mathcal{Y}$, where constraint satisfaction matters most.

A common assumption is that the admissible sets are polyhedral:

$$\mathcal{U} := \{u \in \mathbb{R}^m : \ Au \le b\}, \qquad \mathcal{Y} := \{y \in \mathbb{R}^p : \ Cy \le d\}.$$

In this case, the difficulty is entirely due to the nonlinear steady-state map. The set $\mathcal{Y}$ is simple in the output space, but its inverse image through h is not.

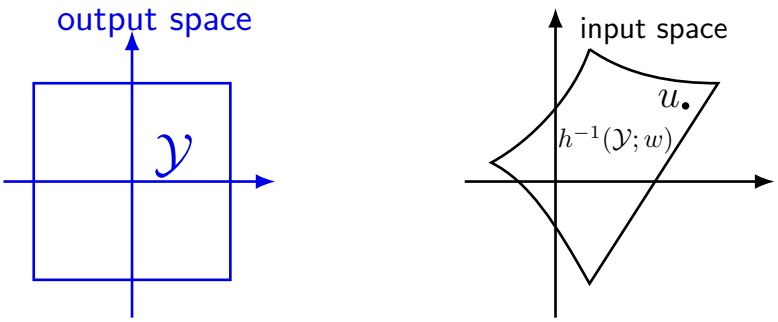

Figure 9: Geometry of the output constraint and its inverse image in the input space. A simple polyhedral constraint set $\mathcal{Y}$ in the output space induces, through the nonlinear map $h(u; w)$, a generally complicated feasible region $h^{-1}(\mathcal{Y}; w)$ in the input space. Intersecting this set with the input constraint set $\mathcal{U}$ yields the feasible set for projected gradient descent.

The natural way to obtain a tractable approximation is to linearize the steady-state map around the current input u. Using a first-order Taylor expansion, for a nearby point u' we have

$$h(u'; w) \approx h(u; w) + \nabla_u h(u; w)^\top (u' - u).$$

If the output constraint is

$$Cy \le d,$$

then substituting the first-order approximation gives

$$C\, h(u'; w) \le d \quad \Rightarrow \quad C\Big(h(u; w) + \nabla_u h(u; w)^\top (u' - u)\Big) \le d.$$

This defines a local first-order approximation of the inverse image $h^{-1}(\mathcal{Y}; w)$ around the current point u. Importantly, this approximation is affine in u', and therefore it produces a polyhedral local approximation of the feasible set in the input space.

So, instead of projecting onto the exact set $h^{-1}(\mathcal{Y}; w)$, which is difficult, one projects onto its linearized approximation. This preserves the main advantage of projected methods, namely explicit constraint handling, while replacing the hard nonlinear projection with a much simpler projection onto a local polytope.

This leads to the following interpretation. Projected gradient descent is conceptually attractive, because it tries to enforce feasibility directly. However, the exact feasible set in the input space is usually too complicated. The practical solution is therefore to replace the exact inverse image of the output constraints with a local first-order approximation based on the steady-state sensitivities

$$\nabla_u h(u; w).$$

Once again, the steady-state Jacobian is the central object: it provides the linear map that translates output constraints into approximate input constraints.

In summary:

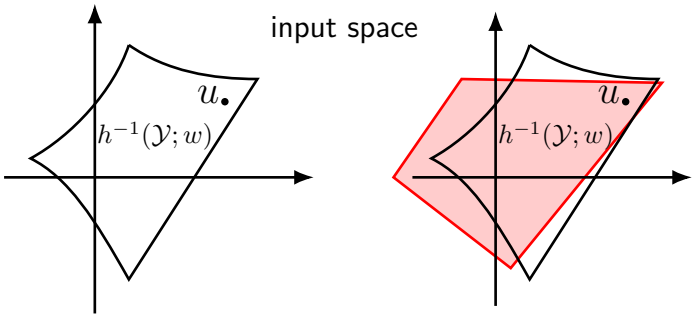

Figure 10: First-order approximation of the inverse image of the output constraint. Linearizing the steady-state map around the current input u transforms the nonlinear feasible set $h^{-1}(\mathcal{Y}; w)$ into a local polyhedral approximation that is much easier to use in a projection step.

- the exact projected method would require projection onto

$$\tilde{\mathcal{U}} = \mathcal{U} \cap h^{-1}(\mathcal{Y}; w),$$

  which is generally difficult;
- if $\mathcal{U}$ and $\mathcal{Y}$ are polyhedral, then a local linearization of h gives a polyhedral approximation of $h^{-1}(\mathcal{Y}; w)$;
- this approximation makes projection computationally tractable and connects projected methods again to the steady-state sensitivity matrix $\nabla_u h(u; w)$.

4.2.6 Projected gradient flow via repeated quadratic programming

The discussion above suggests a practical refinement of projected gradient descent. The exact projection onto

$$\tilde{\mathcal{U}} = \mathcal{U} \cap h^{-1}(\mathcal{Y}; w)$$

is typically hard, because the inverse image of the output constraint through the nonlinear steady-state map is not easy to describe explicitly. However, once a local first-order approximation of this inverse image is available, the projection step can be replaced by the solution of a simple quadratic program.

Assume that the input and output constraint sets are polyhedral:

$$\mathcal{U} := \{u \in \mathbb{R}^m : Au \leq b\}, \qquad \mathcal{Y} := \{y \in \mathbb{R}^p : Cy \leq d\}.$$

As seen above, around the current operating point u we can approximate the output constraint through the linearization

$$h(u'; w) \approx h(u; w) + \nabla_u h(u; w)^\top (u' - u).$$

Therefore, the condition $y \in \mathcal{Y}$, namely

$$Cy \leq d,$$

is locally approximated by

$$C\Big(h(u; w) + \nabla_u h(u; w)^\top (u' - u)\Big) \leq d.$$

This suggests the following idea. Instead of computing the exact Euclidean projection of a gradient step onto the nonlinear feasible set, we search for a correction direction that is as close

as possible to the negative gradient while satisfying the linearized input and output constraints. In this way, each projected step is obtained by solving a quadratic program.

Let

$$g_k := \nabla_u \Phi(u_k, y_k) + \nabla_u h(u_k; w)^\top \nabla_y \Phi(u_k, y_k),$$

where y_k is the measured steady-state output corresponding to the current input u_k. The vector $-g_k$ is the descent direction that would be used in the unconstrained gradient method. Since constraints are present, we cannot move arbitrarily along this direction. We therefore compute a feasible correction δu_k as the solution of the quadratic program

$$\delta u_k := \arg\min_v \ \|v - (-g_k)\|^2$$

subject to

$$A(u_k + \alpha v) \le b,$$
$$C\Big(y_k + \alpha \nabla_u h(u_k; w)^\top v\Big) \le d.$$

The input is then updated according to

$$u_{k+1} = u_k + \alpha \delta u_k.$$

This construction has a clear interpretation. The optimization variable v is a candidate input increment. The quadratic objective forces v to remain as close as possible to the nominal negative gradient direction. The first constraint

$$A(u_k + \alpha v) \le b$$

imposes that the next input remains in the admissible input set $\mathcal{U}$. The second constraint

$$C\Big(y_k + \alpha \nabla_u h(u_k; w)^\top v\Big) \le d$$

is a first-order approximation of the output feasibility condition, centered at the current measurement y_k. So the quadratic program computes the feasible direction that best approximates the desired descent step.

This is why one may speak of *projected gradient flow via repeated quadratic programming*: the projection is no longer performed onto the exact nonlinear feasible set, but onto a local linearized approximation, and this approximate projection is obtained by solving a small QP at each iteration.

Several features of this construction are worth emphasizing.

First, the method uses the measured output y_k rather than the predicted value $h(u_k; w)$ in the constraint linearization. This is consistent with the feedback optimization philosophy: the plant measurement corrects the optimization step online and helps compensate model mismatch in the steady-state map itself.

Second, the Jacobian

$$\nabla_u h(u_k; w)$$

again plays the central role. It provides the local map from input variations to output variations, and therefore determines how output constraints are translated into local linear constraints on the input increment.

Third, the quadratic program is simple. Its cost is quadratic and convex, and its constraints are affine. Hence, under standard regularity conditions, it can be solved efficiently and repeatedly online. This makes the method much more practical than exact projection onto the nonlinear set $\tilde{\mathcal{U}}$.

In summary, the repeated-QP construction provides a tractable approximation of projected gradient descent:

- the unconstrained negative gradient gives the desired descent direction;
- the input and output constraints are enforced through local affine approximations;
- the feasible direction closest to the negative gradient is obtained from a quadratic program;
- the resulting update law preserves the feedback character of the method, because it is based on measured outputs and local steady-state sensitivities.

So the overall controller can be interpreted as a feedback optimization algorithm that repeatedly solves a local QP in order to generate a descent direction compatible with both input and output constraints.

4.3 Duality-based methods

4.3.1 Dual problem

Another important family of first-order methods for constrained optimization is based on *duality*. The idea is to keep the constraints explicitly, but to move them into the objective through Lagrange multipliers. This leads naturally to primal–dual algorithms and dual ascent methods, which are particularly well suited to feedback optimization.

To explain the idea, consider first the generic constrained optimization problem

$$\min_x \ \phi(x) \qquad \text{subject to} \qquad g(x) \le 0,$$

where $g(x) \in \mathbb{R}^q$ collects the inequality constraints.

The associated Lagrangian is

$$\mathcal{L}(x, \lambda) := \phi(x) + \lambda^\top g(x),$$

where the multiplier vector satisfies

$$\lambda \ge 0.$$

The role of λ is to penalize constraint violations: whenever some component of $g(x)$ is positive, the corresponding multiplier increases the Lagrangian cost.

Under suitable convexity assumptions and constraint qualifications, strong duality holds. In that case, the original constrained problem can be written equivalently as

$$\min_{x:\, g(x)\le 0} \phi(x) = \min_x \max_{\lambda \ge 0} \mathcal{L}(x, \lambda) = \max_{\lambda \ge 0} \min_x \mathcal{L}(x, \lambda).$$

This identity is the key reason why dual methods are useful: instead of solving the constrained problem directly, one may solve the saddle-point problem associated with the Lagrangian.

It is worth understanding the first equality. For a fixed x, consider

$$\max_{\lambda \ge 0} \mathcal{L}(x, \lambda) = \max_{\lambda \ge 0} \big(\phi(x) + \lambda^\top g(x)\big).$$

If $g(x) \le 0$, then the maximum is attained at $\lambda = 0$, so

$$\max_{\lambda \ge 0} \mathcal{L}(x, \lambda) = \phi(x).$$

If instead some component of $g(x)$ is positive, then by increasing the corresponding multiplier one obtains

$$\max_{\lambda \ge 0} \mathcal{L}(x, \lambda) = +\infty.$$

So maximizing over the nonnegative multipliers has the effect of enforcing the constraint: infeasible points are assigned infinite cost, while feasible points retain their original objective

value. In this sense, the Lagrangian reformulation replaces the constraint by an indicator-like mechanism.

This viewpoint is particularly useful in feedback optimization, because it leads to algorithms in which the primal variable performs a descent step, while the multiplier performs an ascent step. The multiplier then acts as an internal controller state that stores how strongly the output constraints are being violated.

4.3.2 Primal–dual iterations

The dual reformulation suggests natural iterative schemes for solving constrained optimization problems. Consider again the saddle-point problem

$$\max_{\lambda \geq 0} \min_{x} \mathcal{L}(x, \lambda), \qquad \mathcal{L}(x, \lambda) = \phi(x) + \lambda^\top g(x).$$

A first possibility is to apply gradient descent in the primal variable and gradient ascent in the dual variable. This yields the *primal descent / dual ascent* iteration

$$x_{k+1} = x_k - \alpha \nabla_x \mathcal{L}(x_k, \lambda_k),$$

$$\lambda_{k+1} = \lambda_k + \beta \nabla_\lambda \mathcal{L}(x_k, \lambda_k),$$

followed, if needed, by projection of λ_{k+1} onto the nonnegative orthant in order to preserve the condition $\lambda \geq 0$.

The logic is clear: one tries to decrease the Lagrangian with respect to the primal variable x, while increasing it with respect to the dual variable λ, since the saddle-point problem is a minimization in x and a maximization in λ.

A second possibility is to minimize the Lagrangian exactly with respect to the primal variable at each step, and then update only the multiplier by ascent. This gives the *dual ascent* scheme

$$x_k = \arg\min_{x} \mathcal{L}(x, \lambda_k),$$

$$\lambda_{k+1} = \lambda_k + \beta \nabla_\lambda \mathcal{L}(x_k, \lambda_k).$$

This method can be interpreted as a two-time-scale version of the primal–dual iteration: the primal variable reacts quickly and is assumed to be close to the minimizer of the current Lagrangian, while the multiplier evolves more slowly. In this sense, dual ascent is closely related to saddle-flow ideas with a slow dual dynamics.

A key observation is that

$$\nabla_\lambda \mathcal{L}(x, \lambda) = g(x),$$

because the Lagrangian is affine in the multiplier:

$$\mathcal{L}(x, \lambda) = \phi(x) + \lambda^\top g(x).$$

So the dual update is simply driven by the constraint violation. If the constraint is satisfied with margin, the multiplier tends not to grow; if the constraint is violated, the corresponding component of λ increases. This is exactly the mechanism by which the multiplier enforces feasibility.

4.3.3 Dual ascent for feedback optimization

We now specialize the primal–dual idea to the steady-state optimization problem of feedback optimization. Suppose that the output constraint set can be written as

$$\mathcal{Y} := \{y \in \mathbb{R}^p : \ g(y) \leq 0\},$$

where $g(y) \in \mathbb{R}^q$ collects the inequality constraints on the steady-state output.

Starting from the steady-state optimization problem

$$\min_{u,y} \ \Phi(u,y) \qquad \text{subject to} \qquad u \in \mathcal{U}, \quad y = h(u;w), \quad g(y) \le 0,$$

we form the Lagrangian

$$\mathcal{L}(u,\lambda) := \Phi(u, h(u;w)) + \lambda^\top g(h(u;w)),$$

with

$$\lambda \ge 0.$$

Here the multiplier is associated with the output constraints. It increases the cost whenever the measured or predicted output violates the admissible set.

Applying the primal descent / dual ascent idea gives an iterative controller. The primal variable is the input u, while the dual variable is the multiplier λ. Using measurements for the output and a model only for the steady-state sensitivities, one obtains the update

$$u_{k+1} = u_k - \alpha\Big(\nabla_u \Phi(u_k, y_k) + \nabla_u h(u_k;w)^\top \nabla_y \Phi(u_k, y_k) + \nabla_u h(u_k;w)^\top \nabla g(y_k)^\top \lambda_k\Big),$$

and

$$\lambda_{k+1} = \Pi_{\ge 0}\big[\lambda_k + \beta g(y_k)\big].$$

Here:

- y_k is the measured plant output at iteration k;
- $\nabla_u h(u_k;w)$ is the steady-state sensitivity matrix provided by the model;
- $\Pi_{\ge 0}$ denotes projection onto the nonnegative orthant.

The interpretation of these equations is very important.

The primal update for u_k has the same structure as the gradient-based controllers discussed before, but now there is an extra term,

$$\nabla_u h(u_k;w)^\top \nabla g(y_k)^\top \lambda_k,$$

which comes from the Lagrange multiplier. This term pushes the input in a direction that reduces constraint violations. The stronger the current multiplier λ_k, the stronger the correction induced by the constraint.

The dual update is even more intuitive:

$$\lambda_{k+1} = \Pi_{\ge 0}\big[\lambda_k + \beta g(y_k)\big].$$

If some constraint is violated, then the corresponding component of $g(y_k)$ is positive, and so the corresponding multiplier increases. If instead the constraint is satisfied, the multiplier does not need to grow. The projection ensures that the multipliers remain nonnegative, as required by duality theory.

Once again, the implementation is remarkably light in terms of modeling requirements. The controller does not need a full dynamic model of the plant. It only needs:

- measurements of the current steady-state output y_k;
- the steady-state sensitivity matrix $\nabla_u h(u_k;w)$.

This is fully consistent with the feedback optimization philosophy developed so far.

Role of the multiplier as controller memory. An important conceptual question is: what is the role of the internal variable λ_k? The answer is that λ_k provides *memory* to the controller.

In plain gradient descent, the input update depends only on the current measurement and the current gradient information. In primal–dual control, instead, the multiplier stores information about past constraint violations. If a constraint has been active or violated for several iterations, the corresponding multiplier grows and keeps influencing the future control actions. In this sense, λ_k acts as an internal state that accumulates constraint-related information over time.

This is closely analogous to integral action in regulation problems. Just as an integral state accumulates tracking error in order to remove steady-state offset, the multiplier accumulates constraint violation in order to enforce feasibility. For this reason, the dual variable can be interpreted as a memory state of the feedback optimizer, dedicated specifically to the handling of output constraints.

So the primal–dual controller has a clear structure:

- the primal variable u_k moves in a descent direction for performance optimization;
- the dual variable λ_k integrates constraint violations;
- the interaction between the two produces a feedback law that seeks both optimality and feasibility.

This makes dual ascent and primal–dual iterations a very natural tool for online feedback optimization with constrained steady-state outputs.

4.3.4 An old friend: dual ascent and PI control

The primal–dual viewpoint is not only a mathematical construction. In an important special case, it recovers a very classical feedback controller.

Consider the linear steady-state relation

$$y = Hu + w,$$

where u is the manipulated input, y is the steady-state output, w is a constant exogenous signal, and r is the desired reference. Suppose we want to track the reference while keeping the input effort small. A natural optimization problem is

$$\begin{aligned} \min_{u,y} \quad & \frac{1}{2}\|u\|^2 + \frac{\rho}{2}\|Hu + w - r\|^2 \\ \text{s.t.} \quad & y = r, \\ & y = Hu + w. \end{aligned}$$

Equivalently, the constraint $y = r$ imposes exact tracking, while the term weighted by ρ plays the role of an augmented penalty on the steady-state mismatch. This is an augmented-Lagrangian formulation of the regulation problem.

The corresponding Lagrangian is

$$\mathcal{L}(u, \lambda) = \frac{1}{2}\|u\|^2 + \lambda^\top (Hu + w - r) + \frac{\rho}{2}\|Hu + w - r\|^2.$$

Here the multiplier λ is associated with the tracking constraint

$$Hu + w - r = 0.$$

Let us now apply the dual-ascent idea. As discussed above, one may combine:

- *primal minimization*, by imposing

$$\nabla_u \mathcal{L}(u_k, \lambda_k) = 0,$$

- *dual ascent*, by updating

$$\lambda_{k+1} = \lambda_k + \beta \nabla_\lambda \mathcal{L}(u_k, \lambda_k).$$

Since

$$\nabla_\lambda \mathcal{L}(u_k, \lambda_k) = Hu_k + w - r,$$

the dual update becomes

$$\lambda_{k+1} = \lambda_k + \beta(Hu_k + w - r).$$

If we denote the tracking error by

$$e_k := Hu_k + w - r,$$

this is simply

$$\lambda_{k+1} = \lambda_k + \beta e_k.$$

So the dual variable integrates the tracking error.

To compute the primal step, differentiate the Lagrangian with respect to u:

$$\nabla_u \mathcal{L}(u, \lambda) = u + H^\top \lambda + \rho H^\top (Hu + w - r).$$

Imposing

$$\nabla_u \mathcal{L}(u_k, \lambda_k) = 0$$

gives

$$u_k + H^\top \lambda_k + \rho H^\top (Hu_k + w - r) = 0,$$

that is,

$$u_k = -H^\top \lambda_k - \rho H^\top (Hu_k + w - r).$$

Up to sign conventions in the definition of the tracking error, this is exactly the structure of a proportional–integral controller:

- the term depending directly on the current error

$$H^\top (Hu_k + w - r)$$

  is a proportional action;

- the multiplier λ_k, updated by accumulation of the error, provides the integral action.

So dual ascent recovers a familiar control architecture: a PI controller appears as the primal–dual solution of a steady-state constrained optimization problem. This is a very useful interpretation. It shows that classical feedback structures can be understood as optimization algorithms, and conversely that dual variables in feedback optimization play the same role as integral states in regulation.

In this sense, the multiplier is indeed an "old friend": it is nothing but an integral memory that accumulates constraint or tracking error over time. This is why primal–dual methods often inherit the robustness properties usually associated with integral action, in particular the ability to remove constant steady-state offsets.

4.4 Design insights

4.4.1 Design guidelines and tradeoffs

We can now summarize the main design options discussed so far for online feedback optimization. The three classes of methods considered in this chapter are:

- penalty-based gradient methods;
- projected-gradient methods;

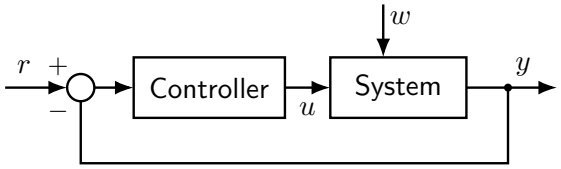

Figure 11: Dual-ascent interpretation of a classical feedback loop. For the linear steady-state relation $y = Hu + w$, primal minimization combined with dual ascent leads to a control law with a proportional term acting on the current tracking error and an integral term represented by the dual variable λ_k.

- primal–dual or saddle-flow methods.

All three rely on the same basic ingredients:

- a fast and stable plant, so that the steady-state map is the relevant object;
- some model information in the form of steady-state sensitivities

$$\nabla_u h(u; w);$$

- output measurements y, which provide the feedback signal correcting the optimization step online.

What changes is how the constraints are handled, and this leads to different practical tradeoffs.

Penalty methods. Penalty methods incorporate the output constraint into the objective. Their main advantage is simplicity: one obtains a smooth unconstrained optimization law, often easy to implement and analyze. Moreover, constraint violation can be made arbitrarily small by choosing a sufficiently strong penalty. However, the violation is not removed exactly, only discouraged. In this sense, the behavior is essentially *proportional*: the correction grows with the violation, but there is no internal memory dedicated to enforcing exact feasibility. A second drawback is that a very steep penalty slows down the optimization dynamics and may lead to poor numerical conditioning or conservative steps.

Projected-gradient methods. Projected-gradient methods enforce feasibility more directly. By projecting the update onto an admissible set, they can guarantee *any-time feasibility*, at least with respect to the approximated constraint set used in the projection. This is very attractive when violating constraints is unacceptable. The price to pay is that one must solve a projection problem, which in practice often becomes a quadratic program. So feasibility is improved, but the controller is computationally heavier. In addition, the admissible stepsize is typically limited by the geometry of the projection and by the accuracy of the local approximation.

Saddle-flow or primal–dual methods. Primal–dual methods handle the constraints through dual variables. Their main advantage is that feasibility is enforced asymptotically through the multiplier dynamics. The dual variable acts as an integral memory, which is why these methods are often reminiscent of proportional–integral control. This makes them conceptually very appealing and often robust. On the other hand, because of the coupled primal and dual dynamics, oscillations may arise, especially if the gains are not tuned carefully or if the time scales of plant and optimizer are not sufficiently separated.

So the three approaches can be summarized as follows:

- **Penalty:** small violation allowed, simple structure, but steep penalties reduce speed;
- **Projected gradient:** feasibility enforced at every iteration, but requires repeated projection or quadratic programming;

- **Saddle flow / primal–dual:** feasibility obtained asymptotically through an integral-like mechanism, but oscillations may appear.

There is therefore no universally best design. The choice depends on the application requirements:

- if small temporary violations are acceptable and simplicity is important, penalty methods are attractive;
- if feasibility must be guaranteed at all times, projected-gradient methods are preferable;
- if one wants a dynamic mechanism that drives the system asymptotically to the constrained optimum, primal–dual methods are natural.

This comparison also clarifies the relation with classical control intuition:

- penalty methods behave in a mainly proportional way;
- primal–dual methods introduce an integral state through the multiplier;
- projected-gradient methods correspond instead to explicit feasibility correction through projection.

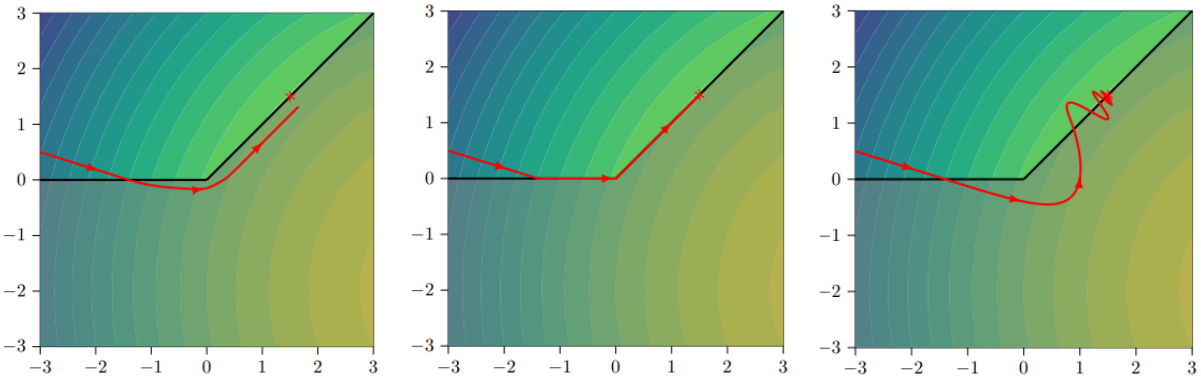

Figure 12: Qualitative comparison of the trajectories produced by penalty, projected-gradient, and saddle-flow methods. Penalty methods may tolerate small constraint violations, projected-gradient methods maintain feasibility through projection, and saddle-flow methods approach feasibility asymptotically but may exhibit oscillatory behavior.

In all cases, the same general message remains valid: feedback optimization does not require a full dynamical model of the plant, but it does require the right steady-state information. The central ingredients are the output measurements and the steady-state sensitivity matrix

$$\nabla_u h(u; w),$$

which together make it possible to embed optimization algorithms directly into the feedback loop.

4.5 Extremum seeking

4.5.1 Extremum seeking

So far, all feedback-optimization schemes relied on at least some approximate knowledge of the steady-state sensitivities

$$\nabla_u h(u; w),$$

or, equivalently, of the gradient of the performance map with respect to the input. In many applications, however, this information may be unavailable or too unreliable to be useful.

This happens, for instance, when the plant is poorly modeled, when the operating conditions vary significantly over time, or when the steady-state map is known only through empirical performance charts.

This raises a natural question: can we optimize a steady-state performance index without any explicit knowledge of the gradient? Extremum-seeking control answers this question positively. It is a class of *zeroth-order* optimization methods, meaning that it seeks the optimizer by using only measurements of the cost or performance, without requiring an analytical model of its gradient.

The general setting is the following. We consider again a fast stable plant, and we associate to each constant input u a scalar performance metric

$$J(u),$$

which we want to minimize or maximize. The key difficulty is that the gradient

$$\nabla_u J(u)$$

is unknown. Extremum seeking overcomes this difficulty by injecting a small oscillatory perturbation into the input, observing the corresponding oscillatory component in the measured performance, and extracting from it an estimate of the local gradient.

This is why extremum seeking is often regarded as a learning-based feedback optimizer: it does not need a model of the steady-state sensitivities, because it estimates the necessary descent information online from measured data. In this sense, it complements the methods studied earlier in the chapter. When sensitivities are available, gradient, projected-gradient, or primal–dual methods are natural. When sensitivities are unknown, one may either estimate them or switch to a purely zeroth-order approach such as extremum seeking.

It is worth stressing one important robustness point. Even when the available sensitivity model is very inaccurate, the feedback-optimization schemes discussed previously may still work reasonably well. However, if one wants to avoid relying on such information altogether, extremum seeking is a natural alternative. This is one of the main motivations for introducing it at the end of the chapter.

Exercise. A useful fact to check is the following: both projected gradient descent and saddle-flow methods cannot have an equilibrium corresponding to an infeasible steady state

$$y \notin \mathcal{Y}.$$

This reinforces the idea that feedback optimization is structurally tied to feasibility. Extremum seeking, on the other hand, addresses a different issue: not the lack of feasibility mechanisms, but the lack of gradient information.

4.5.2 Extremum-seeking architecture

The simplest extremum-seeking idea can be explained for a scalar input. Suppose that the plant is fast and stable, and that the measured steady-state performance is $J(u)$. We want to optimize $J(u)$ without knowing its gradient.

The controller is built around four basic ingredients:

- an *exploration signal*, which perturbs the input with a small sinusoid;
- a *high-pass filter*, which removes the slowly varying offset in the measured performance;
- a *demodulation step*, which multiplies the filtered performance by a sinusoidal signal at the same frequency;

- a *low-pass filter* followed by an integrator, which extracts an average descent direction and updates the nominal input.

Denoting by $\hat{u}_k$ the slowly varying nominal input and by $\rho \sin(\omega k)$ the injected perturbation, the actual plant input is

$$u_k = \hat{u}_k + \rho \sin(\omega k).$$

So the plant is continuously excited around the nominal operating point. The purpose of this small oscillation is to probe the local slope of the performance function.

The measured performance $J(u_k)$ is then passed through a high-pass filter, which removes the average component and keeps the oscillatory part induced by the perturbation. This signal is subsequently multiplied by a sinusoid of the same frequency. This multiplication step is often called *demodulation*, because it extracts the component of the output that is correlated with the injected exploration signal. Finally, a low-pass filter removes the high-frequency terms, leaving only a slowly varying approximation of the gradient. The resulting signal is then integrated to update the nominal input $\hat{u}_k$ in the direction of improvement.

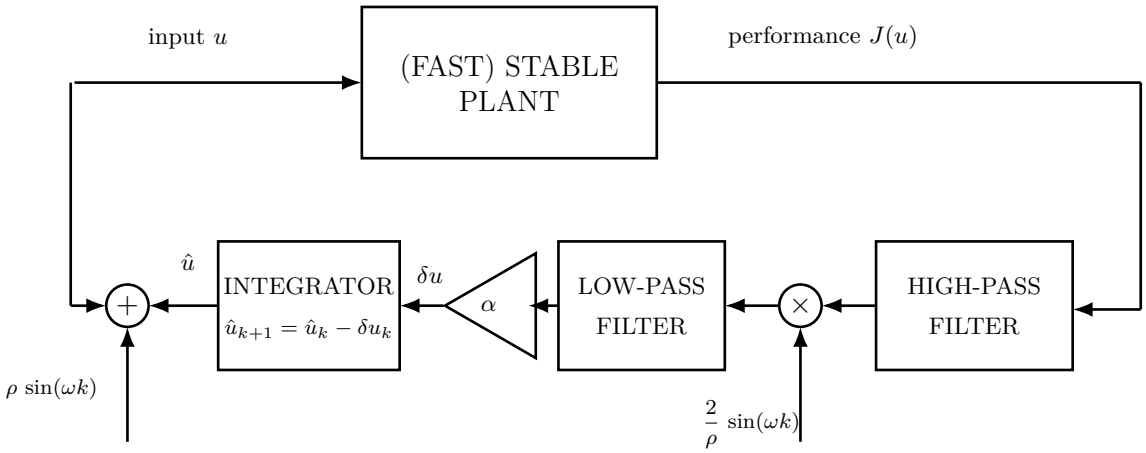

Figure 13: Basic extremum-seeking architecture. A small sinusoidal perturbation is added to the nominal input $\hat{u}_k$ in order to explore the local slope of the steady-state performance $J(u)$. The measured performance is high-pass filtered, demodulated, low-pass filtered, and finally integrated so as to generate an update of the nominal input.

The exploration signal

$$\rho \sin(\omega k)$$

should be thought of as a deliberate excitation used to reveal local gradient information. The demodulation signal, typically proportional to

$$\frac{2}{\rho} \sin(\omega k),$$

is chosen so that, after averaging, the product isolates the first-order term of the Taylor expansion of the performance function.

So the overall architecture can be interpreted as a feedback mechanism that reconstructs a gradient estimate from input–output oscillations, without ever requiring an explicit model of the plant or of the performance map.

4.5.3 Why extremum seeking behaves like gradient descent

The basic mechanism can be understood with a first-order Taylor expansion. Starting from

$$u_k = \hat{u}_k + \rho \sin(\omega k),$$

assume that ρ is small, so that the performance function can be expanded around the nominal input:

$$J(u_k) \approx J(\hat{u}_k) + \nabla_u J(\hat{u}_k)\, \rho \sin(\omega k).$$

The first term is a slowly varying offset, while the second is an oscillatory term whose amplitude is proportional to the unknown gradient.

After the high-pass filter, the offset is removed, and we are left approximately with

$$\nabla_u J(\hat{u}_k)\, \rho \sin(\omega k).$$

The next step is demodulation. Multiplying by

$$\frac{2}{\rho} \sin(\omega k)$$

gives

$$\nabla_u J(\hat{u}_k)\, 2\sin^2(\omega k).$$

Using the identity

$$2\sin^2(\omega k) = 1 - \cos(2\omega k),$$

we see that this signal consists of a constant part,

$$\nabla_u J(\hat{u}_k),$$

plus a fast oscillation at frequency 2ω. The low-pass filter removes the oscillatory component and keeps only the average value. Therefore, after the low-pass filter, the signal is approximately

$$\nabla_u J(\hat{u}_k).$$

If this signal is then multiplied by a gain α, we obtain the update direction

$$\delta u_k = \alpha \nabla_u J(\hat{u}_k).$$

Finally, the integrator implements

$$\hat{u}_{k+1} = \hat{u}_k - \delta u_k,$$

hence

$$\hat{u}_{k+1} = \hat{u}_k - \alpha \nabla_u J(\hat{u}_k).$$

But this is exactly the gradient-descent iteration.

So, at an intuitive level, extremum seeking is nothing but gradient descent in disguise:

- the exploration signal reveals the local slope of the performance map;
- the filtering and demodulation reconstruct the gradient;
- the integrator performs the actual descent step.

Of course, this derivation is only a sketch. A rigorous analysis requires averaging arguments, time-scale separation, and assumptions on the filters and on the plant dynamics. But the basic idea is exactly the one above: extremum seeking creates its own gradient information through oscillatory probing.

4.5.4 Applications of extremum seeking

Extremum seeking is especially attractive in applications with the following features:

- the plant is fast and stable at the time scale of the optimizer;
- the steady-state performance map is poorly known or strongly parameter dependent;
- the optimization problem is low dimensional;
- small oscillatory perturbations of the input are acceptable.

A classical example is photovoltaic maximum-power-point tracking. In a photovoltaic system, the active power generated by the panel depends on the operating voltage, and the power–voltage curve changes significantly with solar irradiance and temperature. Therefore, the location of the optimal voltage is not fixed and may vary continuously during operation. Moreover, an accurate model of the full dependence on the environmental variables may be difficult to maintain online.

In this situation, extremum seeking is very natural. By slightly perturbing the voltage and measuring the resulting power, the controller can estimate whether the current operating point is to the left or to the right of the maximum, and then move the nominal voltage in the correct direction. In this way, the controller tracks the operating voltage that maximizes active power generation without requiring an explicit gradient model.

This is a particularly good use case because:

- the optimum varies with exogenous conditions;
- the model is poor or uncertain;
- the control input is low dimensional;
- the actuator cost of a small oscillatory probing signal is acceptable.

Other relevant applications include internal combustion engines, chemical processes, compressor systems, and meta-optimization problems such as tuning controller parameters. In all these cases, extremum seeking is valuable when the optimizer can rely on measured performance only, while the gradient or the steady-state sensitivity is unavailable.

Final perspective. This closes the chapter with an important conceptual message. Feedback optimization can be organized along a spectrum:

- if reliable steady-state sensitivities are available, one can use gradient, projected-gradient, or primal–dual schemes;
- if sensitivities are inaccurate, one may still rely on robust feedback optimization based on approximate models;
- if sensitivities are completely unknown, one can move to learning or zeroth-order methods such as extremum seeking.

So extremum seeking is not a separate topic disconnected from the rest. It is the natural endpoint of the feedback-optimization viewpoint: when model-based gradient information disappears, the controller can still recover optimization directions directly from feedback measurements.

5 System Identification

So far, the control methods that we have studied, in particular LQR and Model Predictive Control, have relied on the availability of a state-space model of the plant. In discrete time, this is typically written as

$$x_{t+1} = Ax_t + Bu_t, \qquad y_t = Cx_t,$$

or, more generally, with a nonlinear update map. This representation is extremely convenient for control design, because it provides a Markovian description of the system: once the current state is known, the future evolution depends only on the present state and on the input that will be applied. In this sense, the state is the variable that summarizes all the information from the past that is relevant for predicting the future.

Having such a model, however, means having a very detailed description of the plant. One must know the parameters that govern the state update, the way in which the input affects the state evolution, and the way in which the internal state is mapped into the measured outputs. Even more fundamentally, one must decide what the state actually is, that is, which variables are sufficient to obtain a Markovian model. This is already a nontrivial modeling choice, especially when the internal physical variables are not directly measurable.

A first important observation is that a state-space representation is not unique. Different internal coordinates may describe exactly the same external input-output behavior. Indeed, if we introduce a new state variable w_t through an invertible linear change of coordinates

$$x_t = Tw_t,$$

with T nonsingular, then the system can be equivalently written as

$$w_{t+1} = T^{-1}ATw_t + T^{-1}Bu_t, \qquad y_t = CTw_t.$$

Although the matrices appearing in the model have changed, the input-output trajectories remain exactly the same. Therefore, the state should not be interpreted as a uniquely defined physical object, but rather as an internal variable that makes prediction and control possible. What really matters from the external viewpoint is the input-output behavior of the system.

This point is one of the main motivations for system identification. If different state-space models can generate the same measured behavior, then reconstructing a model from data cannot be understood as recovering the "true" internal realization in an absolute sense. Instead, the goal is to build a model that reproduces the observed dynamics well enough for the intended task. In our context, this task is control. Therefore, the identified model should be sufficiently accurate to predict the future effect of the control inputs and to support the synthesis of model-based controllers.

System identification addresses exactly this problem. Rather than deriving the model entirely from first principles, we use measured input-output data to infer a dynamical description of the plant. This is especially useful when a first-principles model is unavailable, too expensive to derive, or too inaccurate for control purposes. The identified model may be a state-space realization, an input-output model, or another predictive representation, depending on the assumptions and on the control architecture of interest.

From this viewpoint, identification can be seen as the bridge between data and model-based control. On the one hand, advanced controllers such as LQR and MPC require a predictive model. On the other hand, the model need not be known a priori: it can be estimated from experiments. The central question of this chapter is therefore how to extract, from measured data, a model that captures the relevant dynamical behavior of the plant and can be reliably used for prediction and control.

A natural way to move from an internal state-space description to an external input-output description is to write the output as a function of the initial condition and of the past inputs.

For the discrete-time linear system

$$x_{t+1} = Ax_t + Bu_t, \qquad y_t = Cx_t,$$

repeated substitution gives

$$y_t = CA^t x_0 + \sum_{j=0}^{t-1} CA^{t-1-j} B\, u_j.$$

This expression clearly separates two contributions. The term $CA^t x_0$ is the response generated by the initial state, while the summation describes how past inputs are propagated to the output through the dynamics. The latter has the form of a discrete-time convolution and shows that, for linear time-invariant systems, the full input-output behavior is encoded in the sequence of matrices

$$CA^{t-1}B, \qquad t > 0.$$

In the SISO case, where both $u_t \in \mathbb{R}$ and $y_t \in \mathbb{R}$, this sequence reduces to a scalar sequence

$$h_t = CA^{t-1}B, \qquad t > 0,$$

which is the impulse response of the system. If the initial condition is zero, then the output produced by an arbitrary input sequence is completely determined by the convolution of the input with $\{h_t\}_{t\geq 1}$. In this sense, the impulse response is a complete representation of the zero-state input-output map of the plant. It contains exactly the information needed to predict future outputs from past inputs, without explicitly referring to the internal state coordinates. This is especially important in identification, because the state itself is not directly unique, whereas the input-output behavior is the physically observable object.

The same idea extends immediately to MIMO systems. If the plant has m inputs and p outputs, then each coefficient

$$H_t = CA^{t-1}B \in \mathbb{R}^{p\times m}, \qquad t > 0,$$

is a matrix. Its entry in position (i, j) represents the response of output i to a unit impulse applied on input j. The sequence $\{H_t\}_{t\geq 1}$ is called the sequence of Markov parameters of the system. These parameters generalize the impulse response and provide a compact description of the input-output behavior of the linear system. Once they are known, the output can be reconstructed from the past inputs through matrix convolution.

This viewpoint is also attractive from an experimental perspective. On a real plant, the Markov parameters can be estimated directly from data by exciting the system and measuring the corresponding outputs. In the ideal noise-free case, one can initialize the plant at zero and apply a unit impulse to one input channel at a time, while keeping the others equal to zero. For a MIMO system, this means that m experiments are sufficient in principle: each experiment isolates one column of the matrices H_t, and by repeating the procedure for all inputs one reconstructs the full sequence of Markov parameters. In practice, however, perfect impulses cannot be generated and measurements are affected by noise, disturbances, and possible offsets in the initial condition. For this reason, identification methods usually rely on richer input signals and estimate the Markov parameters from batches of input-output data, rather than from a single idealized impulse experiment.

The key message is that, before identifying a state-space realization, one may already identify the external behavior of the plant through its impulse response, or equivalently through its Markov parameters. This provides the first bridge between measured data and predictive models, and it motivates the identification techniques developed in the rest of this chapter.

5.1 Controllability, observability, and minimal realizations

When the goal is to identify a state-space model from input-output data, not every realization is equally meaningful. Since the state is not unique, one can always introduce additional coordinates that do not change the external behavior of the system. For identification and control, however, we are interested in realizations in which the state has an actual dynamical role. This naturally leads to the notions of controllability and observability, which characterize whether the state can be influenced by the input and reconstructed from the output.

For the discrete-time linear system

$$x_{t+1} = Ax_t + Bu_t, \qquad y_t = Cx_t,$$

controllability concerns the possibility of steering the system from a given initial condition to a desired target state. To fix ideas, let us start from $x_0 = 0$. After one step, the state is

$$x_1 = Bu_0,$$

so the set of states reachable in one step is exactly the image of B. After two steps,

$$x_2 = ABu_0 + Bu_1,$$

hence the reachable set is the image of the matrix $[\,B \;\; AB\,]$. Continuing in the same way, after n steps the state belongs to the image of

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix},$$

which is called the controllability matrix. Therefore, the system is controllable precisely when every state in $\mathbb{R}^n$ can be reached, that is, when

$$\operatorname{rank}(\mathcal{C}) = n.$$

This condition means that the columns generated by repeatedly propagating the input directions through the dynamics span the whole state space. It is worth stressing that n steps are enough to decide controllability: if the full state space has not been reached by then, taking more steps cannot generate new independent directions.

The controllability matrix also provides a constructive way to compute a control sequence that reaches a desired state. Indeed, for a target $\bar{x}$ reached in n steps, one can write

$$x_n = \mathcal{C} \begin{bmatrix} u_{n-1} \\ \vdots \\ u_0 \end{bmatrix}.$$

If $\mathcal{C}$ is square and invertible, which is possible in particular in the scalar-input case, the input sequence is obtained directly by inversion. In the more general case, $\mathcal{C}$ is not square, and the problem may admit multiple solutions. A standard choice is then the minimum-norm solution, which can be computed through the pseudoinverse.

Dual to controllability is observability. While controllability asks whether the input can excite all internal directions, observability asks whether the effect of the initial state can be detected from the output. Starting again from the linear model, at time $t = 0$ one has

$$y_0 = Cx_0,$$

so any initial state in the kernel of C is invisible at the output at that instant. After one step,

$$y_1 = CAx_0,$$

hence a state that belongs simultaneously to the kernels of C and CA remains indistinguishable over the first two output samples. Proceeding recursively, after n steps the invisible states are those in the kernel of

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix},$$

which is called the observability matrix. The system is observable if and only if

$$\operatorname{rank}(\mathcal{O}) = n.$$

Equivalently, distinct initial states always generate distinct output trajectories over a finite observation window. Also here, n steps are sufficient: if a state cannot be distinguished from the output sequence of length n, it will remain unobservable even if more measurements are collected.

As in the controllability case, observability has a constructive interpretation. Given an output sequence $y_0, \dots, y_{n-1}$, one can write

$$\begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix} = \mathcal{O} x_0.$$

If $\mathcal{O}$ is invertible, the initial state is uniquely determined. More generally, when $\mathcal{O}$ has full column rank, the state can still be reconstructed uniquely, for instance through a least-squares inversion based on the pseudoinverse.

These two notions are central in system identification because they single out the realizations that are meaningful from input-output data. If a realization contains uncontrollable states, part of the state cannot be influenced by the input and is therefore irrelevant for prediction and control. If it contains unobservable states, part of the state has no effect on the measured output and therefore cannot be inferred from data. In both cases, the realization contains redundant internal coordinates. For this reason, in identification we focus on realizations that are both controllable and observable. Such realizations are called minimal, because they do not contain superfluous states and represent the input-output behavior with the smallest possible state dimension.

A simple example illustrates the point. Consider a plant whose output is a moving average of the past three inputs,

$$y_t = a_1 u_{t-1} + a_2 u_{t-2} + a_3 u_{t-3}.$$

A natural state is obtained by storing delayed copies of the input:

$$x_t = \begin{bmatrix} u_{t-1} \\ u_{t-2} \\ u_{t-3} \end{bmatrix}.$$

With this choice, the dynamics become

$$x_{t+1} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} x_t + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} u_t, \qquad y_t = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} x_t.$$

This realization has an immediate interpretation: the state is simply a memory of the past input values needed to compute the current output.

The controllability matrix of this realization is

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B \end{bmatrix} = I,$$

so the system is controllable. This is consistent with the physical meaning of the state, since the input directly writes the first memory cell and, through the shift dynamics, can generate any delayed input profile. The observability matrix is

$$\mathcal{O} = \begin{bmatrix} a_1 & a_2 & a_3 \\ a_2 & a_3 & 0 \\ a_3 & 0 & 0 \end{bmatrix}.$$

Hence observability depends on the coefficients a_1, a_2, a_3. When this matrix has full rank, the three stored delays all leave a visible signature in the output and the realization is minimal. If instead the rank is smaller, then some component of the chosen state does not affect the output independently, which means that a lower-order realization exists.

This example clarifies why controllability and observability are so important in identification. Starting from input-output data, one does not want to recover an arbitrary realization, but rather a realization whose state is both reachable from the input and visible from the output. Only in this case does the state provide a well-posed internal representation of the external behavior of the plant, and only in this case can we hope to obtain a compact and useful model for control design.

5.2 Realization from data and the Ho–Kalman construction

The discussion so far has shown that the input-output behavior of a linear time-invariant system is completely described by its Markov parameters

$$\{CB,\ CAB,\ CA^2B,\ \dots\}.$$

This naturally leads to the realization problem: if these parameters are known from experiments, how can one reconstruct a state-space model

$$x_{t+1} = Ax_t + Bu_t, \qquad y_t = Cx_t$$

that generates them? More generally, given generic input-output data, how can one construct a realization A, B, C consistent with the observed dynamics? The goal is not to recover a unique internal state, since different state-space models may have the same external behavior, but rather to compute a minimal realization, that is, one that is both controllable and observable.

The key object in this construction is the Hankel matrix built from the Markov parameters. If we define the observability and controllability matrices of a minimal realization as

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}, \qquad \mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix},$$

then their product is

$$\mathcal{H} = \mathcal{O}\mathcal{C}.$$

Expanding this product shows that every block of $\mathcal{H}$ is one of the Markov parameters:

$$\mathcal{H} = \begin{bmatrix} CB & CAB & CA^2B & \cdots & CA^{n-1}B \\ CAB & CA^2B & CA^3B & \cdots & CA^nB \\ CA^2B & CA^3B & CA^4B & \cdots & CA^{n+1}B \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ CA^{n-1}B & CA^nB & CA^{n+1}B & \cdots & CA^{2n-2}B \end{bmatrix}.$$

Thus the Hankel matrix can be assembled directly from experimental data as soon as enough Markov parameters are available. This is the crucial bridge between input-output measurements and state-space realizations.

To make the structure visually apparent, it is often helpful to highlight repeated Markov parameters along the anti-diagonals. For example, one may write

$$\mathcal{H} = \begin{bmatrix} CB & CAB & CA^2B & \cdots & CA^{n-1}B \\ CAB & CA^2B & CA^3B & \cdots & CA^nB \\ CA^2B & CA^3B & CA^4B & \cdots & CA^{n+1}B \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ CA^{n-1}B & CA^nB & CA^{n+1}B & \cdots & CA^{2n-2}B \end{bmatrix}.$$

The repeated colors emphasize that each anti-diagonal contains the same Markov parameter. This is exactly the signature of a Hankel structure: the matrix is constant along anti-diagonals, and those constants are the impulse-response coefficients of the system.

The Ho–Kalman algorithm uses this structure in three conceptual steps. First, one constructs the Hankel matrix from the measured Markov parameters. Second, one factorizes the Hankel matrix as

$$\mathcal{H} = \mathcal{O}\mathcal{C}.$$

Third, one extracts a realization A, B, C from the factors $\mathcal{O}$ and $\mathcal{C}$.

At first sight, the first step raises a basic difficulty: how many Markov parameters are needed? The answer depends on the order n of the minimal realization, which is unknown a priori. However, if the realization is minimal, then both $\mathcal{O}$ and $\mathcal{C}$ have rank n, and therefore the Hankel matrix also has rank n. This means that, in principle, one can enlarge the Hankel matrix progressively until its rank stops increasing. That plateau reveals the order of the system. In noise-free settings this idea works exactly; in practice, with noisy data, the same principle survives in approximate form and the effective rank is inferred numerically.

This rank-based interpretation is easy to understand on simple examples. A first-order low-pass filter has state-space model

$$x_{t+1} = ax_t + u_t, \qquad y_t = x_t,$$

so its Markov parameters are $1, a, a^2, \ldots$, and the corresponding Hankel matrix has rank one. By contrast, a frictionless cart model written as

$$v_{t+1} = v_t + u_t, \qquad z_{t+1} = z_t + v_t, \qquad y_t = z_t$$

has a two-dimensional state, and the Hankel matrix reflects this higher order through rank two. In this way, the rank of the Hankel matrix captures the intrinsic dimension of the minimal realization.

The second step of the Ho–Kalman method is the factorization

$$\mathcal{H} = \mathcal{O}\mathcal{C}.$$

An important conceptual point is that this factorization is not unique, but this is not a problem. Indeed, nonuniqueness of the factorization simply reflects nonuniqueness of the state coordinates. If we apply a change of basis

$$\tilde{A} = T^{-1}AT, \qquad \tilde{B} = T^{-1}B, \qquad \tilde{C} = CT,$$

with T invertible, then the corresponding observability and controllability matrices become

$$\tilde{\mathcal{O}} = \begin{bmatrix} \tilde{C} \\ \tilde{C}\tilde{A} \\ \tilde{C}\tilde{A}^2 \\ \vdots \end{bmatrix} = \begin{bmatrix} CT \\ CTAT^{-1}T \\ CTA^2T^{-1}T \\ \vdots \end{bmatrix} = \mathcal{O}T,$$

and similarly

$$\tilde{\mathcal{C}} = T^{-1}\mathcal{C}.$$

Therefore,

$$\tilde{\mathcal{O}}\tilde{\mathcal{C}} = \mathcal{O}TT^{-1}\mathcal{C} = \mathcal{O}\mathcal{C} = \mathcal{H}.$$

So different factorizations of $\mathcal{H}$ correspond exactly to different coordinate representations of the same input-output system. The factorization does not identify a unique realization, but it identifies the system up to similarity transformation, which is the correct notion of uniqueness for state-space models.

Because of this, the factorization step leaves some freedom. In principle, one could take trivial choices such as $\mathcal{O} = I$ and $\mathcal{C} = \mathcal{H}$, or $\mathcal{O} = \mathcal{H}$ and $\mathcal{C} = I$, but these choices are not useful numerically and do not directly reveal a well-scaled state representation. In practice, one prefers factorizations based on the singular value decomposition, because they expose the rank of the Hankel matrix and provide numerically robust coordinates for the state.

Once a factorization $\mathcal{H} = \mathcal{O}\mathcal{C}$ has been obtained, the realization matrices are recovered from the shift structure of $\mathcal{O}$ and $\mathcal{C}$. The matrix C is simply the first block row of $\mathcal{O}$, while B is the first block column of $\mathcal{C}$. The state transition matrix A is then determined by exploiting the fact that consecutive block rows of $\mathcal{O}$ differ by multiplication by A, or equivalently that consecutive block columns of $\mathcal{C}$ differ by the same shift. In this way, the whole state-space realization is reconstructed from the Hankel matrix, and therefore from measured input-output behavior alone.

The significance of the Ho–Kalman construction is that it turns impulse-response data into a minimal state-space model through linear-algebraic operations. It provides one of the cleanest answers to the realization problem: first identify the Markov parameters, then organize them into a Hankel matrix, then factorize this matrix to uncover the underlying controllable and observable state structure.

5.3 SVD-based Ho–Kalman realization

The key missing ingredient in the Ho–Kalman construction is a numerically reliable way to factorize the Hankel matrix. This is provided by the singular value decomposition. If $H \in \mathbb{R}^{m\times n}$ has rank r, then it can be written as

$$H = U\Sigma V^\top,$$

where $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ are orthogonal matrices, while $\Sigma \in \mathbb{R}^{m\times n}$ is diagonal in the rectangular sense, with nonnegative singular values on the diagonal. If the rank of H is r, then only the first r singular values are nonzero, and the decomposition can be partitioned as

$$H = \begin{bmatrix} U_1 & U_2 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_1 & V_2 \end{bmatrix}^\top = U_1\Sigma_1 V_1^\top,$$

where $\Sigma_1 \in \mathbb{R}^{r\times r}$ contains the nonzero singular values. This decomposition is one of the most useful tools in numerical linear algebra: it is computationally efficient, numerically robust, and it reveals directly the rank and the dominant subspaces of the matrix. In particular, the columns of U_1 span the image of H, while the dimension of Σ_1 identifies the rank.

This is exactly the type of factorization we need for the Hankel matrix. Instead of building a square Hankel matrix of unknown dimension, it is convenient to introduce an extended Hankel matrix

$$\mathcal{H}_{k,\ell} = \begin{bmatrix} G_1 & G_2 & \cdots & G_\ell \\ G_2 & G_3 & \cdots & G_{\ell+1} \\ G_3 & G_4 & \cdots & G_{\ell+2} \\ \vdots & \vdots & \ddots & \vdots \\ G_k & G_{k+1} & \cdots & G_{k+\ell-1} \end{bmatrix},$$

where $G_t = CA^{t-1}B$ denotes the t-th Markov parameter. Here k and ℓ are chosen larger than the unknown system order n. If the data are generated by an n-dimensional minimal realization, then this extended Hankel matrix factors as

$$\mathcal{H}_{k,\ell} = \mathcal{O}_k \mathcal{C}_\ell,$$

with

$$\mathcal{O}_k = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{k-1} \end{bmatrix}, \qquad \mathcal{C}_\ell = \begin{bmatrix} B & AB & A^2B & \cdots & A^{\ell-1}B \end{bmatrix}.$$

These are the extended observability and controllability matrices. As soon as $k \geq n$ and $\ell \geq n$, both matrices have rank n, and therefore $\mathcal{H}_{k,\ell}$ also has rank n. This is the fundamental reason why one chooses k and ℓ larger than the system order: once the horizon is long enough, the rank of the Hankel matrix reveals the true dimension of the minimal realization.

At this point, the SVD gives a natural and balanced factorization of $\mathcal{H}_{k,\ell}$. If

$$\mathcal{H}_{k,\ell} = U_1 \Sigma_1 V_1^\top,$$

where Σ_1 contains the nonzero singular values, then the order of the system is simply

$$n = \operatorname{rank}(\mathcal{H}_{k,\ell}) = \dim(\Sigma_1).$$

Moreover, we may decompose the Hankel matrix as

$$\mathcal{H}_{k,\ell} = \underbrace{U_1 \Sigma_1^{1/2}}_{\mathcal{O}_k} \underbrace{\Sigma_1^{1/2} V_1^\top}_{\mathcal{C}_\ell}.$$

This choice is especially convenient because it distributes the scaling symmetrically between the observability and controllability factors. In this way we obtain numerical representatives of the extended observability and controllability matrices:

$$\mathcal{O}_k = U_1 \Sigma_1^{1/2}, \qquad \mathcal{C}_\ell = \Sigma_1^{1/2} V_1^\top.$$

From these matrices, C and B are obtained immediately. Indeed, C is the first block row of $\mathcal{O}_k$, while B is the first block column of $\mathcal{C}_\ell$. Visually, one can emphasize this as

$$\mathcal{O}_k = \begin{bmatrix} \boxed{C} \\ CA \\ CA^2 \\ \vdots \\ CA^{k-1} \end{bmatrix}, \qquad \mathcal{C}_\ell = \begin{bmatrix} \boxed{B} & AB & A^2B & \cdots & A^{\ell-1}B \end{bmatrix}.$$

Thus the first visible block of the observability factor gives the output matrix, and the first visible block of the controllability factor gives the input matrix.

To recover the state transition matrix A, one introduces the shifted Hankel matrix

$$\mathcal{H}_{k,\ell}^{\uparrow} = \begin{bmatrix} G_2 & G_3 & \cdots & G_{\ell+1} \\ G_3 & G_4 & \cdots & G_{\ell+2} \\ G_4 & G_5 & \cdots & G_{\ell+3} \\ \vdots & \vdots & \ddots & \vdots \\ G_{k+1} & G_{k+2} & \cdots & G_{k+\ell} \end{bmatrix}.$$

By construction, this matrix satisfies

$$\mathcal{H}^{\uparrow}_{k,\ell} = \begin{bmatrix} CAB & CA^2B & CA^3B & \cdots \\ CA^2B & CA^3B & CA^4B & \cdots \\ CA^3B & CA^4B & CA^5B & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} = \mathcal{O}_k A \mathcal{C}_\ell.$$

Once $\mathcal{O}_k$ and $\mathcal{C}_\ell$ are known, this relation can be inverted in the least-squares sense, yielding

$$A = \mathcal{O}_k^{\dagger} \mathcal{H}^{\uparrow}_{k,\ell} \mathcal{C}_\ell^{\dagger},$$

where † denotes the pseudoinverse. Therefore the full realization (A, B, C) is reconstructed directly from the measured Markov parameters.

Putting everything together, the Ho–Kalman procedure can be summarized as follows. First, one collects impulse-response data and estimates the Markov parameters of the system. Then one chooses k, ℓ sufficiently large and builds the extended Hankel matrix $\mathcal{H}_{k,\ell}$ together with its shifted version $\mathcal{H}^{\uparrow}_{k,\ell}$. Next, one computes the SVD of $\mathcal{H}_{k,\ell}$, uses the nonzero singular values to determine the system order n, and defines the extended observability and controllability matrices through the balanced factorization

$$\mathcal{O}_k = U_1 \Sigma_1^{1/2}, \qquad \mathcal{C}_\ell = \Sigma_1^{1/2} V_1^{\top}.$$

Finally, one extracts

$$C = \text{first block row of } \mathcal{O}_k, \qquad B = \text{first block column of } \mathcal{C}_\ell,$$

and computes

$$A = \mathcal{O}_k^{\dagger} \mathcal{H}^{\uparrow}_{k,\ell} \mathcal{C}_\ell^{\dagger}.$$

The main message is that, even when a state-space model is not available a priori, it can be reconstructed from data. In this sense, the data themselves contain the model information needed for prediction and control. Once a realization has been identified, the model-based control methods developed in the previous chapters, such as LQR and MPC, can be applied to the identified plant exactly as if the model had been known from first principles.

At the same time, it is important to keep in mind the assumptions underlying this construction. The derivation assumes that the plant is an exact finite-dimensional linear time-invariant system and that the Markov parameters are known without noise. Under these ideal conditions, the rank of the Hankel matrix is sharp, the realization is exact, and the Ho–Kalman algorithm recovers a minimal state-space model up to a change of coordinates. Real identification problems are usually more challenging: measurements are noisy, data are finite, and the true dynamics may be nonlinear or only approximately finite-dimensional. Therefore, the material presented here should be understood as a first and fundamental example of system identification, rather than as a complete treatment of the subject. More advanced settings include noisy and stochastic data, missing measurements, prior structural information, nonlinear models, reduced-order representations, and both parametric and nonparametric identification methods. Still, the ideas developed in this chapter remain central: organize data into structured matrices, reveal the intrinsic dimension through linear algebra, and reconstruct a model that captures the dynamical behavior relevant for control.

6 Data-driven LQR

In this chapter we study the **infinite-horizon linear quadratic regulator** from a data-driven viewpoint. The starting point is the standard stochastic LQR problem for the discrete-time linear system

$$x_{t+1} = Ax_t + Bu_t, \qquad x_0 \sim \mathcal{D},$$

where the objective is to minimize the expected infinite-horizon quadratic cost

$$\min_{\{u_t\}} \mathbb{E}\left[\sum_{t=0}^{\infty} \left(x_t^\top Q x_t + u_t^\top R u_t\right)\right].$$

Throughout, we assume

$$Q \succeq 0, \qquad R \succ 0,$$

and we require the standard structural assumptions

$$(A, B) \text{ stabilizable}, \qquad (Q^{1/2}, A) \text{ detectable}.$$

Under these assumptions, the infinite-horizon LQR problem is well posed and admits an **optimal stationary policy** of the form

$$u_t = K^\star x_t,$$

that is, the optimal controller is a linear state-feedback law.

When the matrices A and B are known, the optimal gain $K^\star$ can be computed in different but equivalent ways. A first route is dynamic programming, which leads to the Riccati recursion and, in the infinite-horizon limit, to the algebraic Riccati equation. A second route is therefore to solve directly the algebraic Riccati equation and recover the optimal gain from its stabilizing solution. A third route is based on a **semidefinite programming reformulation**. These viewpoints are equivalent at the model-based level, but they suggest different ways of reformulating the problem when the model is not available exactly and one would like to reason directly from data.

6.1 LQR parameterization

A useful perspective for data-driven control is to parameterize the LQR problem directly in terms of the feedback matrix K. Instead of optimizing over arbitrary input sequences, we restrict attention to linear state-feedback policies

$$u_t = Kx_t,$$

and **optimize over all stabilizing gains** K.
To make the dependence on K explicit, let us assume for simplicity that the initial condition satisfies

$$\mathbb{E}[x_0 x_0^\top] = I.$$

If the feedback law $u_t = Kx_t$ is applied, then the closed-loop system becomes

$$x_{t+1} = (A + BK)x_t.$$

For a stabilizing gain K, it is natural to introduce the infinite-horizon state covariance

$$P \;=\; \mathbb{E}\left[\sum_{t=0}^{\infty} x_t x_t^\top\right].$$

This matrix collects the second-order state information accumulated over the whole trajectory. Using the closed-loop dynamics, one obtains

$$\begin{aligned} P &= \mathbb{E}[x_0 x_0^\top] + (A+BK)\,\mathbb{E}[x_0 x_0^\top]\,(A+BK)^\top + (A+BK)^2\,\mathbb{E}[x_0 x_0^\top]\,\left((A+BK)^\top\right)^2 + \cdots \\ &= \mathbb{E}[x_0 x_0^\top] + \sum_{k=1}^{\infty} (A+BK)^k \mathbb{E}[x_0 x_0^\top]((A+BK)^\top)^k \\ &= \mathbb{E}[x_0 x_0^\top] + (A+BK)\,P\,(A+BK)^\top. \end{aligned}$$

The final step is valid because one can multiply the first equation on the left by $(A+BK)$ and on the right by $(A+BK)^\top$ obtaining

$$(A+BK)P(A+BK)^\top = (A+BK)\,\mathbb{E}[x_0 x_0^\top]\,(A+BK)^\top + (A+BK)^2\,\mathbb{E}[x_0 x_0^\top]\,\left((A+BK)^\top\right)^2 + \cdots$$

Using $\mathbb{E}[x_0 x_0^\top] = I$, we obtain the **Lyapunov equation**

$$P = I + (A+BK)P(A+BK)^\top.$$

This identity is fundamental. In particular, it shows that P is the unique positive definite solution of the Lyapunov equation **if and only if** the closed-loop matrix $A+BK$ is Schur stable.
Using this parameterization, the infinite-horizon LQR cost can be rewritten in terms of K and P. Indeed, since $u_t = Kx_t$, we have

$$x_t^\top Q x_t + u_t^\top R u_t = x_t^\top (Q + K^\top R K) x_t,$$

Notice that this quantity is a scalar, so we can take the trace and change the order of the factors to obtain

$$\mathrm{Tr}(x_t^\top (Q + K^\top R K) x_t) = \mathrm{Tr}((Q + K^\top R K) x_t x_t^\top).$$

Hence, summing over time and taking expectations gives

$$J(K) = \mathrm{Tr}(QP) + \mathrm{Tr}(K^\top R K\, P).$$

Therefore, the **infinite-horizon LQR problem can be expressed as**

$$\begin{aligned} \min_{K,\, P\succ 0} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}(K^\top R K\, P) \\ \text{s.t.} \quad & P = I + (A+BK)P(A+BK)^\top. \end{aligned}$$

This formulation is important because it makes the optimization variable K explicit and **replaces the dynamical system by a matrix equality constraint**. At the same time, it also makes clear that the problem is *non-convex*: the Lyapunov constraint is bilinear in K and P, and the objective also contains the product of these variables.
This non-convex parameterization is nevertheless extremely useful, because it is the form from which many data-driven LQR methods start. The **main challenge** of data-driven LQR is precisely to recover or approximate the optimal gain $K^\star$ **without explicitly knowing the matrices** A and B, while preserving the stabilizing and performance properties of the model-based solution.

6.2 SDP formulation of the LQR

The parameterization introduced above is very convenient conceptually, but it still leads to a *non-convex* optimization problem:

$$\begin{aligned} \min_{K,\, P\succ 0} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}(K^\top R K\, P) \\ \text{s.t.} \quad & P = I + (A+BK)P(A+BK)^\top. \end{aligned}$$

The non-convexity appears in two places. First, the Lyapunov equality constraint is bilinear in K and P. Second, the term $\mathrm{Tr}(K^\top RK\,P)$ also contains products of the decision variables. A remarkable fact is that this LQR problem can nevertheless be reformulated *exactly* as a **convex semidefinite program** (SDP).
Before deriving the SDP formulation, it is useful to recall a standard tool that is repeatedly used in control and convex optimization.

Schur complement. Consider a symmetric block matrix

$$X = \begin{bmatrix} A & B \\ B^\top & C \end{bmatrix}.$$

If C is invertible, the matrix

$$A - BC^{-1}B^\top$$

is called the **Schur complement** of C in X. The corresponding lemma states that, if $C \succ 0$, then

$$\begin{bmatrix} A & B \\ B^\top & C \end{bmatrix} \succeq 0 \qquad \Longleftrightarrow \qquad A - BC^{-1}B^\top \succeq 0.$$

This result allows us to transform matrix inequalities involving inverses into linear matrix inequalities, which are exactly the constraints used in semidefinite programming.

We now **derive the SDP formulation** of LQR step by step. Start from

$$\begin{aligned} \min_{K,\,P\succ 0} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}(K^\top RK\,P) \\ \text{s.t.} \quad & P = I + (A+BK)P(A+BK)^\top. \end{aligned} \tag{3}$$

A first observation is that we can **replace the equality constraint by the inequality**

$$P \succeq I + (A+BK)P(A+BK)^\top$$

without changing the optimum. This gives the equivalent problem

$$\begin{aligned} \min_{K,\,P\succ 0} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}(K^\top RK\,P) \\ \text{s.t.} \quad & P \succeq I + (A+BK)P(A+BK)^\top. \end{aligned} \tag{4}$$

At first sight, this equivalence may look surprising, so it is worth understanding why it is true. Fix a feedback gain K such that $A+BK$ is Schur stable, and let P_K denote the unique positive definite solution of the Lyapunov equation

$$P_K = I + (A+BK)P_K(A+BK)^\top. \tag{5}$$

Now consider any matrix P satisfying the inequality

$$P \succeq I + (A+BK)P(A+BK)^\top. \tag{6}$$

Define the difference

$$Y := P - P_K.$$

Subtracting (5) from (6), we obtain

$$Y \succeq (A+BK)\,Y\,(A+BK)^\top.$$

Iterating this inequality yields

$$Y \succeq (A+BK)\,Y\,(A+BK)^\top \succeq (A+BK)^2\,Y\,\left((A+BK)^\top\right)^2 \succeq \cdots \succeq 0.$$

Hence $Y \succeq 0$, that is,

$$P \succeq P_K.$$

Therefore, for a fixed stabilizing K, the smallest feasible matrix P is exactly the Lyapunov solution P_K. Since the objective is linear and increasing in P (recall that $Q \succeq 0$ and $R \succ 0$), the optimizer will always choose the smallest feasible P, so the inequality in (4) is active at the optimum. This explains why (3) and (4) are equivalent.

The next step is to **eliminate the bilinear term in the constraint** by introducing the change of variables

$$L := KP.$$

Using this substitution, the constraint in (4) becomes

$$P - I - (AP + BL)P^{-1}(AP + BL)^\top \succeq 0.$$

This expression is still nonlinear because of the inverse P^{-1}, but now it is in a form where the Schur complement can be applied. Since $P \succ 0$, the Schur complement lemma implies that the inequality above is equivalent to the linear matrix inequality

$$\begin{bmatrix} P - I & AP + BL \\ (AP + BL)^\top & P \end{bmatrix} \succeq 0. \tag{7}$$

So the Lyapunov inequality has now been converted into an Linear Matrix Inequality, which is convex in the variables P and L.

We now turn to the objective function. Since $L = KP$, one would like to replace the term $KPK^\top$ by a new matrix variable. Introduce

$$M := KPK^\top.$$

Then

$$\mathrm{Tr}(K^\top RKP) = \mathrm{Tr}(RKPK^\top) = \mathrm{Tr}(RM),$$

and the objective becomes

$$\mathrm{Tr}(QP) + \mathrm{Tr}(RM).$$

However, the identity $M = KPK^\top$ is again nonlinear. As before, we relax it to the inequality

$$M \succeq KPK^\top.$$

This leads to

$$M - KPK^\top \succeq 0.$$

Using $L = KP$, this can be rewritten as

$$M - LP^{-1}L^\top \succeq 0.$$

Applying again the Schur complement with respect to $P \succ 0$, we obtain the equivalent LMI

$$\begin{bmatrix} M & L \\ L^\top & P \end{bmatrix} \succeq 0. \tag{8}$$

As in the previous step, this relaxation is tight at the optimum because the objective is linear in M and $R \succ 0$, so the optimizer will choose the smallest feasible M.

Putting everything together, we arrive at the **SDP formulation of the infinite-horizon LQR**:

$$\begin{aligned} \min_{L,M,P} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}(RM) \\ \text{s.t.} \quad & \begin{bmatrix} M & L \\ L^\top & P \end{bmatrix} \succeq 0, \\ & \begin{bmatrix} P - I & AP + BL \\ (AP + BL)^\top & P \end{bmatrix} \succeq 0, \\ & P \succ 0. \end{aligned} \tag{9}$$

This is now a **convex optimization problem**: the objective is affine in the decision variables and the constraints are positive semidefinite constraints, that is, linear matrix inequalities.

Once the problem has been solved, the optimal feedback gain is recovered from

$$K = LP^{-1}.$$

So, although the original LQR problem was written in a non-convex form in terms of K and P, it admits an equivalent convex reformulation in the lifted variables (L, M, P).

This result is much more than a technical curiosity. Semidefinite programming plays a central role in modern control because many synthesis and analysis problems can be written in terms of LMIs after a suitable change of variables. The LQR problem is one of the cleanest examples of this idea: a nonlinear controller synthesis problem can be converted into a convex optimization problem for which global optimality is guaranteed.
For us, this SDP viewpoint is particularly useful because it opens the door to **data-driven** reformulations. Indeed, once the model-based LQR problem has been written in terms of matrix inequalities, one can try to replace the quantities involving A and B by quantities constructed directly from data.

6.3 From model-based to data-driven LQR

At this point it is useful to step back and summarize the situation. For the infinite-horizon LQR problem, we have seen three equivalent *model-based* solution routes:

- dynamic programming;
- the algebraic Riccati equation;
- the SDP formulation.

Although these approaches look quite different, they all share the same fundamental requirement: **they need the matrices A and B to be known**. Dynamic programming needs the model to write the Bellman recursion, the Riccati approach needs the model to form the algebraic Riccati equation, and the SDP formulation also contains A and B explicitly in its constraints. Therefore, if the system matrices are not available, none of these procedures can be applied directly.
This observation motivates the central question of this chapter:

> *Can we design the optimal LQR controller directly from data, without first having an exact model of the system?*

This question belongs to the broader area of **data-driven control**. At a high level, one can distinguish two main philosophies.

Indirect data-driven control. In the indirect approach, one first uses data to identify a model of the system, together with a description of the uncertainty, and then one designs a controller based on the identified model. In short, the logic is

$$\text{data} \longrightarrow \text{model identification} + \text{uncertainty quantification} \longrightarrow \text{control design}.$$

This is the classical modular viewpoint: first system identification, then control synthesis.

Direct data-driven control. A different philosophy is to *bypass explicit model identification* and attempt to compute a controller directly from data. This is the so-called **direct** data-driven approach. In recent years this viewpoint has attracted a lot of attention, partly because it promises to avoid the intermediate identification step and to exploit data more directly for control design.

However, this promise comes with several trade-offs. In particular, when comparing indirect and direct approaches, one should keep in mind issues such as:

- modularity versus non-modularity;
- computational tractability;
- optimality versus suboptimality;
- robustness and robust stability guarantees;
- the amount of data required.

For the LQR problem, these questions can be addressed quite explicitly, which is one of the reasons why LQR is an ideal benchmark for studying data-driven control design.

6.4 Indirect data-driven control

6.4.1 System identification from input-state data

Let us start from the indirect route. Suppose that the system evolves according to

$$x_{t+1} = Ax_t + Bu_t + w_t,$$

where w_t is a process disturbance or modeling error. Assume that we collect a trajectory of state-input data and arrange it into the matrices

$$X_0 = \begin{bmatrix} x_0 & x_1 & \cdots & x_{T-1} \end{bmatrix}, \qquad U_0 = \begin{bmatrix} u_0 & u_1 & \cdots & u_{T-1} \end{bmatrix},$$

$$W_0 = \begin{bmatrix} w_0 & w_1 & \cdots & w_{T-1} \end{bmatrix}, \qquad X_1 = \begin{bmatrix} x_1 & x_2 & \cdots & x_T \end{bmatrix}.$$

By stacking the dynamics over the whole trajectory, we obtain the compact relation

$$X_1 = AX_0 + BU_0 + W_0.$$

This matrix equation is the starting point for identification from data. It tells us that the unknown matrices A and B must explain, as well as possible, the one-step transitions observed in the data.

A fundamental requirement is that the data be sufficiently informative. In this context, the relevant notion is **persistence of excitation**. More precisely, the data are said to be sufficiently exciting if

$$\operatorname{rank} \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} = n + m,$$

where n is the state dimension and m is the input dimension. This rank condition means that the collected trajectory excites all state and input directions needed to identify the dynamics. If the data do not satisfy this condition, then different pairs (A, B) may be consistent with the same observations, and the model cannot be uniquely reconstructed.

Given the data, a natural identification procedure is ordinary least squares. One solves

$$[\hat{B}, \hat{A}] = \arg\min_{B,A} \left\| X_1 - [B,\ A] \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} \right\|_F .$$

Equivalently, one searches for the matrices A and B that best fit the observed one-step state transitions in the Frobenius norm sense. Under the persistence of excitation condition, this least-squares problem has a unique solution. Hence, with sufficiently rich data, one can recover an estimate $(\hat{A}, \hat{B})$ of the system matrices.

6.4.2 Indirect and certainty-equivalent LQR

Once the model has been identified, the most immediate strategy is simply to replace the true unknown matrices (A, B) by their estimates $(\hat{A}, \hat{B})$ and then solve the LQR problem as if the estimated model were exact. This is the classical **certainty-equivalence principle**.
Using the LQR parameterization introduced earlier, the certainty-equivalent design solves

$$\begin{aligned} \min_{K,\, P \succ 0} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}(K^\top R K\, P) \\ \text{s.t.} \quad & P = I + (\hat{A} + \hat{B}K) P (\hat{A} + \hat{B}K)^\top, \end{aligned}$$

where the identified model is obtained from the least-squares problem

$$[\hat{B}, \hat{A}] = \arg\min_{B,A} \left\| X_1 - [B,\ A] \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} \right\|_F .$$

In words, the procedure is:

1. collect input-state data;
2. identify $(\hat{A}, \hat{B})$ from the data;
3. design the LQR controller for the estimated model.

This is why the method is called **indirect**: the data are used only to build a model, and the controller is then synthesized by a standard model-based technique.
The certainty-equivalent approach is widely used both in theory and in applications. The main reasons are clear:

- it is **modular**, because identification and control design are separated;
- it is **tractable**, since after identification one can use the Riccati equation or the SDP formulation developed earlier;
- it requires only the minimum amount of excitation needed to identify the model, namely enough data to recover $n + m$ independent directions.

At the same time, it also has **clear limitations**. Since the controller is designed for the estimated model rather than the true one, the resulting gain is generally only **suboptimal** for the real system. Moreover, **robustness** is only moderate unless one explicitly accounts for identification errors and uncertainty in the design step. In other words, certainty-equivalence is simple and powerful, but it does not by itself provide a complete answer to the issue of robustness.

This discussion highlights a key tension in data-driven control. The indirect route is attractive because it preserves the classical architecture of *identification first, control second*, and therefore remains easy to analyze and implement. On the other hand, it may fail to fully exploit the information in the data for the actual control objective.

6.5 Direct data-driven LQR via trajectory parameterization

The indirect approach discussed above follows the classical two-step pipeline

$$\text{data} \longrightarrow (\hat{A}, \hat{B}) \longrightarrow K.$$

A natural question is whether one can avoid the explicit identification step and parameterize the controller directly in terms of the collected data. This leads to a **direct** data-driven formulation of LQR.
The key idea is to use the input-state trajectory itself as a parameterization device. Recall that, for the noisy system

$$x_{t+1} = Ax_t + Bu_t + w_t,$$

the stacked data matrices satisfy

$$X_1 = AX_0 + BU_0 + W_0.$$

Assume again that the data are persistently exciting, namely

$$\operatorname{rank} \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} = n + m.$$

This rank condition has an important consequence: for *every* feedback gain K there exists a matrix G such that

$$\begin{bmatrix} K \\ I \end{bmatrix} = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} G. \tag{10}$$

So the pair (K, I) can be represented as a linear combination of the columns of the measured data matrix. This is the basic trajectory parameterization.
Once (10) is available, the closed-loop matrix can also be written directly in terms of the data. Indeed,

$$A + BK = [B \;\; A] \begin{bmatrix} K \\ I \end{bmatrix} = [B \;\; A] \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} G.$$

Using the data equation

$$[B \;\; A] \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} = X_1 - W_0,$$

we obtain

$$A + BK = (X_1 - W_0)G. \tag{11}$$

This identity is crucial: it shows that the unknown closed-loop matrix can be represented from data through the variable G.
If the noise term W_0 is neglected, the relation simplifies to

$$A + BK = X_1 G.$$

This is exactly the certainty-equivalent viewpoint in direct form: **one behaves as if the measured data were generated by a noise-free system**. In that sense, the indirect certainty-equivalent approach and the direct certainty-equivalent approach are closely related.

Substituting the relation $A + BK = X_1 G$ into the LQR parameterization yields a direct data-driven reformulation:

$$\begin{aligned} \min_{P \succeq I,\, K,\, G} \quad & \operatorname{Tr}(QP) + \operatorname{Tr}(K^\top RKP) \\ \text{s.t.} \quad & X_1 G P G^\top X_1^\top - P + I \preceq 0, \\ & \begin{bmatrix} K \\ I \end{bmatrix} = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} G. \end{aligned}$$

The first constraint is simply the Lyapunov inequality written using the data-based expression for the closed-loop matrix, while the second constraint imposes consistency between the gain K and the trajectory parameterization G.
A useful **interpretation** is obtained by solving the second constraint explicitly. Since

$$\begin{bmatrix} K \\ I \end{bmatrix} = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} G,$$

we get

$$K = U_0 G, \qquad I = X_0 G.$$

Hence the optimization variables can be reduced to (G, P) and the problem becomes

$$\begin{aligned} \min_{G,\, P \succ 0} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}\big(G^\top U_0^\top R U_0 G P\big) \\ \text{s.t.} \quad & X_1 G P G^\top X_1^\top - P + I \preceq 0, \\ & X_0 G = I. \end{aligned}$$

This makes clear that G itself plays the role of the **policy parameter**. Indeed, the control law is

$$u_t = K x_t = U_0 G x_t,$$

and, correspondingly, the closed-loop dynamics become

$$x_{t+1} = X_1 G x_t,$$

again in the noise-free certainty-equivalent interpretation.

6.5.1 Non-uniqueness of the trajectory parameterization

An important feature of this direct formulation is that the matrix G is generally *not unique*. The reason is that the linear system

$$\begin{bmatrix} K \\ I \end{bmatrix} = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} G$$

may have infinitely many solutions whenever the data matrix

$$\begin{bmatrix} U_0 \\ X_0 \end{bmatrix}$$

has more columns than rows. In that case, the set of feasible G contains an affine subspace whose direction is precisely the nullspace

$$\ker \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}.$$

So two different matrices G may generate the same gain K and still satisfy the consistency constraint.
This non-uniqueness is not just a technical detail. It means that the direct formulation admits extra degrees of freedom that are invisible at the level of K. Those degrees of freedom can be used either in a favorable way or in a harmful way, depending on how the optimization problem is regularized. This is why, in practice, one usually augments the direct formulation with a **regularizer** that selects one particular representative among all feasible G.

Broadly speaking, two types of regularization are of special interest:

- a **certainty-equivalence-promoting** regularizer, which selects the solution most closely related to the least-squares model estimate;
- a **robustness-promoting** regularizer, which exploits the extra freedom in G to improve robustness with respect to noise and uncertainty.

So, in the direct approach, regularization is not only a numerical device: it is part of the control design itself.

6.5.2 Equivalence between direct and indirect certainty-equivalent LQR

The previous discussion suggests that there should be a precise connection between the direct and indirect approaches. In fact, under a suitable choice of regularization, the direct certainty-equivalent formulation is equivalent to the indirect one based on least-squares identification.

To see the mechanism, recall that among all solutions of

$$\begin{bmatrix} K \\ I \end{bmatrix} = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} G,$$

one can single out the minimum-norm solution

$$G = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^{\dagger} \begin{bmatrix} K \\ I \end{bmatrix},$$

where † denotes the pseudoinverse. Equivalently, one can enforce that G be orthogonal to the nullspace of the data matrix. Writing

$$\Pi := I - \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^{\dagger} \begin{bmatrix} U_0 \\ X_0 \end{bmatrix},$$

the matrix Π is the orthogonal projector onto

$$\ker \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}.$$

Therefore, the orthogonality condition can be written as

$$\Pi G = 0.$$

It is useful to remember the main properties of this projector:

$$\Pi^2 = \Pi, \qquad \Pi^{\top} = \Pi, \qquad \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} \Pi = 0.$$

Now substitute the minimum-norm expression for G into the direct formulation. Since

$$G = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^{\dagger} \begin{bmatrix} K \\ I \end{bmatrix},$$

we obtain

$$X_1 G = X_1 \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^{\dagger} \begin{bmatrix} K \\ I \end{bmatrix}.$$

On the other hand, the least-squares estimates introduced before satisfy

$$[\hat{B} \;\; \hat{A}] = X_1 \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^{\dagger}.$$

Hence

$$X_1 G = [\hat{B} \;\; \hat{A}] \begin{bmatrix} K \\ I \end{bmatrix} = \hat{A} + \hat{B}K.$$

This shows that the data-based closed-loop matrix appearing in the direct approach coincides exactly with the identified closed-loop matrix of the indirect least-squares model.

As a consequence, once the orthogonality constraint $\Pi G = 0$ is imposed, the direct certainty-equivalent problem becomes equivalent to the indirect certainty-equivalent LQR problem

$$\begin{aligned} \min_{P \succeq I,\, K} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}(K^\top R K P) \\ \text{s.t.} \quad & (\hat{A} + \hat{B}K) P (\hat{A} + \hat{B}K)^\top - P + I \preceq 0, \end{aligned}$$

with

$$[\hat{B} \;\; \hat{A}] = \arg\min_{B,A} \left\| X_1 - [B \;\; A] \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} \right\|_F .$$

So the indirect least-squares route is not disconnected from the direct one: it corresponds to a specific way of choosing the trajectory parameterization, namely the **minimum-norm choice of G orthogonal to the nullspace of the data matrix**.

This equivalence is conceptually important. It tells us that certainty-equivalence is not tied to the explicit identification of a model; rather, it can also be understood as a particular regularization of the direct data-driven parameterization. At the same time, it clarifies where direct approaches may depart from indirect ones: by exploiting the nullspace freedom in G, one may design regularizers that do *not* reproduce least-squares identification and that instead aim at improved robustness.

6.5.3 Certainty-equivalence-promoting regularization

We now make the previous equivalence more operational. In the trajectory-parameterized direct formulation, the ambiguity in G comes from the nullspace of the data matrix

$$\begin{bmatrix} U_0 \\ X_0 \end{bmatrix}.$$

If one wants the direct solution to reproduce the same behavior as the indirect least-squares approach, then one should select the representative of G that is orthogonal to this nullspace. Recall the projector

$$\Pi := I - \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\dagger \begin{bmatrix} U_0 \\ X_0 \end{bmatrix},$$

which is the orthogonal projector onto

$$\ker \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}.$$

The exact orthogonality constraint

$$\Pi G = 0$$

forces G to belong to the orthogonal complement of the nullspace, and therefore selects the minimum-norm solution

$$G = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\dagger \begin{bmatrix} K \\ I \end{bmatrix}.$$

As we have seen, this is precisely the choice that makes the direct certainty-equivalent formulation equivalent to the indirect least-squares LQR design.

Instead of imposing the hard constraint $\Pi G = 0$, one can **lift it to the objective** and use it as a regularizer. This leads to the optimization problem

$$\begin{aligned} \min_{P \succeq I,\, K,\, G} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}(K^\top R K P) + \lambda \|\Pi G\| \\ \text{s.t.} \quad & X_1 G P G^\top X_1^\top - P + I \preceq 0, \\ & \begin{bmatrix} K \\ I \end{bmatrix} = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} G. \end{aligned}$$

Here $\lambda \geq 0$ is a tuning parameter. When λ is large, the optimizer is strongly encouraged to make ΠG small, that is, to choose G almost orthogonal to the nullspace of the data matrix. In the limit, this reproduces the same effect as the hard orthogonality constraint.

This regularizer is therefore **certainty-equivalence-promoting**: it biases the direct data-driven problem toward the same solution that would be obtained by first identifying a least-squares model and then designing the LQR controller for that model.

The parameter λ has a clear interpretation. It interpolates between two extremes:

- if $\lambda = 0$, the optimizer is free to exploit the whole family of feasible matrices G, including the nullspace directions;
- if λ is very large, the optimizer is pushed toward the minimum-norm solution associated with least-squares system identification.

In this sense, λ acts as a knob that interpolates between **pure control design** and **system-identification-driven design**. This is conceptually useful because it shows that direct and indirect methods are not two completely disconnected worlds, but rather two endpoints of a continuum of regularized formulations.

6.5.4 Robustness-promoting regularization for trajectory parameterization

The previous regularizer promotes equivalence with least-squares identification, but it does not explicitly address robustness with respect to noise. To understand how robustness enters, it is useful to go back to the noisy data relation

$$A + BK = (X_1 - W_0)G.$$

In the certainty-equivalent formulation, one replaces this by the nominal approximation

$$A + BK \approx X_1 G,$$

thereby neglecting the noise term $W_0 G$. The issue is that this noise term is multiplied by G, so large values of G amplify the effect of data perturbations on the reconstructed closed-loop matrix.

The same phenomenon appears in the Lyapunov inequality. The nominal direct constraint is

$$X_1 G P G^\top X_1^\top - P + I \preceq 0,$$

whereas the true noisy relation would involve

$$(X_1 - W_0) G P G^\top (X_1 - W_0)^\top - P + I \preceq 0.$$

So the discrepancy between the nominal and true constraints depends on the matrix

$$G P G^\top.$$

This suggests the following principle: **for robustness, the quantity $GPG^\top$ should be kept small**. Intuitively, if $GPG^\top$ is small, then the effect of the unknown term W_0 on the closed-loop

Lyapunov inequality is less pronounced.
This motivates the robustness-promoting regularized problem

$$\begin{aligned} \min_{P\succeq I,\, K,\, G} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}(K^\top RKP) + \lambda \|GPG^\top\| \\ \text{s.t.} \quad & X_1 GPG^\top X_1^\top - P + I \preceq 0, \\ & \begin{bmatrix} K \\ I \end{bmatrix} = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} G. \end{aligned}$$

The regularization term penalizes solutions for which the policy parameterization is too sensitive to data perturbations. In contrast with the certainty-equivalence-promoting regularizer, which selects the minimum-norm representative in the affine family of feasible G, this new term explicitly aims at reducing the amplification of noise in the Lyapunov constraint.

There is also a useful connection between the two types of regularization. Since the nullspace directions of G are precisely those that are invisible in the consistency constraint

$$\begin{bmatrix} K \\ I \end{bmatrix} = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} G,$$

they may still enlarge $GPG^\top$ without changing the nominal gain K. For this reason, the projection regularizer $\|\Pi G\|$ can also indirectly help to keep $GPG^\top$ small. So the certainty-equivalence-promoting and robustness-promoting regularizers are different, but not completely unrelated.

6.5.5 Summary and limitations of direct trajectory parameterization

We can now summarize the direct LQR formulation based on trajectory parameterization. The starting point is the fact that, under persistence of excitation,

$$\forall K \;\; \exists G \;\text{ such that }\; \begin{bmatrix} K \\ I \end{bmatrix} = \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} G,$$

and hence

$$A + BK = (X_1 - W_0)G.$$

Neglecting the noise yields the nominal direct certainty-equivalent relation

$$A + BK = X_1 G.$$

This leads to a direct data-driven LQR formulation in which the controller is parameterized by the matrix G instead of by an explicit model (A, B).

Because G is not unique, regularization is essential. Two important choices are:

- the **certainty-equivalence-promoting regularizer** $\|\Pi G\|$, which selects the least-squares-compatible representative;
- the **robustness-promoting regularizer** $\|GPG^\top\|$, which tries to reduce sensitivity to noise.

This framework is attractive because it bypasses explicit model identification and keeps the entire design directly at the data level. However, it also has important drawbacks.

The first drawback is that the **dimension of the optimization variable G grows with the amount of data**. If the experiment contains many samples, then G has many rows, and the optimization problem becomes increasingly large. So, unlike model-based or indirect approaches,

this parameterization does not have a complexity that is determined only by the state and input dimensions.
The second drawback is that **regularization is not optional**. Because of the non-uniqueness of G, some regularizing principle must be introduced in order to select a meaningful solution. This is not merely a numerical issue: different regularizers correspond to different control design philosophies, such as certainty-equivalence or robustness.
These observations motivate the search for alternative direct parameterizations that retain the advantages of data-driven design but avoid the explicit dependence on the full data length. This is exactly the role of the *sample covariance parameterization*, which we discuss next.

6.6 Direct data-driven LQR via sample covariance parameterization

The trajectory-based parameterization uses the full data matrices X_0, U_0, and X_1. A different possibility is to compress the information contained in the data through sample covariances. This leads to a direct formulation whose dimension no longer grows with the number of collected samples.

Starting from the same data equation

$$X_1 = AX_0 + BU_0 + W_0,$$

define the sample covariance matrices

$$\Phi := \frac{1}{T}\begin{bmatrix} U_0 \\ X_0 \end{bmatrix}\begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top, \qquad \Phi^+ := \frac{1}{T}X_1\begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top.$$

Under the persistence of excitation condition,

$$\operatorname{rank}\begin{bmatrix} U_0 \\ X_0 \end{bmatrix} = n + m,$$

the matrix Φ is positive definite:

$$\Phi \succ 0.$$

This is the covariance analogue of the full-row-rank property used in the trajectory parameterization.
The covariance parameterization is based on the observation that, for every gain K, there exists a matrix V such that

$$\begin{bmatrix} K \\ I \end{bmatrix} = \frac{1}{T}\begin{bmatrix} U_0 \\ X_0 \end{bmatrix}\begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top V = \Phi V. \tag{12}$$

So the role played previously by G is now taken by a matrix V living in the smaller space associated with the sample covariance matrix.
Using (12), the closed-loop matrix becomes

$$A + BK = [B \;\; A]\begin{bmatrix} K \\ I \end{bmatrix} = [B \;\; A]\Phi V.$$

From the data equation, this can be rewritten as

$$A + BK = \frac{1}{T}(X_1 - W_0)\begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top V.$$

If noise is neglected, one obtains the certainty-equivalent approximation

$$A + BK = \Phi^+ V.$$

So, exactly as in the trajectory case, the unknown closed-loop matrix is replaced by a quantity constructed directly from the data, but now only through the empirical covariances Φ and Φ^+. Substituting this relation into the LQR parameterization gives the covariance-based direct formulation

$$\begin{aligned} \min_{K,\, P\succeq I,\, V} \quad & \mathrm{Tr}(QP) + \mathrm{Tr}(K^\top RKP) \\ \text{s.t.} \quad & P = I + (\Phi^+ V)P(\Phi^+ V)^\top, \\ & \begin{bmatrix} K \\ I \end{bmatrix} = \Phi V. \end{aligned}$$

Equivalently, since $\Phi \succ 0$, the second constraint determines V uniquely as

$$V = \Phi^{-1} \begin{bmatrix} K \\ I \end{bmatrix}.$$

This is a major difference with respect to the trajectory parameterization: in the covariance parameterization the auxiliary variable is **unique**, so the nullspace ambiguity disappears.
This has several important consequences:

- the dimension of the parameterization is **independent of the data length** T;
- the covariance matrices admit **recursive rank-one updates**, so the parameterization is naturally suitable for online data accumulation;
- the auxiliary variable V is uniquely determined, so no nullspace regularization is needed to select among infinitely many representatives.

Moreover, this covariance formulation is again equivalent to indirect certainty-equivalent LQR. Indeed, from

$$\Phi^+\Phi^{-1} = \frac{1}{T} X_1 \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top \left(\frac{1}{T} \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top \right)^{-1},$$

one recovers the same least-squares estimate as before:

$$[\hat{B} \;\; \hat{A}] = \Phi^+\Phi^{-1}.$$

Therefore,

$$\Phi^+ V = \Phi^+\Phi^{-1} \begin{bmatrix} K \\ I \end{bmatrix} = [\hat{B} \;\; \hat{A}] \begin{bmatrix} K \\ I \end{bmatrix} = \hat{A} + \hat{B}K.$$

So the covariance-based direct parameterization is another way of writing the same certainty-equivalent closed-loop matrix, but in a compressed form that depends only on sample covariances.

6.6.1 Robustness-promoting regularization for covariance parameterization

The covariance formulation removes the nullspace ambiguity, but it does not remove the effect of noise. Indeed, the exact noisy relation is

$$A + BK = \frac{1}{T}(X_1 - W_0) \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top V.$$

When this expression is substituted into the Lyapunov inequality, one obtains terms of the form

$$\frac{1}{T^2}(X_1 - W_0) \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top V P V^\top \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} (X_1 - W_0)^\top.$$

The nominal certainty-equivalent approximation replaces this by

$$\frac{1}{T^2} X_1 \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top V P V^\top \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} X_1^\top.$$

The gap between the noisy and nominal terms is governed by the middle factor

$$\frac{1}{T} \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top V P V^\top \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}.$$

Its trace can be written as

$$\operatorname{Tr}\left(\frac{1}{T} \begin{bmatrix} U_0 \\ X_0 \end{bmatrix}^\top V P V^\top \begin{bmatrix} U_0 \\ X_0 \end{bmatrix} \right) = \operatorname{Tr}(\Phi V P V^\top).$$

This suggests that, in the covariance parameterization, a natural robustness-promoting quantity to keep small is

$$\operatorname{Tr}(\Phi V P V^\top).$$

So the covariance-based counterpart of robustness regularization penalizes the energy of the middle covariance-weighted factor through a term such as

$$\lambda \operatorname{Tr}(\Phi V P V^\top).$$

The intuition is exactly the same as before: the smaller this quantity is, the less sensitive the Lyapunov inequality becomes to the noise entering through W_0.

This concludes the main constructions based on direct data-driven parameterizations. We now have two complementary viewpoints:

- the **trajectory parameterization**, which is very direct and transparent but whose dimension grows with the data length and requires explicit regularization;
- the **sample covariance parameterization**, which compresses the data into fixed-size matrices, yields a unique auxiliary variable, and recovers the same certainty-equivalent solution in a more compact form.

6.7 Comparison of data-driven LQR methods

We conclude this part of the chapter by comparing the main data-driven LQR approaches that we have introduced so far. The goal of this comparison is not to declare a universally best method, but rather to highlight the trade-offs between *modularity*, *optimality*, *robustness*, and *computational size*.
The methods we compare are:

- **indirect certainty-equivalent LQR**, where one first identifies $(\hat{A}, \hat{B})$ and then solves the corresponding model-based LQR problem;
- **direct LQR with trajectory parameterization**, in three variants: without regularization, with certainty-equivalence-promoting regularization, and with robustness-promoting regularization;
- **direct LQR with sample covariance parameterization**, with and without robustness regularization.

A compact summary is reported in Table 1.

We now explain the meaning of each row in the table.

	Indirect CE	Direct LQR + trajectory parameterization			Direct LQR + covariance parameterization	
	LQR	No reg.	CE reg.	Robust reg.	No reg.	Robust reg.
Modular	Yes	No	No	No	No	No
Optimality	Good	Bad	Good	Bad	Good	Good
Robust stability	Moderate	Bad	Moderate	Good	Moderate	Good
Problem size	Constant	Scales linearly with data size			Constant	

Table 1: Comparison of the main data-driven LQR methods discussed in this chapter.

Modularity. The indirect certainty-equivalent approach is the only **modular** one. Indeed, it keeps the classical separation

$$\text{identification} \longrightarrow \text{control design},$$

so the modeling step and the controller synthesis step can be developed and analyzed independently. By contrast, the direct methods are **non-modular**: data and control design are intertwined in a single optimization problem. This lack of modularity is not necessarily a drawback in itself, but it does make the overall design less transparent.

Optimality. The optimality row refers to how well the resulting controller aligns with the certainty-equivalent LQR objective.

The **indirect CE-LQR** method is marked as *good* because it computes exactly the LQR controller associated with the least-squares identified model. So, within the certainty-equivalent philosophy, it is the natural benchmark.
For **trajectory parameterization without regularization**, the performance is marked as *bad*. The reason is that the auxiliary variable G is not unique, and without any principle to select among the feasible solutions, the optimization may exploit unfavorable nullspace directions. This generally breaks the connection with least-squares identification and leads to a controller that is no longer well aligned with the nominal LQR objective.
When **certainty-equivalence-promoting regularization** is added, optimality becomes *good*, because the projector regularizer $\|\Pi G\|$ drives the solution toward the minimum-norm representative associated with least-squares identification. Hence the direct trajectory formulation recovers the same nominal solution as the indirect approach.
With **robustness-promoting regularization** in the trajectory formulation, optimality is again marked as *bad*. This does not mean the controller is useless; rather, it means that the design is no longer trying to reproduce the certainty-equivalent optimum. Instead, it sacrifices nominal optimality in order to improve robustness.
For the **covariance parameterization**, optimality is marked as *good* both without and with robustness regularization. Without regularization, this follows from the fact that the covariance parameterization is uniquely determined and is equivalent to indirect certainty-equivalent LQR. With robustness regularization, the table still marks the method as good, reflecting the fact that the covariance formulation preserves a much cleaner and more well-posed connection with the certainty-equivalent solution than the unregularized trajectory formulation.

Robust stability. The robustness row summarizes how reliable the different methods are in the presence of noisy data and modeling errors.

For **indirect CE-LQR**, robustness is classified as *moderate*. This is consistent with the discussion above: certainty-equivalence is simple and often effective, but it does not explicitly optimize robustness unless additional uncertainty-aware design steps are introduced.
For **trajectory parameterization without regularization**, robustness is *bad*. This is the least protected formulation, since the non-uniqueness of G may amplify noise significantly and

there is no mechanism preventing harmful solutions.
With **certainty-equivalence-promoting regularization**, robustness improves to *moderate*. In this case, the direct method essentially reproduces the least-squares indirect solution, so its robustness properties are similar to those of indirect CE-LQR.
With **robustness-promoting regularization**, the trajectory-based direct method becomes *good* from the robustness viewpoint, because the added penalty explicitly aims at reducing quantities such as $GPG^\top$, which control the sensitivity of the Lyapunov constraint to noise.
The same logic applies to the **covariance parameterization**. Without extra regularization, its robustness is *moderate*, since it is again tied to certainty-equivalence. Once a suitable robustness-promoting penalty is added, robustness becomes *good*.

Problem size. The last row concerns the computational dimension of the optimization problem.

For **indirect CE-LQR**, the problem size is **constant** once the model has been identified, because the controller synthesis depends only on the state and input dimensions (n, m).
For the **trajectory-parameterized direct methods**, the problem size **scales linearly with the amount of data**. This is because the matrix G has as many rows as there are data points in the experiment. Hence, as more samples are collected, the optimization problem grows.
For the **covariance-parameterized direct methods**, the problem size is again **constant**, because the data enter only through the fixed-size matrices Φ and Φ^+. This is one of the main reasons why the covariance formulation is attractive in practice.

The table makes the trade-offs quite visible.
Indirect certainty-equivalent LQR is attractive because it is modular, has fixed problem size, and provides a good nominal solution, but its robustness is only moderate.
Direct trajectory parameterization is conceptually very flexible and can be made robust, but it has two main weaknesses: its dimension grows with data size, and regularization is essential.
Direct covariance parameterization keeps the direct data-driven spirit while avoiding the growth with data size. It also preserves the connection with certainty-equivalent design in a much more compact way, and it can be enhanced with robustness-promoting regularization.

So, roughly speaking:

- **indirect CE-LQR** is the simplest modular baseline;
- **direct trajectory parameterization** offers the most design freedom, but at a higher computational and regularization cost;
- **direct covariance parameterization** provides a compact compromise between directness, nominal performance, and tractability.

This comparison will serve as a useful reference point for interpreting the strengths and limitations of the different data-driven LQR constructions developed in this chapter.

7 Data-Driven Predictive Control

Before moving to data-driven predictive control, it is useful to rewrite the input-output behavior of a linear system over a finite horizon in a compact lifted form. This representation will later allow us to reason directly in terms of trajectories collected from data.
Consider the discrete-time linear system

$$x_{t+1} = Ax_t + Bu_t, \qquad y_t = Cx_t,$$

and fix a horizon length L. By recursively expanding the state dynamics, the output at time t can be written as

$$y_t = CA^t x_0 + \sum_{j=0}^{t-1} CA^{t-1-j} B u_j.$$

This expression has two parts:

- the *free response* $CA^t x_0$, which depends only on the initial condition,
- the *forced response* (or convolution term), which depends on the past inputs.

7.1 Lifted input–output representation

To simplify the notation, let us first focus on the SISO case, namely

$$u_t \in \mathbb{R}, \qquad y_t \in \mathbb{R}.$$

Stacking the outputs and inputs over a horizon of length L, define

$$y_{[0,L-1]} := \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{L-1} \end{bmatrix}, \qquad u_{[0,L-1]} := \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{L-1} \end{bmatrix}.$$

Then the finite-horizon input–output relation becomes

$$y_{[0,L-1]} = \mathcal{O}_L x_0 + \mathcal{G}_L u_{[0,L-1]}, \tag{13}$$

where

$$\mathcal{O}_L := \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{L-1} \end{bmatrix} \in \mathbb{R}^{L \times n}$$

is the *extended observability matrix*, and

$$\mathcal{G}_L := \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 \\ CB & 0 & 0 & \cdots & 0 \\ CAB & CB & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ CA^{L-2}B & CA^{L-3}B & \cdots & CB & 0 \end{bmatrix} \in \mathbb{R}^{L \times L}$$

is the *convolution matrix*.
Equation (13) is important because it separates clearly the role of the initial condition from the role of the input sequence. The term $\mathcal{O}_L x_0$ describes all outputs that can be generated with zero input, while $\mathcal{G}_L u_{[0,L-1]}$ captures how the applied inputs shape the trajectory.

7.2 Free response and observability

If we set the input to zero, then only the free response remains:

$$y_{[0,L-1]} = \mathcal{O}_L x_0$$

This is exactly the finite-horizon observability relation. It tells us that reconstructing the initial state from an output sequence is equivalent to solving a linear system with matrix $\mathcal{O}_L$.

Assume now that the system is observable. In the SISO case this means that, for a sufficiently large horizon, the matrix $\mathcal{O}_L$ has full column rank:

$$\operatorname{rank}(\mathcal{O}_L) = n,$$

where n is the state dimension.
The consequences are immediate:

- if $L < n$, then $\mathcal{O}_L$ has fewer rows than columns, so in general there is not enough information to reconstruct x_0 uniquely;
- if $L = n$ and $\mathcal{O}_n$ is invertible, then x_0 can be reconstructed uniquely as

$$x_0 = \mathcal{O}_n^{-1} y_{[0,n-1]};$$

- if $L > n$, then the problem is overdetermined: if the measured output sequence is consistent with the model, then x_0 can still be reconstructed uniquely, but not every arbitrary sequence $y_{[0,L-1]}$ is valid.

So for $L > n$ the image of $\mathcal{O}_L$ contains exactly the output sequences that can arise as free responses of the system. This point is fundamental: model-consistent trajectories do not fill the whole space $\mathbb{R}^L$, but only an n-dimensional subspace.

7.2.1 Example: scalar system

Consider now the scalar system

$$x_{t+1} = \frac{1}{2} x_t + u_t, \qquad y_t = x_t,$$

for which the state dimension is $n = 1$. Since $C = 1$ and $A = \frac{1}{2}$, the extended observability matrix is

$$\mathcal{O}_L = \begin{bmatrix} 1 \\ \frac{1}{2} \\ \frac{1}{4} \\ \vdots \\ \left(\frac{1}{2}\right)^{L-1} \end{bmatrix}.$$

Assume there is no input, so that $u_t = 0$ for all t. Then the free response is simply

$$y_t = \left(\frac{1}{2}\right)^t x_0.$$

Hence the whole output sequence is completely determined by the single scalar x_0.

Can we reconstruct x_0 from $y_0 = 1$? Yes. Since $y_0 = x_0$, we immediately obtain

$$x_0 = 1$$

Can we reconstruct x_0 from $y_0 = 2$, $y_1 = 1$? Yes. From the first measurement we get $x_0 = 2$. The second measurement is consistent with the model because

$$y_1 = \frac{1}{2}x_0 = \frac{1}{2} \cdot 2 = 1.$$

So the sequence is valid, and the reconstructed initial state is

$$x_0 = 2.$$

Can we reconstruct x_0 from $y_0 = 1$, $y_1 = 1$? No. The first measurement implies $x_0 = 1$, but then the model predicts

$$y_1 = \frac{1}{2}x_0 = \frac{1}{2},$$

which is not equal to the given value 1. Therefore the pair $(y_0, y_1) = (1, 1)$ is *not* a valid free-response trajectory of this system. In other words, there exists no initial state x_0 that generates this sequence with zero input.

What are all output sequences produced by the free response? All valid free-response output sequences are exactly those of the form

$$y_t = \left(\frac{1}{2}\right)^t x_0, \qquad t = 0, 1, \ldots, L-1,$$

for some scalar $x_0 \in \mathbb{R}$. Equivalently, they are the sequences satisfying the recursion

$$y_{t+1} = \frac{1}{2}y_t,$$

and they form the one-dimensional subspace

$$\mathrm{im}(\mathcal{O}_L) = \left\{ \begin{bmatrix} 1 \\ \frac{1}{2} \\ \frac{1}{4} \\ \vdots \\ \left(\frac{1}{2}\right)^{L-1} \end{bmatrix} x_0 : \; x_0 \in \mathbb{R} \right\}.$$

This example illustrates a general fact that will be crucial later on: trajectories generated by a dynamical system are highly structured. Even when we only look at input–output data over a finite horizon, the admissible sequences belong to low-dimensional subspaces determined by the system dynamics. Data-driven predictive control exploits exactly this structure, but without explicitly identifying the matrices A, B, and C first.

7.3 Known input sequence and state reconstruction

Let us now move to the more general case in which the input sequence is not zero, but it is known. Starting from the lifted input–output relation

$$y_{[0,L-1]} = \mathcal{O}_L x_0 + \mathcal{G}_L u_{[0,L-1]},$$

we can ask again the same question as before: can we reconstruct the initial state x_0 from measured inputs and outputs?
Since the input sequence is assumed to be known, the term $\mathcal{G}_L u_{[0,L-1]}$ is also known. Therefore we can subtract its contribution from the measured output sequence and obtain

$$y_{[0,L-1]} - \mathcal{G}_L u_{[0,L-1]} = \mathcal{O}_L x_0.$$

This is exactly the same reconstruction problem as in the zero-input case, except that now the measured outputs must first be corrected by removing the forced response generated by the known input sequence.

So the answer to the question *"Can you determine x_0?"* is: yes, provided that the corrected output sequence

$$y_{[0,L-1]} - \mathcal{G}_L u_{[0,L-1]}$$

belongs to the column image of $\mathcal{O}_L$. Equivalently, there must exist an initial condition x_0 such that

$$\mathcal{O}_L x_0 = y_{[0,L-1]} - \mathcal{G}_L u_{[0,L-1]}.$$

Just as before, the horizon length L relative to the state dimension n determines whether the problem is underdetermined, exactly determined, or overdetermined:

- if $L < n$, then there are in general not enough equations to reconstruct x_0 uniquely;
- if $L = n$ and $\mathcal{O}_n$ is invertible, then the initial state is uniquely given by

$$x_0 = \mathcal{O}_n^{-1}(y_{[0,n-1]} - \mathcal{G}_n u_{[0,n-1]});$$

- if $L > n$, then the problem is overdetermined: one can reconstruct x_0 only if the measured input–output sequence is compatible with the system dynamics.

This compatibility requirement is already an important message. For $L > n$, not every arbitrary pair of sequences (u, y) can come from the system. Once the known input contribution is removed, the remaining part of the output must lie in an n-dimensional subspace. Hence admissible input–output trajectories are constrained by the dynamics, even if we do not write the state explicitly.

7.4 Behavioral description of finite-horizon trajectories

The lifted representation can be written in an even more compact form by stacking the input and output sequences together:

$$\begin{bmatrix} u_{[0,L-1]} \\ y_{[0,L-1]} \end{bmatrix} = \begin{bmatrix} I_L & 0 \\ \mathcal{G}_L & \mathcal{O}_L \end{bmatrix} \begin{bmatrix} u_{[0,L-1]} \\ x_0 \end{bmatrix}.$$

It is convenient to define the matrix

$$\Lambda_L := \begin{bmatrix} I_L & 0 \\ \mathcal{G}_L & \mathcal{O}_L \end{bmatrix} \in \mathbb{R}^{(L+L)\times(L+n)}.$$

Then every trajectory of length L generated by the system satisfies

$$\begin{bmatrix} u_{[0,L-1]} \\ y_{[0,L-1]} \end{bmatrix} = \Lambda_L \begin{bmatrix} u_{[0,L-1]} \\ x_0 \end{bmatrix}.$$

This expression is extremely useful because it describes all system trajectories in one shot. The free variables are the initial state x_0 and the input sequence $u_{[0,L-1]}$; once they are fixed, the output sequence is uniquely determined.
Observe also that, if $L \geq n$ and the system is observable, then $\mathcal{O}_L$ has rank n. Since the upper block contains the identity matrix I_L, it follows that

$$\operatorname{rank}(\Lambda_L) = L + n.$$

So Λ_L has full column rank, and its column image is an $(L + n)$-dimensional subspace of $\mathbb{R}^{2L}$.

7.4.1 Behavioral viewpoint

The previous construction leads naturally to the behavioral viewpoint. The key idea is to describe the system not primarily through states and equations, but through the set of all trajectories that are compatible with the dynamics.
From the identity above, the admissible trajectories are exactly those vectors of the form

$$\begin{bmatrix} u_{[0,L-1]} \\ y_{[0,L-1]} \end{bmatrix} \in \mathrm{im}(\Lambda_L).$$

In other words, a pair of input and output sequences is feasible for the system if and only if it belongs to the column image of Λ_L.

This is called a *behavioral representation* of the system. Instead of saying that the system is described by the matrices A, B, and C, we may equivalently say that it is described by the set of all finite-horizon trajectories it can generate. The matrix Λ_L is a complete description of this set, exactly in the same sense in which (A, B, C) is a complete model description.
This viewpoint has an important practical consequence. If we know Λ_L, then given any candidate input–output sequence we can immediately test whether it is compatible with the plant: compatibility means precisely that there exists some initial state x_0 such that the trajectory can be generated by the system. Equivalently,

$$\begin{bmatrix} u_{[0,L-1]} \\ y_{[0,L-1]} \end{bmatrix} \text{ is admissible} \qquad \Longleftrightarrow \qquad \begin{bmatrix} u_{[0,L-1]} \\ y_{[0,L-1]} \end{bmatrix} \in \mathrm{im}(\Lambda_L).$$

This perspective is central in behavioral system theory, developed by Jan C. Willems. The main philosophical shift is that one does not start by classifying variables as inputs, outputs, and states and then deriving trajectories. Instead, one starts directly from the set of all trajectories that the system can exhibit. The state-space model is then just one possible representation of that behavior.
For our purposes, this is exactly the bridge toward data-driven predictive control. If the system can be characterized through its trajectory space, then in principle we may hope to learn and manipulate this space directly from measured data, without first identifying a parametric state-space model. This is the core idea behind the data-driven predictive control methods studied next.

7.4.2 Why the horizon $L = n + 1$ is the first predictive horizon

A particularly important case is when the horizon length equals the state dimension, that is, $L = n$. In this case,

$$\begin{bmatrix} u_{[0,n-1]} \\ y_{[0,n-1]} \end{bmatrix} = \underbrace{\begin{bmatrix} I_n & 0 \\ \mathcal{G}_n & \mathcal{O}_n \end{bmatrix}}_{\Lambda_n} \begin{bmatrix} u_{[0,n-1]} \\ x_0 \end{bmatrix}.$$

The matrix Λ_n has dimension $2n \times 2n$. Since the system is observable, $\mathcal{O}_n$ has rank n, and therefore

$$\mathrm{rank}(\Lambda_n) = 2n.$$

So Λ_n is invertible. As a consequence, every input–output sequence of length n can be uniquely explained by a suitable initial condition x_0. This is exactly the observability problem: once n input–output samples are known, the initial state can be reconstructed.
To see this in the simplest possible case, consider the scalar integrator

$$x_{t+1} = x_t + u_t, \qquad y_t = x_t.$$

Here $n = 1$, so already from one output sample we get $x_0 = y_0$. There is no additional compatibility condition yet: any pair (u_0, y_0) can be explained by choosing $x_0 = y_0$.
Now let us increase the horizon to $L = n + 1$. Then

$$\begin{bmatrix} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ y_0 \\ \vdots \\ y_{n-1} \\ y_n \end{bmatrix} = \underbrace{\begin{bmatrix} I_{n+1} & 0 \\ \mathcal{G}_{n+1} & \mathcal{O}_{n+1} \end{bmatrix}}_{\Lambda_{n+1}} \begin{bmatrix} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ x_0 \end{bmatrix}.$$

The matrix Λ_{n+1} has size

$$2(n+1) \times (2n+1),$$

and rank

$$\operatorname{rank}(\Lambda_{n+1}) = (n+1) + n = 2n + 1.$$

So the admissible input–output trajectories of length $n + 1$ form a subspace of dimension $2n + 1$ inside the ambient space $\mathbb{R}^{2(n+1)}$. This means that not every arbitrary vector in $\mathbb{R}^{2(n+1)}$ is a valid trajectory anymore: one scalar compatibility condition must be satisfied.
This is the first horizon length at which the dynamics impose a genuine restriction on the input–output sequence. Up to length n, every sequence can be explained by a suitable initial condition; from length $n + 1$ onward, only a proper subspace of all possible stacked vectors is admissible.

7.4.3 Multi-step prediction as trajectory completion

This immediately leads to a one-step prediction problem. Suppose the inputs

$$u_0, \ldots, u_n$$

and the first n outputs

$$y_0, \ldots, y_{n-1}$$

are known, while the last output y_n is missing. Then we can write

$$\begin{bmatrix} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ y_0 \\ \vdots \\ y_{n-1} \\ y_n \end{bmatrix} = \underbrace{\begin{bmatrix} I_{n+1} & 0 \\ \mathcal{G}_{n+1} & \mathcal{O}_{n+1} \end{bmatrix}}_{\Lambda_{n+1}} \begin{bmatrix} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ x_0 \end{bmatrix}.$$

Can y_n be reconstructed? Yes. The reason is simple: from the first n input–output samples we can reconstruct x_0, because that is exactly the observability problem already discussed. Once x_0 is known, and since the input sequence is known up to time n, we can propagate the system

dynamics one step further and determine y_n uniquely.
The same idea extends to multi-step prediction. Let $L = n + K$ with $K \geq 1$. Then

$$\begin{bmatrix} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ \vdots \\ u_{n+K-1} \\ y_0 \\ \vdots \\ y_{n-1} \\ y_n \\ \vdots \\ y_{n+K-1} \end{bmatrix} = \underbrace{\begin{bmatrix} I_{n+K} & 0 \\ \mathcal{G}_{n+K} & \mathcal{O}_{n+K} \end{bmatrix}}_{\Lambda_{n+K}} \begin{bmatrix} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ \vdots \\ u_{n+K-1} \\ x_0 \end{bmatrix}.$$

Suppose now that the sequence

$$y_n, \ldots, y_{n+K-1}$$

is missing. Can it be reconstructed? Again yes. From the first n input–output samples we reconstruct the initial state x_0, and once x_0 is known, the known future inputs determine the future outputs uniquely. So the missing output block is exactly the unique completion for which the stacked vector belongs to the trajectory subspace of the system.
This gives a useful predictive-control interpretation. The block

$$u_0, \ldots, u_{n-1}$$

represents the past inputs, the block

$$y_0, \ldots, y_{n-1}$$

represents the past outputs, the block

$$u_n, \ldots, u_{n+K-1}$$

represents the future input sequence we choose, and the block

$$y_n, \ldots, y_{n+K-1}$$

is the future output sequence implied by trajectory compatibility.

7.4.4 Basis representation of the trajectory space

All of this admits a clean linear-algebra interpretation. For any horizon $L > n$, the set of valid input–output trajectories is a subspace of dimension $L + n$:

$$\operatorname{im}(\Lambda_L) \subseteq \mathbb{R}^{2L}, \qquad \dim\big(\operatorname{im}(\Lambda_L)\big) = L + n.$$

Therefore every admissible trajectory can be written as

$$\begin{bmatrix} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ \vdots \\ u_{n+K-1} \\ y_0 \\ \vdots \\ y_{n-1} \\ y_n \\ \vdots \\ y_{n+K-1} \end{bmatrix} = Hg,$$

where

$$H \in \mathbb{R}^{2L\times(L+n)}$$

has columns that form a basis of the trajectory subspace, and

$$g \in \mathbb{R}^{L+n}$$

is a vector of coefficients.
A natural choice for such a basis is given by the columns of Λ_L. However, constructing Λ_L requires explicit knowledge of A, B, and C. This suggests a different question: can we build another basis for the same subspace directly from data?
Since the trajectory space has dimension $L+n$, any set of $L+n$ linearly independent valid trajectories of length L forms a basis. Suppose we collect $L+n$ admissible input–output trajectories

$$\begin{bmatrix} u^{(1)}_{[0,L-1]} \\ y^{(1)}_{[0,L-1]} \end{bmatrix}, \quad \begin{bmatrix} u^{(2)}_{[0,L-1]} \\ y^{(2)}_{[0,L-1]} \end{bmatrix}, \quad \dots, \quad \begin{bmatrix} u^{(L+n)}_{[0,L-1]} \\ y^{(L+n)}_{[0,L-1]} \end{bmatrix},$$

and assume they are linearly independent. Then all and only the valid trajectories can be represented as

$$\begin{bmatrix} u_{[0,L-1]} \\ y_{[0,L-1]} \end{bmatrix} = \underbrace{\begin{bmatrix} u^{(1)}_{[0,L-1]} & u^{(2)}_{[0,L-1]} & u^{(3)}_{[0,L-1]} & \cdots & u^{(L+n)}_{[0,L-1]} \\ y^{(1)}_{[0,L-1]} & y^{(2)}_{[0,L-1]} & y^{(3)}_{[0,L-1]} & \cdots & y^{(L+n)}_{[0,L-1]} \end{bmatrix}}_{H} g, \qquad g \in \mathbb{R}^{L+n}.$$

This is the key step toward a data-driven description of the system behavior. Instead of generating the basis from the model matrices, we generate it from measured trajectories. A new admissible trajectory is then obtained as a linear combination of previously observed admissible trajectories. Conceptually, one can think of the measured signals as building blocks: if they span the trajectory space, then every other valid trajectory can be synthesized by combining them with suitable coefficients.

At this point it is useful to check these ideas on a very simple example.

7.4.5 Example: rank of the trajectory matrix

Exercise. Consider the scalar SISO system with $n = 1$

$$x_{t+1} = x_t + u_t, \qquad y_t = x_t,$$

and trajectories of length $L = 3$.
If we generate several input–output trajectories of length L by choosing random initial conditions and random input sequences, and then stack them as columns of the matrix

$$H = \begin{bmatrix} u^{(1)}_{[0,2]} & u^{(2)}_{[0,2]} & \cdots \\ y^{(1)}_{[0,2]} & y^{(2)}_{[0,2]} & \cdots \end{bmatrix},$$

what is the maximum rank that H can achieve?
Since here $n = 1$ and $L = 3$, the dimension of the trajectory subspace is

$$L + n = 3 + 1 = 4.$$

Therefore the rank of H can never exceed 4, no matter how many columns we add. If the collected trajectories are sufficiently rich, then as we keep increasing the number of columns, the rank of H will eventually saturate at

$$\operatorname{rank}(H) = 4.$$

So the maximum possible rank is 4. This is exactly what we should expect from the behavioral viewpoint: all valid trajectories of length 3 live in a 4-dimensional subspace of $\mathbb{R}^6$.

7.5 From trajectory bases to Hankel-based prediction

7.5.1 Reusing a single experiment through overlapping windows

The discussion so far suggests that, in order to build a basis of the trajectory space, we need many trajectories of length L. However, this seems wasteful: data are often expensive to collect, and generating many independent experiments may be impractical. It is therefore natural to ask whether one can reuse more efficiently the samples of a single long experiment.
Suppose that we want to predict K steps into the future for a system of dimension n. Then, as discussed above, we need trajectories of length

$$L = n + K.$$

Moreover, since the trajectory space has dimension $L + n$, we need at least

$$L + n = (n + K) + n = K + 2n$$

linearly independent trajectories to span it.
A very efficient way to extract many such trajectories from one long signal is to use overlapping windows. Consider a long input sequence

$$u_0, u_1, u_2, \ldots$$

and slide a window of length L along it. Each window produces one candidate trajectory segment:

$$u_0\ u_1\ u_2\ \cdots\ u_{L-1}, \qquad u_1\ u_2\ u_3\ \cdots\ u_L, \qquad u_2\ u_3\ u_4\ \cdots\ u_{L+1}, \quad \ldots$$

Instead of treating these windows separately, we arrange them as columns of a matrix:

$$\mathcal{H}_L(u) = \begin{bmatrix} u_0 & u_1 & u_2 & \cdots & u_{L+n-1} \\ u_1 & u_2 & u_3 & \cdots & u_{L+n} \\ u_2 & u_3 & u_4 & \cdots & u_{L+n+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ u_{L-1} & u_L & u_{L+1} & \cdots & u_{2L+n-2} \end{bmatrix}.$$

This is the *Hankel matrix* built from the signal u. The same construction can be carried out for the output signal:

$$\mathcal{H}_L(y) = \begin{bmatrix} y_0 & y_1 & y_2 & \cdots & y_{L+n-1} \\ y_1 & y_2 & y_3 & \cdots & y_{L+n} \\ y_2 & y_3 & y_4 & \cdots & y_{L+n+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ y_{L-1} & y_L & y_{L+1} & \cdots & y_{2L+n-2} \end{bmatrix}.$$

So the relevant object is no longer a matrix whose columns are trajectories coming from many different experiments, but a matrix built from many shifted windows of one long experiment. This is much more economical in terms of data usage.

7.5.2 Hankel matrices as data-driven trajectory bases

The key claim is that all system trajectories of length L can be generated as linear combinations of the columns of the stacked Hankel matrix

$$\begin{bmatrix} \mathcal{H}_L(u) \\ \mathcal{H}_L(y) \end{bmatrix} g,$$

where

$$g \in \mathbb{R}^{L+n}$$

is a free vector of coefficients. Notice that this requires input and output sequences of total length

$$2L + n - 1,$$

because that is exactly the amount of data needed to build a Hankel matrix with $L + n$ columns and window length L.

This representation should be compared with the abstract basis representation introduced before. There we wrote any admissible trajectory as

$$\begin{bmatrix} u_{[0,L-1]} \\ y_{[0,L-1]} \end{bmatrix} = Hg.$$

Now we are proposing a concrete and data-driven choice for the basis matrix H, namely

$$H = \begin{bmatrix} \mathcal{H}_L(u) \\ \mathcal{H}_L(y) \end{bmatrix}.$$

So the role of the Hankel matrix is precisely to provide a trajectory basis directly from measured data.

7.5.3 Data-driven prediction with past and future blocks

This immediately yields a data-driven K-step predictor. Recall that for prediction we choose

$$L = n + K.$$

We then split the input and output trajectories into past and future parts exactly as before:

$$\begin{bmatrix} u_{\text{past}} \\ u_{\text{future}} \\ y_{\text{past}} \\ y_{\text{future}} \end{bmatrix} = \begin{bmatrix} \mathcal{H}_L(u_{\text{data}}) \\ \mathcal{H}_L(y_{\text{data}}) \end{bmatrix} g.$$

Here the blocks have the following meaning:

- u_{past} is the measured past input trajectory,
- u_{future} is the future input sequence chosen by the controller,
- y_{past} is the measured past output trajectory,
- y_{future} is the unknown future output sequence to be predicted.

In the notation used in the figures, one often highlights the unknown quantities by writing

$$g \in \mathbb{R}^{2n+K} \qquad \text{and} \qquad y_{\text{future}} \in \mathbb{R}^{K}.$$

These are exactly the unknowns of the prediction problem. Their total number is

$$(2n + K) + K = 2(n + K) = 2L.$$

On the other hand, the stacked equation above provides exactly $2L$ scalar equations. So, at least at the counting level, the predictor is square: the number of unknowns matches the number of equations.

The interpretation is very appealing. Given the measured past input–output data and a candidate future input sequence, we solve for a coefficient vector g such that the complete trajectory is compatible with the data matrix. Once such a g is found, the future output block is obtained automatically. In other words, prediction is performed by enforcing membership in the trajectory space spanned by the measured data.

7.6 From data-driven prediction to data-enabled predictive control

7.6.1 From prediction to control

This data-driven predictor has several natural uses:

- filtering,
- recovering missing data,
- prediction and simulation,
- and, most importantly for us, control.

This is the crucial point where trajectory-based prediction becomes predictive control. Instead of only asking which future output is compatible with a chosen future input, we will soon ask which future input should be chosen so that the predicted future output satisfies our control objective.

We are now ready to turn this predictor into a control law. To make the connection with the standard MPC formulation as transparent as possible, let us first recall the model-based

output-tracking MPC problem in the SISO case.
At each time step, the current state x is measured and one solves

$$\min_{u,x,y} \sum_{k=0}^{K-1} y_k^2 + r u_k^2$$

subject to

$$x_{k+1} = Ax_k + Bu_k, \qquad y_k = Cx_k, \qquad x_0 = x,$$

and the constraints

$$u_k \in \mathcal{U}_k \quad \forall k, \qquad y_k \in \mathcal{Y}_k \quad \forall k.$$

The role of the model equations is to enforce that the decision variables u and y are genuine trajectories of the plant over the next K steps. In other words, the model is exactly the mechanism that couples candidate future inputs to dynamically feasible future outputs.

7.6.2 The data-enabled predictive control formulation

The behavioral viewpoint suggests a different way to impose exactly the same requirement. Instead of using the parametric state-space model, we can use the trajectory space generated by the measured data. This leads to the *data-enabled predictive control* formulation.
Using the Hankel representation introduced above, we solve

$$\min_{u,y,g} \sum_{k=0}^{K-1} y_k^2 + r u_k^2$$

subject to

$$\begin{bmatrix} \mathcal{H}_L(u_{\text{data}}) \\ \mathcal{H}_L(y_{\text{data}}) \end{bmatrix} g = \begin{bmatrix} u_{\text{past}} \\ u \\ y_{\text{past}} \\ y \end{bmatrix},$$

and

$$u_k \in \mathcal{U}_k \quad \forall k, \qquad y_k \in \mathcal{Y}_k \quad \forall k.$$

Here the state-space model has been replaced by the new behavioral model. The variables

$$u_{\text{past}}, \qquad y_{\text{past}}$$

are the last measured input and output samples of the system. They play exactly the role that the initial condition $x_0 = x$ played in the model-based formulation: they encode all the information needed to determine which future trajectories are compatible with the current operating condition of the plant.

Compared with the model-based MPC problem, a new decision variable g has appeared. This variable does not have a direct physical meaning such as state, input, or output. Its purpose is simply to express the predicted trajectory as a linear combination of columns of the data matrix. In this sense, g is the coefficient vector that selects one admissible trajectory inside the behavioral subspace.
It is instructive to compare the two formulations side by side. In model-based MPC, the dynamic feasibility of the future signals is enforced through

$$x_{k+1} = Ax_k + Bu_k, \qquad y_k = Cx_k, \qquad x_0 = x.$$

In data-enabled predictive control, the same role is played by the single trajectory-consistency equation

$$\begin{bmatrix} \mathcal{H}_L(u_{\text{data}}) \\ \mathcal{H}_L(y_{\text{data}}) \end{bmatrix} g = \begin{bmatrix} u_{\text{past}} \\ u \\ y_{\text{past}} \\ y \end{bmatrix}.$$

So the logic of the optimization problem is unchanged: we still optimize over future inputs and outputs subject to input and output constraints. The only difference is how we describe the set of feasible trajectories. In one case we use the matrices A, B, C; in the other we use measured data.

7.6.3 Comparison with model-based MPC

This replacement has a few immediate consequences.

- The data-enabled formulation is computationally more complex, because it introduces $2n + K$ additional decision variables through the coefficient vector g.
- Its memory footprint is larger, because we store a data matrix rather than a compact parametric model (A, B, C).
- On the other hand, no additional state estimator is needed: the past input–output trajectory $(u_{\text{past}}, y_{\text{past}})$ already plays the role of the initial condition.

This last point is conceptually important. In model-based MPC, if the state is not directly measured, one typically needs an observer or estimator to reconstruct it. In the behavioral formulation, the recent past trajectory already contains the information needed to anchor the prediction to the current system condition. So the controller works directly with measured input–output data, without explicitly reconstructing a state.
We should therefore think of data-enabled predictive control as a trajectory-based counterpart of standard MPC:

- same objective,
- same input and output constraints,
- same receding-horizon logic,
- but a different internal description of the plant.

The model-based controller enforces feasibility through equations in the state; the data-enabled controller enforces feasibility through membership in the trajectory space spanned by data.

7.7 Practical issues: excitation, MIMO extension, and noise

7.7.1 Persistence of excitation

Before going further, it is worth pausing for a few practical remarks. The previous construction relied on one crucial assumption: the collected data must be sufficiently informative. In system identification language, this is the assumption of *persistence of excitation*.

More precisely, we assumed that the input used for data collection is rich enough so that the resulting Hankel data matrix has full rank. Technically, this is needed to ensure that the measured trajectories span the relevant trajectory subspace. Intuitively, the meaning is simple: if the data do not excite all relevant modes of the system, then the collected trajectories cannot represent the full dynamics of the plant, and the behavioral model built from them will be incomplete.

This requirement is completely standard in identification. In practice, it is typically handled in one of the following ways:

- by applying deliberately rich excitation signals during data collection, for example random inputs or PRBS-type inputs;
- in some applications, by relying on ambient or process noise to provide enough excitation;
- by using historical data and then checking *a posteriori* whether the resulting data matrix has the required rank.

So persistence of excitation is not a special drawback of data-enabled predictive control; it is the usual price one pays whenever one wants to learn a dynamical model from data.

7.7.2 Extension from SISO to MIMO

Up to now we have mostly discussed the SISO case, because it makes the geometry of the construction easier to see. However, the theory extends naturally to the MIMO case. Suppose now that

$$u_t \in \mathbb{R}^m, \qquad y_t \in \mathbb{R}^p, \qquad x_t \in \mathbb{R}^n.$$

The main conceptual difference is that, in the multi-input multi-output setting, the number of past samples needed to encode the current state information is not necessarily equal to the state dimension n. Instead, the relevant quantity is the *lag* of the system.
The lag is defined as the smallest integer ℓ such that the extended observability matrix

$$\mathcal{O}_\ell = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{\ell-1} \end{bmatrix}$$

has rank n. In other words, ℓ is the smallest number of consecutive output samples needed so that the observability matrix reaches full column rank. This quantity plays exactly the role that n played in the SISO discussion. In particular, a past trajectory of length

$$T_{\text{ini}} \geq \ell$$

is needed in order to play the role of the initial condition. Thus, in the MIMO case, the past input–output data block

$$(u_{\text{past}}, y_{\text{past}})$$

must contain at least T_{ini} samples, with T_{ini} no smaller than the lag.

The amount of data required to build the Hankel representation also changes. For MIMO systems, if we want to predict over a horizon of length K, and if we use a past window of length T_{ini}, then an input–output sequence of length at least

$$T \;\geq\; (m+1)(T_{\text{ini}} + K) + n - 1$$

is needed. For SISO systems, where $m = 1$ and $\ell = n$, this reduces to the condition already encountered earlier.
This observation has an important practical implication. Since always

$$\ell \leq n,$$

in practice one often needs at least an approximate estimate of the minimal realization order n, or at least of a suitable upper bound. This is because the choice of T_{ini} and the amount of data

needed both depend on these structural quantities.

How can such an estimate be obtained? There are two common routes. One is to use first-principles knowledge of the plant. For example, in many electro-mechanical systems one often has a fairly good idea of the system order from physics. The other is to perform an *a posteriori* verification from data, for example by building increasingly large Hankel matrices and checking when the relevant rank conditions stabilize.
Once an estimate of the order is available, the safe choice

$$T_{\text{ini}} = n$$

is always valid, although in some cases side information allows one to use a smaller value, closer to the true lag ℓ. The prediction horizon K, instead, is chosen from the control specifications, exactly as in standard MPC.

So the practical design procedure is conceptually quite similar to model-based predictive control:

- choose K from the control task;
- choose T_{ini} large enough to encode the current condition of the system;
- collect sufficiently exciting data so that the Hankel matrix has the right rank properties.

These remarks clarify an important point. Data-enabled predictive control does not remove the need for structural information entirely. Rather, it shifts the emphasis: instead of identifying explicitly the matrices A, B, C, one needs enough information to choose the data window lengths correctly and enough excitation so that the data span the relevant trajectory space.

7.7.3 Effect of noise on the trajectory representation

A final important issue is the effect of noise. Up to this point, the discussion has implicitly relied on exact trajectories and exact algebraic relations. In that ideal setting, adding more and more columns to the data matrix does not hurt. On the contrary, it is beneficial:

- it helps when we are uncertain about the true system order n,
- it increases the chance that all system modes have been sufficiently excited,
- and, once enough informative trajectories have been collected, the rank of the data matrix stops increasing.

In the noiseless case, this saturation is exactly what we expect. The set of valid trajectories has finite dimension, so after collecting enough independent columns, new columns can only be linear combinations of the previous ones. For a MIMO system with input dimension m, the relevant dimension is $mL + n$, and therefore the rank of the trajectory matrix H cannot grow beyond that value.
However, this conclusion is true only in the noiseless case. In the presence of noise in the data, the situation changes qualitatively: the measured trajectories are no longer exactly confined to the true trajectory subspace. As a consequence, the rank of the empirical matrix H may continue to increase past the nominal value $mL + n$. In other words, noise artificially inflates the apparent dimension of the trajectory space.

This is more than a purely algebraic inconvenience. It can have a serious control consequence, because the controller may start interpreting noisy directions as if they were true dynamical directions of the system.

To understand the mechanism, consider the following pathological example. Suppose that one component of the output is identically zero, so that the output equation has the form

$$y_k = Cx_k = \begin{bmatrix} \star & \star & \star \\ 0 & 0 & 0 \end{bmatrix} x_k.$$

Then every true output vector must be of the form

$$y_k = \begin{bmatrix} \star \\ 0 \end{bmatrix}.$$

So the second output component is completely unaffected by the state: it is structurally zero and cannot be influenced by the control input.

Now imagine that the data used to build the trajectory matrix are noisy. Then some measured output trajectories may contain a small nonzero component in that second output coordinate. Symbolically, one of the columns of the data matrix may look like

$$\underbrace{\begin{bmatrix} u_L^{(1)} & u_L^{(2)} & u_L^{(3)} & \cdots \\ y_L^{(1)} & y_L^{(2)} & y_L^{(3)} + \varepsilon & \cdots \end{bmatrix}}_{H} g.$$

Because the controller only sees the data matrix, it may incorrectly infer that the input sequence $u_L^{(3)}$ is capable of affecting that last output component. But this is false: the apparent effect is caused only by measurement noise.

This misunderstanding can be dangerous. The optimization may try to exploit that fake direction by assigning a large coefficient to the corresponding column. In other words, it may choose a large g, and therefore indirectly a large input, in an attempt to regulate an output component that is in fact uncontrollable. So noise can lead to aggressive and meaningless control actions if the raw trajectory parameterization is used without protection.

7.7.4 Regularization and slack variables

This is the motivation for *regularized* data-enabled predictive control. The key observation is that, in theory, only $L+n$ independent components of g are really needed. If many more columns are present in the data matrix, then the representation of a trajectory through g becomes highly non-unique, especially in the presence of noise. A natural way to promote robustness is therefore to penalize the size of g.

A common choice is to add an ℓ_1-regularization term:

$$\min_{u,y,g} \sum_{k=0}^{K-1} y_k^2 + r u_k^2 + \lambda_g \|g\|_1$$

subject to

$$\begin{bmatrix} \mathcal{H}_L(u_{\text{data}}) \\ \mathcal{H}_L(y_{\text{data}}) \end{bmatrix} g = \begin{bmatrix} u_{\text{past}} \\ u \\ y_{\text{past}} \\ y \end{bmatrix},$$

and

$$u_k \in \mathcal{U}_k \quad \forall k, \qquad y_k \in \mathcal{Y}_k \quad \forall k.$$

The role of the term $\lambda_g \|g\|_1$ is to discourage the use of unnecessarily many columns of the data matrix. In other words, it promotes sparse coefficient vectors, which is consistent with the idea that only a limited number of truly independent directions should matter. This improves robustness to noise, because spurious noisy directions are less likely to be exploited heavily.
There is, however, a tradeoff. Larger values of λ_g increase robustness, but they also bias the solution and may degrade nominal optimality. Intuitively, stronger regularization corresponds to choosing a simpler effective model hidden inside the data. In fact, one may interpret a larger value of λ_g as promoting a description with a smaller effective order. From this viewpoint, the sparsity pattern of g also carries structural information: *a posteriori*, the quantity

$$\|g\|_0$$

gives an indication of how many columns are actually being used, and therefore of the effective model size suggested by the data.

So far, the regularization addressed noise in the *offline data matrix*. But noise may also affect the *real-time measurements* used as initial conditions, namely the past input–output block $(u_{\text{past}}, y_{\text{past}})$. This creates a second difficulty.
Indeed, the measured sequence of length T_{ini} may itself fail to be exactly compatible with the trajectory space generated by the data matrix. In other words, because of online measurement noise, there may exist no exact g satisfying

$$\begin{bmatrix} \mathcal{H}_L(u_{\text{data}}) \\ \mathcal{H}_L(y_{\text{data}}) \end{bmatrix} g = \begin{bmatrix} u_{\text{past}} \\ u \\ y_{\text{past}} \\ y \end{bmatrix}.$$

To handle this, one introduces a slack variable σ on the noisy past output block and penalizes it:

$$\min_{u,y,g,\sigma} \sum_{k=0}^{K-1} y_k^2 + r u_k^2 + \lambda_g \|g\|_1 + \lambda_\sigma \|\sigma\|_1$$

subject to

$$\begin{bmatrix} \mathcal{H}_L(u_{\text{data}}) \\ \mathcal{H}_L(y_{\text{data}}) \end{bmatrix} g = \begin{bmatrix} u_{\text{past}} \\ u \\ y_{\text{past}} + \sigma \\ y \end{bmatrix},$$

and

$$u_k \in \mathcal{U}_k \quad \forall k, \qquad y_k \in \mathcal{Y}_k \quad \forall k.$$

Here the slack σ allows the past measured output trajectory to be adjusted slightly so that it becomes compatible with the data-driven model. The penalty $\lambda_\sigma \|\sigma\|_1$ keeps this correction as small as possible. So the optimization is now balancing four goals at once:

- good tracking performance,
- moderate control effort,
- sparse and robust trajectory coefficients g,
- and small corrections of the noisy real-time measurements.

This should be compared with the classical identification-based pipeline. There too, one must deal with noise: first in identifying a model, then in estimating the state, and then again in applying MPC on top of the estimated model. The data-enabled approach does not eliminate

the noise problem, but it handles it directly at the trajectory level. In the course perspective, this is one of its main appeals: instead of going through the chain

$$\text{data} \;\rightarrow\; \text{system identification} \;\rightarrow\; \text{state-space model} \;\rightarrow\; \text{MPC},$$

one works directly with

$$\text{data} \;\rightarrow\; \text{data-enabled control}.$$

There is increasing recent evidence that this direct route can be more robust to noise than the indirect identification-then-control pipeline, especially when the available data are limited or the identified model is imperfect.

So the main takeaway is the following. In the noiseless setting, trajectory spaces have the exact low-dimensional structure predicted by the theory. In the noisy setting, this structure is blurred, and the controller must be regularized. Penalizing g protects against spurious directions in the offline data matrix, while penalizing a slack variable σ protects against inconsistency in the online measurements. These two modifications are what make data-enabled predictive control usable in practice.

7.8 Limitations and final perspective

7.8.1 Main limitations

We conclude the chapter with a few remarks on the limitations of data-driven predictive control. Up to this point, the method may have looked like a very general replacement for model-based MPC. It is powerful, but it is not universally the best choice.
A first difficulty appears when *state constraints* are important. The behavioral formulation is naturally expressed in terms of input and output trajectories. If a particular internal state is known, has a physical meaning, and must be explicitly constrained, then a model-based approach may be preferable. In a state-space model, such constraints can be imposed directly on the state variable. In a purely data-driven formulation, by contrast, the state is not explicit, so incorporating such information is more cumbersome.

A second limitation concerns *prior or side information*. Sometimes one does not know the full model, but one does know something important about the system: admissible parameter ranges, nominal values, structural properties, conservation laws, or the specific form of the dynamics. Classical system identification can often incorporate this kind of information naturally into the model structure or the estimation procedure. A pure trajectory-based description is less flexible in this respect, because it treats the plant mainly through observed behaviors rather than through interpretable parameters.

A third issue is *nonlinearity*. The whole theory developed in this chapter relies strongly on linear time-invariant system theory. The finite-dimensional trajectory-space representation, the Hankel parameterization, and the exact behavioral characterization are all fundamentally LTI results. Extending these ideas to nonlinear systems is much more complicated. There are important research directions in this area, but the clean linear-algebraic picture we used here does not carry over directly.

Finally, there is the issue of *computational complexity*. A data representation is typically less compact than a state-space model. Once a concise model (A, B, C) is available, predictions can be generated with a small number of variables and equations. In contrast, the data-enabled formulation stores entire trajectory matrices and introduces the coefficient vector g, whose

dimension grows with the data length and the chosen horizons. So the price one pays for avoiding explicit modeling is increased memory usage and, often, increased online computation time.
These limitations should not be read as a rejection of data-driven control. Rather, they clarify when the method is especially attractive and when a model-based route may still have an advantage. If one has a good low-order model, strong prior knowledge, or crucial state constraints, then model-based MPC remains very compelling. If instead one wants to avoid an explicit identification step and has rich measured trajectories available, then the data-enabled route can be very attractive.

7.8.2 When data-driven predictive control is attractive

It is also worth ending on a broader perspective. At first sight, one might think that such a data-driven method is too fragile to be trusted on large-scale engineered systems, especially when:

- the plant is extremely complex,
- the actuators are very large and expensive,
- and the controller must satisfy real-time, safety, and stability requirements.

Yet this intuition would be too pessimistic. Data-enabled predictive control has already attracted attention precisely in demanding application areas, including large-scale power systems and high-voltage direct-current links. The reason is simple: in such systems, obtaining an accurate model can itself be extremely difficult, while high-quality trajectory data are often available. In those situations, a direct trajectory-based controller can become competitive, and sometimes very appealing, because it bypasses part of the modeling burden.
So the final message of the chapter is balanced. Data-driven predictive control is not magic, and it is not a universal substitute for modeling. It has clear weaknesses: handling state constraints is harder, incorporating side information is less natural, nonlinear systems remain challenging, and the data-based representation is computationally heavier. At the same time, it is much more than a theoretical curiosity. Even in highly demanding applications, the approach can work surprisingly well and can offer a serious alternative to the classical pipeline

$$\text{data} \;\rightarrow\; \text{identify model} \;\rightarrow\; \text{estimate state} \;\rightarrow\; \text{run MPC}.$$

This is what makes the topic interesting: the method is limited, but genuinely useful.

8 Markov Decision Processes

8.1 Motivation: stochastic decision problems

So far, most of the control problems considered in these notes have been written in the form of a deterministic state-space model

$$x_{k+1} = f(x_k, u_k),$$

possibly with constraints and disturbances. This representation is extremely useful when the state is continuous, the dynamics are known with sufficient accuracy, and the effect of the input can be described by deterministic equations.

However, there are many systems for which this description is not the most natural one. A typical example is the control of traffic lights at an intersection. The control input is discrete: at each time we decide which light is red and which light is green. The state may also be naturally discrete: for instance, the number of vehicles in each queue. Moreover, the evolution is stochastic, because new cars arrive randomly. In such a setting, writing a smooth deterministic state-space model is not very convenient.
Consider, for simplicity, a single queue with queue length q_k. At each time step, a new vehicle arrives with probability p. If the light is red, no cars are served. If the light is green, up to three cars can leave the queue. A simple model is therefore

$$a_k = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1-p \end{cases}$$

and

$$q_{k+1} = \begin{cases} q_k + a_k & \text{if the light is red} \\ \max(0, q_k - 3) + a_k & \text{if the light is green.} \end{cases}$$

If the queue length is bounded by a maximum capacity $q_{\max}$, one can additionally saturate the update at $q_{\max}$.
This is still a Markovian model: once the current queue length q_k and the current action are known, the probability distribution of q_{k+1} is fully determined. The future does not depend on the entire past history of the queue, but only on the present state and the action chosen now. The difference with the deterministic models used before is that the next state is not a single value, but a probability distribution over possible next states.
This motivates the introduction of Markov Decision Processes.

8.2 Finite Markov Decision Processes

A finite Markov Decision Process, or MDP, is a controlled stochastic dynamical system described by the following objects:

$$\mathcal{X} = 1, \dots, N, \qquad \mathcal{U} = 1, \dots, M,$$

where $(\mathcal{X})$ is the finite set of states and $(\mathcal{U})$ is the finite set of actions.
The dynamics are described by transition probabilities

$$P : \mathcal{X} \times \mathcal{U} \times \mathcal{X} \to [0, 1].$$

We write

$$P^u_{x,x'} = \mathbb{P}[x_{k+1} = x' \mid x_k = x,\ u_k = u].$$

Thus $P^u_{x,x'}$ is the probability of moving from state x to state x' when action u is applied.
For every fixed state-action pair (x, u), the transition probabilities satisfy

$$P^u_{x,x'} \geq 0, \qquad \sum_{x' \in \mathcal{X}} P^u_{x,x'} = 1.$$

The Markov property means precisely that

$$\mathbb{P}[x_{k+1} = x' \mid x_k, u_k, x_{k-1}, u_{k-1}, \dots, x_0] = \mathbb{P}[x_{k+1} = x' \mid x_k, u_k].$$

In words, the distribution of the next state depends only on the current state and current action, not on how the system reached the current state.

The immediate cost is a function

$$c : \mathcal{X} \times \mathcal{U} \times \mathcal{X} \to \mathbb{R},$$

where

$$c(x, u, x')$$

is the cost incurred when the system is in state x, action u is chosen, and the next state turns out to be x'. Since x' is random, it is useful to define the expected one-stage cost

$$C_x^u := \mathbb{E}[c(x, u, x_{k+1}) \mid x_k = x,\ u_k = u] = \sum_{x' \in \mathcal{X}} P_{x,x'}^u c(x, u, x').$$

Finite MDP

A finite Markov Decision Process is described by

$$(\mathcal{X}, \mathcal{U}, P, c),$$

where (X) is a finite state space, $\mathcal{U}$ is a finite action space, $(P_{x,x'}^u)$ are the transition probabilities, and $(c(x, u, x'))$ is the one-step cost.

8.2.1 Policies

The control design problem for an MDP consists in selecting actions based on the current state. This is done through a policy. A **deterministic policy** is a map

$$\mu : \mathcal{X} \to \mathcal{U}.$$

If the system is in state x, the controller applies

$$u = \mu(x).$$

Thus a deterministic policy assigns exactly one action to each state.

A **stochastic policy**, also called a mixed policy, assigns a probability distribution over actions:

$$\pi : \mathcal{X} \times \mathcal{U} \to [0, 1],$$

where

$$\pi(x, u) = \mathbb{P}[u_k = u \mid x_k = x].$$

For every state x,

$$\pi(x, u) \geq 0, \qquad \sum_{u \in \mathcal{U}} \pi(x, u) = 1.$$

Thus, when the system is in state x, the action is sampled according to the probabilities $\pi(x, u)$.

Deterministic policies are a special case of stochastic policies. Indeed, a deterministic policy μ corresponds to the stochastic policy

$$\pi(x, u) = \begin{cases} 1, & u = \mu(x),\ 0, \\ u \neq \mu(x). \end{cases}$$

In many control applications, one ultimately wants a deterministic controller. Nevertheless, the stochastic formulation is very convenient mathematically, because it allows us to write policy optimization problems as optimization problems over probability distributions. This viewpoint will also be important when discussing learning-based methods.

8.3 Finite-horizon dynamic programming

8.3.1 Finite-horizon sequential decision problems

We now consider an MDP evolving through time. The index k denotes the time step. At time k, the system is in state $x_k \in \mathcal{X}$, the controller selects an action $u_k \in \mathcal{U}$, and the next state x_{k+1} is generated according to the transition probabilities

$$\mathbb{P}[x_{k+1} = x' \mid x_k = x,\ u_k = u] = P^u_{x,x'}.$$

The one-stage cost incurred at time k is

$$c_k = c(x_k, u_k, x_{k+1}).$$

Thus c_k is the cost paid when the system moves from x_k to x_{k+1} under the input u_k.

As control designers, we are interested in choosing a policy that gives good performance over a time horizon. In a finite-horizon problem, a natural performance index is

$$\sum_{k=0}^{K} c_k.$$

Since the system is stochastic, this quantity is random. Therefore, for a policy sequence

$$\pi_0, \pi_1, \ldots, \pi_K,$$

we evaluate performance through the conditional expected cost

$$V^\pi(x) = \mathbb{E}\left[\sum_{k=0}^{K} c_k \;\middle|\; x_0 = x\right].$$

The goal is to find a policy sequence that minimizes this expected cumulative cost.

8.3.2 Dynamic programming recursion for a fixed policy

To solve the finite-horizon problem recursively, we introduce the value function from an intermediate time k:

$$V^\pi_k(x) = \mathbb{E}\left[\sum_{i=k}^{K} c_i \;\middle|\; x_k = x\right].$$

This quantity represents the expected future cost from time k to the end of the horizon, assuming that the system is currently in state x and that the policy sequence

$$\pi_k, \pi_{k+1}, \ldots, \pi_K$$

is used from time k onward.

The key point is that the value function can be written recursively. Indeed,

$$V^\pi_k(x) = \mathbb{E}\left[\sum_{i=k}^{K} c_i \;\middle|\; x_k = x\right]$$

can be split into the immediate cost at time k and the future cost from time $k+1$:

$$V^\pi_k(x) = \mathbb{E}\left[c_k + \sum_{i=k+1}^{K} c_i \;\middle|\; x_k = x\right].$$

Now we condition on the action u chosen at state x, and on the next state x'. The policy chooses action u with probability $\pi_k(x,u)$, and the system moves to x' with probability $P^u_{x,x'}$. Therefore,

$$V^\pi_k(x) = \sum_{u\in\mathcal{U}} \pi_k(x,u) \left[C^u_x + \sum_{x'\in\mathcal{X}} P^u_{x,x'} \mathbb{E}\left[\sum_{i=k+1}^{K} c_i \;\middle|\; x_{k+1} = x' \right] \right],$$

where

$$C^u_x = \mathbb{E}[c_k \mid x_k = x,\ u_k = u]$$

is the expected immediate cost when action u is applied in state x.

By definition of the value function at the next time step, we have

$$\mathbb{E}\left[\sum_{i=k+1}^{K} c_i \;\middle|\; x_{k+1} = x' \right] = V^\pi_{k+1}(x').$$

Hence the recursion becomes

$$V^\pi_k(x) = \sum_{u\in\mathcal{U}} \pi_k(x,u) \left[C^u_x + \sum_{x'\in\mathcal{X}} P^u_{x,x'} V^\pi_{k+1}(x') \right].$$

This is the dynamic programming recursion for the value of a fixed policy.

The recursion should be read backward in time. At the final stage, only the last stage cost remains. Equivalently, one can set

$$V^\pi_{K+1}(x) = 0$$

and apply the same recursion for $k = K, K-1, \ldots, 0$. Thus the finite-horizon value function is computed by backward induction.

8.3.3 Bellman's optimality principle and backward induction

The optimal value from stage k is defined as the minimum expected cost that can be achieved from state x:

$$V^\star_k(x) = \min_{\pi_k,\pi_{k+1},\ldots,\pi_K} \mathbb{E}\left[\sum_{i=k}^{K} c_i \;\middle|\; x_k = x \right].$$

By Bellman's optimality principle, once the first action has been selected and the next state has been reached, the remaining decisions must be optimal for the subproblem starting from that new state. Therefore the optimal value satisfies

$$V^\star_k(x) = \min_{\pi_k(x,\cdot)} \sum_{u\in\mathcal{U}} \pi_k(x,u) \left[C^u_x + \sum_{x'\in\mathcal{X}} P^u_{x,x'} V^\star_{k+1}(x') \right].$$

This equation is the stochastic analogue of the deterministic Bellman recursion. The term C^u_x is the immediate expected cost, while

$$\sum_{x'\in\mathcal{X}} P^u_{x,x'} V^\star_{k+1}(x')$$

is the expected optimal future cost after applying action u.

For each fixed state x, the minimization is over the probability distribution $\pi_k(x,\cdot)$. Hence it is a linear program over the probability simplex

$$\Delta(\mathcal{U}) = \left\{ \nu \in \mathbb{R}^M : \nu_u \geq 0,\ \sum_{u\in\mathcal{U}} \nu_u = 1 \right\}.$$

Indeed, for fixed $V^\star_{k+1}$, the quantity

$$C^u_x + \sum_{x' \in \mathcal{X}} P^u_{x,x'} V^\star_{k+1}(x')$$

is just a scalar number associated with action u. Therefore the objective is linear in the probabilities $\pi_k(x,u)$.

Since a linear function over a simplex attains its minimum at an extreme point, under the usual finite-state and finite-action assumptions there exists an optimal deterministic action. Thus the Bellman recursion can be written equivalently as

$$V^\star_k(x) = \min_{u \in \mathcal{U}} \left[C^u_x + \sum_{x' \in \mathcal{X}} P^u_{x,x'} V^\star_{k+1}(x') \right].$$

An optimal deterministic policy is then obtained by selecting

$$\mu^\star_k(x) \in \arg\min_{u \in \mathcal{U}} \left[C^u_x + \sum_{x' \in \mathcal{X}} P^u_{x,x'} V^\star_{k+1}(x') \right].$$

Finite-horizon Bellman recursion

For a finite-horizon MDP, set $V^\star_{K+1}(x) = 0$. Then, for $k = K, K-1, \dots, 0$,

$$V^\star_k(x) = \min_{\pi_k(x,\cdot)} \sum_{u \in \mathcal{U}} \pi_k(x,u) \left[C^u_x + \sum_{x' \in \mathcal{X}} P^u_{x,x'} V^\star_{k+1}(x') \right].$$

Equivalently, since an optimal deterministic action exists,

$$V^\star_k(x) = \min_{u \in \mathcal{U}} \left[C^u_x + \sum_{x' \in \mathcal{X}} P^u_{x,x'} V^\star_{k+1}(x') \right].$$

The finite-horizon problem is therefore solved by backward induction:

- $V^\star_{K+1}$ acts as the base case;
- $K+1$ backward steps are performed;
- at each step, one policy π_k is found;
- for each state x, the backward step is a linear program over the action probabilities;
- under weak conditions, the optimal policy can be chosen deterministic.

This is a very general method: the state space is discrete, the dynamics may be nonlinear, and the evolution is stochastic. The catch is computational. If the state space is very large, then the value $V^\star_k(x)$ must be computed and stored for many states, and this quickly becomes expensive. This is the usual curse of dimensionality of dynamic programming.

8.4 Infinite-horizon discounted MDPs

8.4.1 Stationary problems and discounted cost

In many MDP problems, there is no natural final time. For instance, a traffic light controller is not designed to operate only until a fixed terminal time; it should run continuously. This leads to infinite-horizon problems.

In the infinite-horizon setting, we typically focus on stationary problems:

- the transition probabilities do not depend on time;
- the expected cost C_x^u does not depend on time;
- the policy π is time invariant.

Thus the same policy is used at every time step:

$$\pi(x,u) = \mathbb{P}[u_k = u \mid x_k = x], \qquad k = 0,1,2,\dots$$

A standard infinite-horizon performance index is the discounted cost

$$J = \sum_{k=0}^{\infty} \gamma^k c_k,$$

where

$$0 \le \gamma < 1$$

is the discount factor.

8.4.2 Discount factor

The discount factor has several interpretations.

From a mathematical perspective, the condition $\gamma < 1$ ensures that the infinite sum is finite, provided that the one-stage costs remain bounded. Indeed, the weights

$$1, \gamma, \gamma^2, \dots$$

form a convergent geometric sequence.

In some applications, the discount factor represents the lower impact of future costs. For example, in economics, money spent later may be considered less costly than money spent today, because money available today could be invested.

Another interpretation is probabilistic. The factor γ can be seen as a proxy for a Bernoulli termination probability. If at each time step the process continues with probability γ, then future costs are weighted by the probability that the process has not yet terminated.

The extreme cases are also informative:

- if $\gamma = 0$, only the immediate cost is considered;
- if $0 < \gamma < 1$, the controller balances immediate cost and future cost;
- if γ is close to 1, future costs become almost as important as current costs.

8.4.3 Closed-loop MDP and stationary distribution

Once a policy π is fixed, the MDP becomes a Markov chain. Indeed, the action is no longer a free decision variable: it is sampled according to the policy. Therefore, the transition probability from x to x' under the closed-loop policy is

$$P^{\pi}_{x,x'} = \sum_{u \in \mathcal{U}} \mathbb{P}[x' \mid x,u]\mathbb{P}[u \mid x] = \sum_{u \in \mathcal{U}} \pi(x,u) P^u_{x,x'}.$$

The matrix P^π is the transition matrix of the closed-loop Markov chain.

Let $d_k(x)$ be the probability that the system is in state x at time k. If d_k is written as a column vector, then the distribution evolves as

$$d_{k+1} = (P^\pi)^\top d_k.$$

Equivalently, if distributions are written as row vectors, then

$$d_{k+1}^\top = d_k^\top P^\pi.$$

This evolution is linear in the probability distribution.

A stationary distribution for the closed-loop Markov chain is a probability vector $\bar{d}_\pi$ satisfying

$$\bar{d}_\pi^\top = \bar{d}_\pi^\top P^\pi.$$

Thus $\bar{d}_\pi$ is a left eigenvector of P^π associated with the eigenvalue 1, normalized so that its entries sum to one. The component $\bar{d}_\pi(x)$ gives the steady-state probability that the system is in state x.

If the Markov chain is ergodic, meaning that every state can be reached from every other state through some path, then this stationary distribution is unique and describes the long-run fraction of time spent in each state.

8.4.4 Closed-loop queue example

Consider again the traffic-light queue example, with states

$$\mathcal{X} = \{0, 1, 2, 3\}.$$

Suppose we use the deterministic policy

$$\mu(q) = \begin{cases} \text{green} & \text{if } q \geq 3, \\ \text{red} & \text{otherwise.} \end{cases}$$

Hence the light is red for $q = 0, 1, 2$, and it turns green when the queue reaches length 3.

Using the state ordering $0, 1, 2, 3$, the closed-loop transition matrix is

$$P^\pi = \begin{bmatrix} 1-p & p & 0 & 0 \\ 0 & 1-p & p & 0 \\ 0 & 0 & 1-p & p \\ 1-p & p & 0 & 0 \end{bmatrix}.$$

The first row means that, from queue length 0, the queue stays at 0 with probability $1 - p$ and moves to 1 with probability p. The same logic applies to queue lengths 1 and 2. The last row is different because, when $q = 3$, the light is green and three cars are served; after that, either no new car arrives and the next queue length is 0, or one new car arrives and the next queue length is 1.

The stationary distribution is obtained by solving

$$\bar{d}_\pi^\top = \bar{d}_\pi^\top P^\pi$$

together with the normalization condition

$$\sum_{x \in \mathcal{X}} \bar{d}_\pi(x) = 1.$$

For example, if $p = 0.2$, one obtains approximately

$$\bar{d}_\pi = \begin{bmatrix} 0.267 \\ 0.333 \\ 0.333 \\ 0.067 \end{bmatrix}.$$

Therefore, under this policy, the queue is empty about 26.7% of the time, has one car about 33.3% of the time, has two cars about 33.3% of the time, and has three cars about 6.7% of the time.

8.5 Policy evaluation and Bellman optimality

8.5.1 Infinite-horizon value of a policy

For a fixed stationary policy π, the infinite-horizon discounted value is defined as

$$V^{\pi}(x) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k c_k \;\middle|\; x_0 = x\right].$$

This is the conditional expected future cost when the process starts from state x and policy π is used forever.

The value of a policy satisfies the discounted Bellman equation

$$V^{\pi}(x) = \sum_{u \in \mathcal{U}} \pi(x,u) \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^{\pi}(x')\right].$$

To see why, split the infinite-horizon sum into the immediate cost and the discounted future cost:

$$V^{\pi}(x) = \mathbb{E}\left[c_0 + \gamma \sum_{k=1}^{\infty} \gamma^{k-1} c_k \;\middle|\; x_0 = x\right].$$

After the first transition, the expected remaining discounted cost is exactly $V^{\pi}(x_1)$. Conditioning on the action u and on the next state x' gives the Bellman equation above.

This equation is a consistency equation. It says that the value of a state must be equal to the expected immediate cost plus the discounted value of the next state. The system dynamics are encoded in $P_{x,x'}^u$, while the cost function is encoded in C_x^u.

This interpretation is particularly important in learning. The value $V^{\pi}(x)$ is a predictor of the quality of a candidate policy. The Bellman equation decomposes this predictor into two parts:

- the immediate observable cost C_x^u;
- the estimate of future cost $V^{\pi}(x')$.

8.5.2 Computational complexity of policy evaluation

For a fixed policy π, define the expected cost vector

$$C^{\pi}(x) = \sum_{u \in \mathcal{U}} \pi(x,u) C_x^u$$

and the closed-loop transition matrix

$$P_{x,x'}^{\pi} = \sum_{u \in \mathcal{U}} \pi(x,u) P_{x,x'}^u.$$

Then the discounted Bellman equation can be written in vector form as

$$V^{\pi} = C^{\pi} + \gamma P^{\pi} V^{\pi}.$$

Rearranging gives

$$(I - \gamma P^{\pi}) V^{\pi} = C^{\pi},$$

and therefore

$$V^{\pi} = (I - \gamma P^{\pi})^{-1} C^{\pi}.$$

Policy evaluation

For a fixed stationary policy π, the value function is the unique solution of

$$V^\pi = C^\pi + \gamma P^\pi V^\pi.$$

Equivalently,

$$V^\pi = (I - \gamma P^\pi)^{-1} C^\pi.$$

The matrix $I - \gamma P^\pi$ is invertible for $\gamma \in [0, 1)$. Intuitively, the eigenvalues of P^π lie in the unit disk for a stochastic matrix, and multiplying by $\gamma < 1$ moves them strictly inside the unit disk. Thus 1 cannot be an eigenvalue of γP^π, and $I - \gamma P^\pi$ is nonsingular.

If the MDP has N states, the Bellman equation for a fixed policy is a system of N linear equations. Solving it directly can be computationally expensive for large systems. In a dense representation, the cost is on the order of

$$O(N^3 + N^2 M),$$

where the $O(N^3)$ term comes from solving the linear system and the $O(N^2 M)$ term comes from constructing the closed-loop quantities from all state-action transition matrices.

As an alternative, one can iteratively update the Bellman equation. This is possible because the discounted Bellman operator for a fixed policy is contractive when $\gamma < 1$.

8.5.3 Queue example for policy evaluation

For the queue example with the policy described above, the closed-loop matrix is

$$P^\pi = \begin{bmatrix} 1-p & p & 0 & 0 \\ 0 & 1-p & p & 0 \\ 0 & 0 & 1-p & p \\ 1-p & p & 0 & 0 \end{bmatrix}.$$

If the cost is the number of cars in the queue, then

$$C^\pi = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}.$$

The value of the policy is therefore

$$V^\pi = (I - \gamma P^\pi)^{-1} C^\pi.$$

This gives, for each initial queue length, the expected discounted total number of cars in the queue when the policy is used forever.

8.5.4 Bellman principle for infinite-horizon MDPs

The optimal value is defined as the best value achievable among all policies:

$$V^\star(x) = \min_\pi V^\pi(x).$$

Using the Bellman equation for a policy, we can write

$$V^\star(x) = \min_\pi \sum_{u \in \mathcal{U}} \pi(x, u) \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\pi(x') \right].$$

Bellman's optimality principle turns this into a recursive equation for $V^\star$:

$$V^\star(x) = \min_{\pi} \sum_{u \in \mathcal{U}} \pi(x,u) \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\star(x') \right].$$

As before, the minimization is linear in the action probabilities $\pi(x,u)$. Therefore, if the minimum is achieved at a deterministic policy, the equation is equivalent to

$$V^\star(x) = \min_{u \in \mathcal{U}} \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\star(x') \right].$$

This is a system of N nonlinear equations. The nonlinearity does not come from nonlinear dynamics, but from the minimum operator over the finite action set.

Infinite-horizon Bellman optimality equation

The optimal value function of a discounted finite MDP satisfies

$$V^\star(x) = \min_{u \in \mathcal{U}} \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\star(x') \right], \qquad x \in \mathcal{X}.$$

An optimal deterministic policy is obtained by choosing

$$\mu^\star(x) \in \arg\min_{u \in \mathcal{U}} \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\star(x') \right].$$

8.6 Iterative computation of the optimal policy

8.6.1 Greedy policy improvement

The Bellman optimality equation characterizes the optimal value function, but it does not immediately give a practical way to compute the optimal policy. We now discuss two classical iterative methods for doing this: policy iteration and value iteration.

The first ingredient is **policy evaluation**. Given a policy π, we can compute the value function V^π associated with that policy by solving the discounted Bellman equation

$$V^\pi = C^\pi + \gamma P^\pi V^\pi.$$

Equivalently,

$$V^\pi = (I - \gamma P^\pi)^{-1} C^\pi.$$

This step tells us how good the current policy is.

At the same time, once we know the value V^π, we can try to improve the policy. The idea is to ask the following question: if the system is currently in state x, can we reduce the cost by deviating from the current policy for one step, and then following the old policy again from the next state onward?

For a candidate stochastic action distribution $\nu(x,\cdot) \in \Delta(\mathcal{U})$, the one-step lookahead cost is

$$\sum_{u \in \mathcal{U}} \nu(x,u) \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\pi(x') \right].$$

Therefore, the improved policy is obtained by solving

$$\pi'(x,\cdot) \in \arg\min_{\nu(x,\cdot) \in \Delta(\mathcal{U})} \sum_{u \in \mathcal{U}} \nu(x,u) \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\pi(x') \right].$$

Since this minimization is linear over the probability simplex, the optimizer can be chosen as a deterministic policy. Hence we can equivalently write

$$\mu'(x) \in \arg\min_{u \in \mathcal{U}} \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\pi(x') \right].$$

The interpretation is simple: at each state x, we choose the action that gives the smallest sum of immediate expected cost and discounted future value, assuming that after this first action we continue with the old policy π.

8.6.2 Policy improvement theorem

The greedy improvement step always improves the value, unless the current policy is already optimal. More precisely, let π' be the greedy policy obtained from V^π. Then

$$V^{\pi'}(x) \leq V^\pi(x) \qquad \forall x \in \mathcal{X}.$$

Moreover, unless V^π is already the optimal value, there exists at least one state x' such that

$$V^{\pi'}(x') < V^\pi(x').$$

Policy improvement theorem

Let π be a policy and let π' be obtained by the greedy one-step improvement with respect to V^π. Then

$$V^{\pi'}(x) \leq V^\pi(x) \qquad \forall x \in \mathcal{X}.$$

If the greedy improvement is strict for some state, then the new policy is strictly better for at least one state. If no strict improvement is possible, then π is optimal.

Proof. We prove the inequality

$$V^{\pi'}(x) \leq V^\pi(x).$$

By definition of the greedy policy π', for every state x we have

$$V^\pi(x) \geq \sum_{u \in \mathcal{U}} \pi'(x,u) \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\pi(x') \right].$$

The right-hand side is the expected value of

$$c_0 + \gamma V^\pi(x_1)$$

when the first action is chosen according to π'. Hence

$$V^\pi(x) \geq \mathbb{E}_{\pi'} \left[c_0 + \gamma V^\pi(x_1) \mid x_0 = x \right].$$

Now we apply the same inequality again from the next state x_1. This gives

$$V^\pi(x) \geq \mathbb{E}_{\pi'} \left[c_0 + \gamma \left(c_1 + \gamma V^\pi(x_2) \right) \mid x_0 = x \right],$$

or equivalently

$$V^\pi(x) \geq \mathbb{E}_{\pi'} \left[c_0 + \gamma c_1 + \gamma^2 V^\pi(x_2) \;\middle|\; x_0 = x \right].$$

Repeating the same argument recursively, we obtain

$$V^\pi(x) \geq \mathbb{E}_{\pi'} \left[c_0 + \gamma c_1 + \gamma^2 c_2 + \gamma^3 c_3 + \cdots \;\middle|\; x_0 = x \right].$$

The right-hand side is precisely the value of the improved policy:

$$\mathbb{E}_{\pi'}\left[c_0 + \gamma c_1 + \gamma^2 c_2 + \gamma^3 c_3 + \cdots \;\middle|\; x_0 = x\right] = V^{\pi'}(x).$$

Therefore

$$V^{\pi}(x) \geq V^{\pi'}(x) \qquad \forall x \in \mathcal{X}.$$

The strict inequality follows if the greedy step gives a strict improvement for at least one state, while being non-worse for all the others.

As a consequence, after a finite number of strict policy improvements, we reach an optimal policy. The reason is that there are only finitely many deterministic policies in a finite MDP, and policy improvement never returns to a strictly worse policy.

8.6.3 Policy iteration algorithm

Policy iteration alternates between the two operations described above:

- evaluate the current policy;
- greedily improve the current policy using the value function just computed.

Policy iteration algorithm

Initialize a policy guess

$$\pi \leftarrow \pi_0.$$

For each iteration:

1. **Policy evaluation.** Compute the value V associated with the current policy π:

$$V \leftarrow (I - \gamma P^{\pi})^{-1} C^{\pi}.$$

2. **Greedy policy update.** For every state x, update the policy by solving

$$\pi_{\text{new}}(x, \cdot) \in \underset{\nu(x,\cdot)\in\Delta(\mathcal{U})}{\arg\min} \sum_{u\in\mathcal{U}} \nu(x,u) \left[C_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u V(x') \right].$$

   Equivalently, since the minimizer can be chosen deterministic,

$$\mu_{\text{new}}(x) \in \underset{u\in\mathcal{U}}{\arg\min} \left[C_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u V(x') \right].$$

3. Set

$$\pi \leftarrow \pi_{\text{new}}.$$

Stop when the policy no longer changes.

The computational cost is split between the two steps. Policy evaluation is the expensive part, because it requires solving a system of N linear equations:

$$V = (I - \gamma P^{\pi})^{-1} C^{\pi}.$$

For a dense MDP, this has complexity approximately

$$O(N^3 + N^2 M).$$

The term $O(N^3)$ comes from solving the linear system, while the term $O(N^2 M)$ comes from constructing the closed-loop transition matrix and expected cost.

Policy improvement is usually simpler. For each state x, one compares the M possible actions and selects the one with smallest one-step lookahead cost. Thus the improvement step is essentially a minimization over M alternatives, repeated N times.

The policy improvement theorem guarantees that every nontrivial iteration improves the policy. Since the number of deterministic policies is finite, policy iteration terminates after a finite number of improvements.

8.6.4 Value iteration

Policy iteration evaluates each intermediate policy exactly. An alternative is to look directly for a fixed point of the Bellman optimality equation.

Recall that the optimal value satisfies

$$V^\star(x) = \min_{\pi(x,\cdot)} \sum_{u\in\mathcal{U}} \pi(x,u) \left[C_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u V^\star(x') \right].$$

Equivalently, when the optimum is attained by a deterministic policy,

$$V^\star(x) = \min_{u\in\mathcal{U}} \left[C_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u V^\star(x') \right].$$

A natural way to compute this fixed point is to start from an initial guess $V^{(0)}$ and iterate the Bellman optimality equation:

$$V^{(t+1)}(x) = \min_{\pi(x,\cdot)} \sum_{u\in\mathcal{U}} \pi(x,u) \left[C_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u V^{(t)}(x') \right],$$

or, equivalently,

$$V^{(t+1)}(x) = \min_{u\in\mathcal{U}} \left[C_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u V^{(t)}(x') \right].$$

Here t is not the time index of the MDP. It is the iteration index of the algorithm.

To state the convergence property compactly, define the Bellman optimality operator T as

$$(TV)(x) = \min_{u\in\mathcal{U}} \left[C_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u V(x') \right].$$

Then value iteration is simply

$$V^{(t+1)} = TV^{(t)}.$$

The key convergence result is that the Bellman optimality operator is a contraction in the infinity norm. If V and W are two value functions, then

$$\|TV - TW\|_\infty \le \gamma \|V - W\|_\infty,$$

where

$$\|V\|_\infty := \max_{x\in\mathcal{X}} |V(x)|.$$

Since $0 \le \gamma < 1$, the operator T is contractive. Therefore, value iteration converges to the unique fixed point $V^\star$. In particular,

$$\left\|V^{(t)} - V^\star\right\|_\infty \le \gamma^t \left\|V^{(0)} - V^\star\right\|_\infty.$$

Thus the convergence rate is governed by γ. When γ is close to one, future costs are important, but the convergence of value iteration becomes slower.

8.6.5 Value iteration algorithm

Value iteration algorithm

Initialize a value guess

$$V \leftarrow V_0.$$

For each iteration, apply the Bellman iteration

$$V(x) \leftarrow \min_{\pi(x,\cdot)} \sum_{u \in \mathcal{U}} \pi(x,u) \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V(x') \right].$$

Equivalently,

$$V(x) \leftarrow \min_{u \in \mathcal{U}} \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V(x') \right], \qquad x \in \mathcal{X}.$$

At convergence, extract the optimal greedy policy:

$$\pi^\star(x,\cdot) \in \arg\min_{\nu(x,\cdot) \in \Delta(\mathcal{U})} \sum_{u \in \mathcal{U}} \nu(x,u) \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\star(x') \right].$$

Equivalently, one can extract a deterministic optimal policy by choosing

$$\mu^\star(x) \in \arg\min_{u \in \mathcal{U}} \left[C_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\star(x') \right].$$

The computational cost of value iteration is different from policy iteration. Each Bellman iteration is usually cheaper than solving the full policy evaluation linear system. For a dense finite MDP, one Bellman iteration has complexity approximately

$$O(N^2 M).$$

However, the convergence is asymptotic rather than finite, and the rate is controlled by γ. Thus policy iteration usually performs fewer but more expensive iterations, while value iteration performs cheaper iterations but may need many of them.

9 Monte Carlo Learning

9.1 From model-based dynamic programming to learning

All the algorithmic steps introduced for MDPs relied on model information, in particular on the transition probabilities

$$P^u_{x,x'} = \mathbb{P}[x_{k+1} = x' \mid x_k = x,\ u_k = u].$$

For instance, in policy iteration, the policy evaluation step required the closed-loop transition matrix

$$P^\pi_{x,x'} = \sum_{u \in \mathcal{U}} \pi(x,u) P^u_{x,x'},$$

and the value was computed as

$$V^\pi = (I - \gamma P^\pi)^{-1} R^\pi.$$

Here we have switched notation from costs to rewards. Therefore R and r denote rewards, and the objective is now to maximize value functions instead of minimizing cost functions.

The policy improvement step also required the transition probabilities. Indeed, once V^π is known, a greedy improvement is obtained by solving

$$\pi_{\text{new}}(x,\cdot) \in_{\nu(x,\cdot)\in\Delta(\mathcal{U})} \sum_{u\in\mathcal{U}} \nu(x,u) \left[R^u_x + \gamma \sum_{x'\in\mathcal{X}} P^u_{x,x'} V^\pi(x') \right].$$

Equivalently, when a deterministic maximizer is selected,

$$\mu_{\text{new}}(x) \in_{u\in\mathcal{U}} \left[R^u_x + \gamma \sum_{x'\in\mathcal{X}} P^u_{x,x'} V^\pi(x') \right].$$

The same issue appears in value iteration. The Bellman optimality update is

$$V(x) \leftarrow \max_{\pi(x,\cdot)} \sum_{u\in\mathcal{U}} \pi(x,u) \left[R^u_x + \gamma \sum_{x'\in\mathcal{X}} P^u_{x,x'} V(x') \right],$$

or, equivalently,

$$V(x) \leftarrow \max_{u\in\mathcal{U}} \left[R^u_x + \gamma \sum_{x'\in\mathcal{X}} P^u_{x,x'} V(x') \right].$$

Thus, both policy iteration and value iteration are model-based algorithms: they require knowing how likely every next state x' is after applying every action u in every state x.

The central question in learning is therefore the following: can we find a good, or even optimal, policy without explicitly knowing the transition probabilities?

There are two different settings. In the first setting, we have access to collected data, typically in the form of repeated episodes,

$$(x_0,u_0,r_0),\ (x_1,u_1,r_1),\ \ldots,\ (x_T,u_T,r_T).$$

These episodes may come from repeated simulations of the control task, or from repeated experiments in a controlled environment. In the second setting, learning happens online during the control task itself, with no prior training. This is much harder, because the controller must learn and control at the same time. This is the classical dual-control difficulty: the controller must collect information while also trying to achieve good performance.

In this chapter we focus on the first viewpoint: learning from episodes. The goal is to replace explicit model-based policy evaluation with empirical estimates obtained from data.

9.2 The Q-function

A useful reformulation is obtained by introducing the action-value function, or Q-function. For a fixed policy π, the Q-function is defined as the expected discounted future reward obtained by:

- starting in state x;
- taking action u immediately;
- applying policy π at all subsequent times.

Therefore,

$$Q^\pi(x,u) = R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\pi(x').$$

The difference between V^π and Q^π is the following. The value function $V^\pi(x)$ is defined only over states: it measures the expected return when one starts from x and then follows policy π. The Q-function $Q^\pi(x,u)$, instead, is defined over state-action pairs: it measures the expected return when the first action is fixed to be u, and the policy π is followed only afterward.

Thus, if the MDP has N states and M actions, then

$$V^\pi \quad \text{has } N \text{ entries}, \qquad Q^\pi \quad \text{has } NM \text{ entries}.$$

The two objects are directly related. Since under policy π, action u is selected with probability $\pi(x,u)$, we have

$$V^\pi(x) = \sum_{u \in \mathcal{U}} \pi(x,u) Q^\pi(x,u).$$

In words, the value of a state is the average of the Q-values of all possible actions, weighted by the probabilities with which the policy selects those actions.

9.3 Bellman equations in Q-form

The standard Bellman equation for the value function is

$$V^\pi(x) = \sum_{u \in \mathcal{U}} \pi(x,u) \left[R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\pi(x') \right].$$

Using the relation

$$V^\pi(x') = \sum_{u' \in \mathcal{U}} \pi(x',u') Q^\pi(x',u'),$$

we can write the Bellman equation directly in terms of Q^π:

$$Q^\pi(x,u) = R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \sum_{u' \in \mathcal{U}} \pi(x',u') Q^\pi(x',u').$$

This is again a consistency equation. The Q-value of (x,u) is equal to the immediate expected reward R_x^u, plus the discounted expected Q-value of the next state-action pair. The next state x' is generated by the transition probabilities, while the next action u' is generated by the policy π.

The model information required is essentially the same as before: one still needs $P_{x,x'}^u$ and R_x^u. However, the advantage of the Q-function is that policy improvement becomes much simpler. If Q^π is known, improving the policy does not require one-step model predictions anymore. It only requires comparing the values $Q^\pi(x,u)$ over the actions available at the current state.

9.4 Bellman optimality in Q-form

In reward maximization, the optimal value function satisfies

$$V^\star(x) = \max_{u\in\mathcal{U}} \left[R_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u V^\star(x') \right].$$

The corresponding optimal Q-function satisfies

$$Q^\star(x,u) = R_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u \left(\max_{u'\in\mathcal{U}} Q^\star(x',u') \right).$$

Indeed, after taking action u in state x, the next state x' is random. Once the system has reached x', optimal behavior means choosing an action u' that maximizes the optimal Q-value at x'. Therefore,

$$V^\star(x') = \max_{u'\in\mathcal{U}} Q^\star(x',u').$$

Equivalently,

$$Q^\star(x,u) = R_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u V^\star(x').$$

The important structural change is that the maximization over the next action has moved inside the Q-form Bellman equation. The expectation is still with respect to the next state, but the optimal action at the next state is chosen by maximizing $Q^\star(x',u')$. Once $Q^\star$ is known, the optimal policy is immediately obtained as

$$\mu^\star(x) \in_{u\in\mathcal{U}} Q^\star(x,u).$$

Thus the Q-function implicitly defines the optimal strategy.

9.5 Policy iteration with Q-functions

Let us now rewrite policy iteration using the Q-function. In the value-function formulation, policy iteration had two steps:

1. compute V^π for the current policy π;
2. improve the policy greedily by using the model and the value V^π.

The improvement step required the expression

$$R_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u V^\pi(x').$$

Therefore, model information entered both the policy evaluation step and the policy improvement step.

In the Q-function formulation, the first step becomes the computation of Q^π, which is defined by the Bellman equation

$$Q^\pi(x,u) = R_x^u + \gamma \sum_{x'\in\mathcal{X}} P_{x,x'}^u \sum_{u'\in\mathcal{U}} \pi(x',u') Q^\pi(x',u').$$

Once Q^π is known, the greedy policy update is simply

$$\pi_{\text{new}}(x,\cdot) \in_{\nu(x,\cdot)\in\Delta(\mathcal{U})} \sum_{u\in\mathcal{U}} \nu(x,u) Q^\pi(x,u).$$

Equivalently, for a deterministic policy,

$$\mu_{\text{new}}(x) \in_{u\in\mathcal{U}} Q^\pi(x,u).$$

This is the key advantage of Q-functions in a model-free setting. The policy evaluation step is still difficult, because computing Q^π exactly requires model information and involves NM unknowns. However, the policy improvement step becomes trivial and requires no model information: for each state, one simply chooses the action with the largest estimated Q-value.

Therefore, if we can learn or estimate Q^π from data, we can improve the policy without ever explicitly estimating the transition probabilities.

9.6 Experimental policy evaluation

The idea of Monte Carlo policy evaluation is to delegate the policy evaluation step to experiments or simulations. Suppose that the system is controlled by a fixed policy π, and that we collect one long episode

$$(x_0, u_0, r_0),\ (x_1, u_1, r_1),\ \ldots,\ (x_T, u_T, r_T).$$

For each time k, define the empirical discounted return

$$g_k = \sum_{i=k}^{T} \gamma^{i-k} r_i.$$

This is the reward accumulated from time k until the end of the episode, with discounting. It is a sample realization of the return obtained after observing the state-action pair (x_k, u_k).

Therefore, a first empirical estimate is

$$Q^\pi(x_k, u_k) \approx g_k.$$

If the same state-action pair (x, u) is visited several times in the episode, one averages all the corresponding returns:

$$Q^\pi(x, u) = \frac{\sum_{t=0}^{T} \mathbf{1}[x_t = x,\ u_t = u]\, g_t}{\sum_{t=0}^{T} \mathbf{1}[x_t = x,\ u_t = u]}.$$

This is the Monte Carlo estimate of the Q-function. It says: whenever the episode visits the state-action pair (x, u), look at the discounted return observed from that point onward; then average all such observed returns.

This procedure relies on an ergodicity assumption. In order to estimate $Q^\pi(x, u)$, the state-action pair (x, u) must be visited sufficiently often. If a pair is never visited, then the data contain no information about its return.

There are a few important observations.

First, the policy cannot be evaluated until the episode has terminated, because g_k depends on all future rewards

$$r_k, r_{k+1}, \ldots, r_T.$$

Thus Monte Carlo policy evaluation is naturally episodic: it computes returns after complete trajectories have been observed.

Second, the empirical estimate of Q^π does not exactly satisfy the Bellman equation for a finite amount of data. The Bellman equation is an expectation identity, whereas the Monte Carlo estimate is based on sample averages. Only in the limit of infinitely many representative samples do these empirical averages converge to the corresponding expected values.

Third, this method does not exploit the Markov property to make the computation of Q^π more efficient. Each return g_k is computed using the full future tail of the episode, rather than recursively bootstrapping from the value of the next state-action pair. This is one of the main differences between Monte Carlo learning and temporal-difference methods.

9.7 Exploration and exploitation

If we update the policy greedily too early, we may get stuck in a suboptimal behavior. The reason is simple: at the beginning of learning, the estimate of Q^π is rough. If the policy becomes greedy with respect to this inaccurate estimate, it may stop trying actions that currently look bad only because they have not been explored enough.

This is the exploration-exploitation tradeoff. Exploitation means choosing the action that currently appears best. Exploration means deliberately trying other actions in order to collect more information.

A common solution is ε-greedy policy improvement:

$$\pi(x,u) = \begin{cases} 1-\varepsilon+\dfrac{\varepsilon}{|\mathcal{U}|} & \text{if } u \in_{\bar{u}\in\mathcal{U}} Q(x,\bar{u}), \\ \dfrac{\varepsilon}{|\mathcal{U}|} & \text{otherwise.} \end{cases}$$

Equivalently, one can describe the action selection rule as

$$u \leftarrow \begin{cases} \bar{u}\in\mathcal{U} Q(x,\bar{u}) & \text{with probability } 1-\varepsilon, \\ \text{Uniform}(\mathcal{U}) & \text{with probability } \varepsilon. \end{cases}$$

A large value of ε produces more exploration, while a small value of ε produces more exploitation.

Another common policy improvement rule is the Boltzmann, or softmax, policy:

$$\pi(x,u) = \frac{e^{\beta Q(x,u)}}{\sum_{\bar{u}\in\mathcal{U}} e^{\beta Q(x,\bar{u})}}.$$

Here β is an inverse-temperature parameter. If β is small, the probabilities are close to uniform and the policy explores more. If β is large, the action with the largest Q-value receives most of the probability mass, and the policy becomes nearly greedy.

As learning progresses from episode to episode, the amount of exploration is usually reduced. For instance, one may take

$$\varepsilon \to 0 \qquad \text{or} \qquad \beta \to \infty.$$

The intuition is that early episodes should explore the state-action space broadly, whereas later episodes should exploit the information collected so far. If this is done properly, then with probability one each relevant state-action pair is visited infinitely often, while the policy eventually converges to a greedy policy.

9.8 Scalability and parametrization of Q

In principle, for a finite MDP, one could store the Q-function in a table with one entry for each state-action pair. This is the tabular representation:

$$Q = \{Q(x,u) : x \in \mathcal{X},\ u \in \mathcal{U}\}.$$

However, this quickly becomes infeasible when the state and action spaces are large. If the state has dimension n, the number of possible states typically grows exponentially with n. Multiple-input systems increase the number of actions, and continuous state or action spaces make exact tabular storage impossible.

For this reason, the Q-function is very often parametrized by a lower-dimensional vector of parameters:

$$Q(x,u) = Q_\theta(x,u), \qquad \theta \in \mathbb{R}^d.$$

The idea is that d should be much smaller than NM, so that the Q-function can be stored and learned more efficiently.

This has several advantages:

- the memory requirement is reduced from NM entries to d parameters;
- the problem size is apparently independent of the number of states N, which is essential for continuous state spaces;
- a good parametrization can generalize from observed state-action pairs to unobserved ones, through interpolation or extrapolation;
- prior knowledge about the problem can be encoded in the choice of features.

There are also important disadvantages:

- the number of degrees of freedom is reduced;
- the true solution of the Bellman equation for a given policy may not belong to the chosen function class
$$\mathcal{Q} = \{Q_\theta : \theta \in \mathbb{R}^d\};$$
- convergence of policy iteration and optimality of the limiting policy may be affected;
- computing the parameter θ that best approximates the data collected from an episode may itself be computationally expensive.

Thus parametrization is necessary for scalability, but it also introduces approximation error.

9.9 Linear parametrization

A simple and important case is linear parametrization. More advanced parametrizations, such as neural networks, are often used in practice, but the linear case already contains the main idea.

We choose basis functions
$$\phi_\ell(x, u), \qquad \ell = 1, \dots, d,$$
and write
$$Q_\theta(x, u) = \sum_{\ell=1}^{d} \phi_\ell(x, u)\theta_\ell = \phi(x, u)^\top \theta,$$
where
$$\phi(x, u) = \begin{bmatrix} \phi_1(x, u) \\ \vdots \\ \phi_d(x, u) \end{bmatrix}.$$
The functions $\phi_\ell(x, u)$ may be simple polynomial features such as
$$x, \quad u, \quad x^2, \quad u^2, \quad xu, \dots,$$
quadratic forms such as
$$x^\top A_\ell x,$$
or problem-specific features. For example, in a game-like environment, one might use quantities such as total height, number of holes, or other hand-crafted descriptors.

The key point is that the Q-function is no longer represented by one independent value for each state-action pair. Instead, all Q-values are tied together through the common parameter vector θ.

9.10 Projected Bellman equation

With parametrization, we cannot generally solve the Bellman equation exactly. For a fixed policy π, the Q-Bellman equation is

$$Q(x,u) = R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \sum_{u' \in \mathcal{U}} \pi(x',u') Q(x',u').$$

Define the Bellman operator

$$\mathcal{B}(Q)(x,u) := R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \sum_{u' \in \mathcal{U}} \pi(x',u') Q(x',u').$$

The exact Bellman equation is

$$Q = \mathcal{B}(Q).$$

However, the function $\mathcal{B}(Q_\theta)$ will in general not belong to the parametrized class

$$\mathcal{Q} = \{Q_\theta : \theta \in \mathbb{R}^d\}.$$

Therefore, after applying the Bellman operator, we project the result back onto the function class.

Let ρ be a diagonal $NM \times NM$ weight matrix. It defines the weighted norm

$$\|Q\|_\rho^2 = Q^\top \rho Q.$$

The best approximation operator is

$$\Pi(Q) = \arg\min_{Q_\theta \in \mathcal{Q}} \|Q_\theta - Q\|_\rho^2 .$$

The projected Bellman equation is then

$$Q_\theta = \Pi(\mathcal{B}(Q_\theta)).$$

This equation says that Q_θ should be the best approximation, within the chosen parametrized class, of its Bellman update.

9.11 Matrix form of the projected Bellman equation

To write the projected Bellman equation in matrix form, enumerate all state-action pairs and stack the corresponding values into a vector

$$Q \in \mathbb{R}^{NM}.$$

Similarly, define

$$R \in \mathbb{R}^{NM}$$

as the vector collecting the expected immediate rewards R_x^u for all state-action pairs.

Define the state-action transition matrix $\mathsf{T}^\pi \in \mathbb{R}^{NM \times NM}$ by

$$\mathsf{T}^\pi_{(x,u),(x',u')} = P_{x,x'}^u \pi(x',u').$$

This is the transition probability between state-action pairs: starting from (x,u), the system moves to x' with probability $P_{x,x'}^u$, and then policy π selects u' with probability $\pi(x',u')$.

Then the Bellman operator can be written compactly as

$$\mathcal{B}(Q) = R + \gamma \mathsf{T}^\pi Q.$$

For the linear parametrization, collect the feature vectors in the matrix

$$\Phi = \begin{bmatrix} \phi(x_1, u_1)^\top \\ \phi(x_1, u_2)^\top \\ \vdots \\ \phi(x_N, u_M)^\top \end{bmatrix} \in \mathbb{R}^{NM \times d}.$$

Then

$$Q_\theta = \Phi\theta.$$

The projected Bellman equation becomes

$$\Phi\theta = \Pi\left(R + \gamma \mathsf{T}^\pi \Phi\theta\right).$$

The solution is characterized by the normal equations

$$\Phi^\top \rho \Phi\, \theta = \gamma \Phi^\top \rho \mathsf{T}^\pi \Phi\, \theta + \Phi^\top \rho R.$$

Equivalently,

$$\left(\Phi^\top \rho \Phi - \gamma \Phi^\top \rho \mathsf{T}^\pi \Phi\right) \theta = \Phi^\top \rho R.$$

Thus, instead of solving a system with NM unknown Q-values, we solve a system of d linear equations in the parameter vector θ.

Derivation. Since $Q_\theta = \Phi\theta$, the Bellman update is

$$\mathcal{B}(Q_\theta) = R + \gamma \mathsf{T}^\pi \Phi\theta.$$

The projection step searches for a parameter $\hat{\theta}$ minimizing

$$\left\| \Phi\hat{\theta} - R - \gamma \mathsf{T}^\pi \Phi\theta \right\|_\rho^2.$$

For a weighted least-squares problem, the normal equations are

$$\Phi^\top \rho \Phi\, \hat{\theta} = \Phi^\top \rho \left(R + \gamma \mathsf{T}^\pi \Phi\theta\right).$$

At a fixed point of the projected Bellman equation we have $\hat{\theta} = \theta$, and therefore

$$\Phi^\top \rho \Phi\, \theta = \gamma \Phi^\top \rho \mathsf{T}^\pi \Phi\, \theta + \Phi^\top \rho R.$$

This gives the equation above.

9.12 Model-free computation of the projected Bellman equation

The equation

$$\left(\Phi^\top \rho \Phi - \gamma \Phi^\top \rho \mathsf{T}^\pi \Phi\right) \theta = \Phi^\top \rho R$$

still appears to contain model information through ρ, T^π, and R. The key observation is that these matrices can be estimated from data.

Suppose we observe an episode generated under policy π:

$$(x_0, u_0, r_0),\ (x_1, u_1, r_1),\ \ldots,\ (x_T, u_T, r_T).$$

For each sample, define

$$\phi_k := \phi(x_k, u_k), \qquad \phi_{k+1} := \phi(x_{k+1}, u_{k+1}).$$

Then for every transition in the episode we construct the three quantities

$$\phi(x_k, u_k)\phi(x_k, u_k)^\top,$$

$$\phi(x_k, u_k)\phi(x_{k+1}, u_{k+1})^\top,$$

and

$$\phi(x_k, u_k) r_k.$$

Their empirical averages estimate the corresponding matrices:

$$\widehat{M}_0 = \frac{1}{T}\sum_{k=0}^{T-1} \phi_k \phi_k^\top \approx \Phi^\top \rho \Phi,$$

$$\widehat{M}_1 = \frac{1}{T}\sum_{k=0}^{T-1} \phi_k \phi_{k+1}^\top \approx \Phi^\top \rho \mathsf{T}^\pi \Phi,$$

and

$$\widehat{b} = \frac{1}{T}\sum_{k=0}^{T-1} \phi_k r_k \approx \Phi^\top \rho R.$$

Therefore, the projected Bellman equation can be approximated from data by

$$\left(\widehat{M}_0 - \gamma \widehat{M}_1\right)\theta = \widehat{b}.$$

Equivalently,

$$\left(\frac{1}{T}\sum_{k=0}^{T-1} \phi_k \phi_k^\top - \gamma \frac{1}{T}\sum_{k=0}^{T-1} \phi_k \phi_{k+1}^\top\right)\theta = \frac{1}{T}\sum_{k=0}^{T-1} \phi_k r_k.$$

This is a model-free policy evaluation method for linearly parametrized Q-functions. The transition probabilities are not explicitly known, but their effect is captured through observed successive pairs

$$(x_k, u_k) \quad \text{and} \quad (x_{k+1}, u_{k+1}).$$

The matrix ρ is also not chosen explicitly in this empirical version: it is encoded by the frequency with which the different state-action pairs appear in the dataset. This is exactly the (i, j)-entry of $\Phi^\top \rho \Phi$.

The same reasoning applies to the other two terms. The average of

$$\phi(x_k, u_k)\phi(x_{k+1}, u_{k+1})^\top$$

estimates the transition-weighted matrix

$$\Phi^\top \rho \mathsf{T}^\pi \Phi,$$

while the average of

$$\phi(x_k, u_k) r_k$$

estimates

$$\Phi^\top \rho R.$$

Thus, the matrices appearing in the projected Bellman equation can be replaced by empirical averages computed directly from episodes.

9.13 Empirical interpretation of the weighted matrices

Let us make the connection between empirical averages and the weighted matrices even more explicit. Enumerate the state-action pairs by an index

$$s = 1, \dots, S, \qquad S = NM.$$

For each state-action pair s, let

$$\phi(s) = \begin{bmatrix} \phi_1(s) \\ \vdots \\ \phi_d(s) \end{bmatrix} \in \mathbb{R}^d$$

be the feature vector. The feature matrix is then

$$\Phi = \begin{bmatrix} \phi(s_1)^\top \\ \phi(s_2)^\top \\ \vdots \\ \phi(s_S)^\top \end{bmatrix} \in \mathbb{R}^{S \times d}.$$

Equivalently, if we denote by $\phi_j \in \mathbb{R}^S$ the j-th column of Φ, then

$$\Phi = \begin{bmatrix} \phi_1 & \cdots & \phi_d \end{bmatrix}.$$

Consider first the matrix

$$\Phi^\top \rho \Phi.$$

Since ρ is diagonal, we can write

$$\Phi^\top \rho \Phi = \begin{bmatrix} \phi_1^\top \\ \vdots \\ \phi_d^\top \end{bmatrix} \begin{bmatrix} \rho(1) & & \\ & \ddots & \\ & & \rho(S) \end{bmatrix} \begin{bmatrix} \phi_1 & \cdots & \phi_d \end{bmatrix}.$$

A generic element in position (i, j) is therefore

$$\left[\Phi^\top \rho \Phi\right]_{ij} = \phi_i^\top \begin{bmatrix} \rho(1) & & \\ & \ddots & \\ & & \rho(S) \end{bmatrix} \phi_j = \sum_{s=1}^{S} \rho(s) \phi_i(s) \phi_j(s).$$

This is exactly the same expression obtained from the empirical average of the sample matrices

$$\phi(s)\phi(s)^\top.$$

Indeed, when many samples are collected, the frequency with which the state-action pair s appears in the dataset plays the role of the weight $\rho(s)$.

A similar reasoning applies to the other two matrices. The empirical average of

$$\phi(x_k, u_k)\phi(x_{k+1}, u_{k+1})^\top$$

estimates

$$\Phi^\top \rho \mathsf{T}^\pi \Phi,$$

because it correlates the feature vector of the current state-action pair with the feature vector of the next state-action pair. Similarly, the empirical average of

$$\phi(x_k, u_k) r_k$$

estimates

$$\Phi^\top \rho R,$$

because it correlates the current feature vector with the observed immediate reward.

Thus the matrices appearing in the projected Bellman equation can be constructed directly from an episode, without explicitly estimating either the transition probabilities or the full reward model.

9.14 Monte Carlo learning with linear approximation

We can now combine the previous ideas into a complete learning procedure. The method alternates between two steps:

1. policy evaluation from data;
2. greedy policy improvement with exploration.

9.14.1 Policy evaluation

Fix a policy π and run an experiment, or one simulated episode, under this policy. This generates a time series

$$x_k,\ u_k,\ r_k, \qquad k = 1, \dots, K.$$

From this episode, we build the empirical estimates

$$\Phi^\top \rho \Phi \approx \frac{1}{K} \sum_{k=0}^{K-1} \phi(x_k, u_k)\phi(x_k, u_k)^\top,$$

$$\Phi^\top \rho \mathsf{T}^\pi \Phi \approx \frac{1}{K} \sum_{k=0}^{K-1} \phi(x_k, u_k)\phi(x_{k+1}, u_{k+1})^\top,$$

and

$$\Phi^\top \rho R \approx \frac{1}{K} \sum_{k=0}^{K-1} \phi(x_k, u_k) r_k.$$

These empirical averages are then inserted into the projected Bellman equation

$$\left(\Phi^\top \rho \Phi - \gamma \Phi^\top \rho \mathsf{T}^\pi \Phi\right) \theta = \Phi^\top \rho R.$$

Solving this linear system determines the parameter vector θ, and therefore the approximate Q-function

$$Q_\theta(x, u) = \phi(x, u)^\top \theta.$$

In empirical form, this means solving

$$\left(\frac{1}{K} \sum_{k=0}^{K-1} \phi(x_k, u_k)\phi(x_k, u_k)^\top - \gamma \frac{1}{K} \sum_{k=0}^{K-1} \phi(x_k, u_k)\phi(x_{k+1}, u_{k+1})^\top \right) \theta = \frac{1}{K} \sum_{k=0}^{K-1} \phi(x_k, u_k) r_k.$$

The result is an approximate evaluation of the current policy π, obtained without knowing the transition probabilities $P^u_{x,x'}$.

9.14.2 Greedy policy improvement

Once the approximate Q-function has been computed, the policy can be improved greedily. In the purely greedy case, we would choose

$$\mu_{\text{new}}(x) \in_{u \in \mathcal{U}} Q_\theta(x, u).$$

However, as discussed before, a purely greedy update may stop exploration too early. Therefore, in learning we typically keep some exploration noise. For example, with an ε-greedy policy we use

$$u \leftarrow \begin{cases} \bar{u} \in \mathcal{U} Q_\theta(x, \bar{u}) & \text{with probability } 1 - \varepsilon, \\ \text{Uniform}(\mathcal{U}) & \text{with probability } \varepsilon. \end{cases}$$

Thus the new policy mostly exploits the current approximation Q_θ, but still explores random actions with probability ε.

The complete scheme is therefore:

1. fix the current policy π;
2. collect one or more episodes under π;
3. use the data to estimate the matrices in the projected Bellman equation;
4. solve the linear system to compute θ;
5. define $Q_\theta(x, u) = \phi(x, u)^\top \theta$;
6. improve the policy greedily with respect to Q_θ, while keeping enough exploration.

9.15 Key points

The Q-function representation is useful because it separates the two parts of policy iteration in a way that is well suited to learning:

- policy improvement becomes trivial, since one only needs to compare the values $Q(x, u)$ for the available actions;
- policy evaluation can be delegated to an experiment or simulation.

By alternating between policy improvement and episodic experiments, one can progressively compute better strategies without explicitly identifying the transition probabilities.

However, sufficient exploration must be guaranteed. If the policy becomes too greedy too soon, some state-action pairs may not be visited often enough, and the learned Q-function may be poor in precisely the regions where information is missing.

Q-function approximation is essential for scalability. A tabular Q-function requires one value for every state-action pair, which becomes infeasible in large or continuous problems. A parametrized approximation

$$Q_\theta(x, u)$$

reduces the memory requirement and can generalize across different state-action pairs, but it also introduces approximation error.

Many variations of this basic idea exist. In the scientific literature, the terminology can vary: methods of this type are often related to Monte Carlo policy evaluation, generalized policy iteration, and approximate policy iteration. The common idea is always the same: use data to evaluate a policy, then use the resulting value or Q-function estimate to improve the policy.

10 Reinforcement Learning

10.1 From model information to learning from data

The dynamic programming algorithms discussed so far rely on model information. More precisely, they require either the transition probabilities of the MDP, or a sufficiently rich trajectory from which the relevant quantities can be estimated.

The key question is therefore:

Can we find the optimal policy without model information?

There are two different settings.

First, one may work with collected data, typically in the form of repeated episodes:

$$(x_0, u_0, r_0),\ (x_1, u_1, r_1),\ \ldots,\ (x_T, u_T, r_T).$$

These episodes may come from repeated executions of the control task in a numerical simulator, or from repeated experiments in a controlled environment. In this case, learning is performed offline or between episodes: the system is allowed to generate data, and the policy is improved using this data.

Second, one may try to learn online during the control task, with no prior training. This is substantially harder, because the controller must learn and control at the same time:

$$\text{online learning} \quad \Longrightarrow \quad \text{adaptive control and dual control.}$$

This is challenging because the input has two roles: it must achieve good performance, but it must also excite the system enough to reveal useful information. For this reason, online model-free optimal control is a difficult dual-control problem and, in its most general form, comes with few guarantees.

In the following, we focus on learning methods that use data to estimate value or Q-functions. The central idea is to replace explicit model-based policy evaluation by empirical or recursive updates based on observed transitions.

10.2 Monte Carlo policy evaluation

Recall the difficult step in policy iteration. For a fixed policy π, we would like to evaluate the Q-function

$$Q^\pi(x,u) = R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u V^\pi(x'),$$

where

$$V^\pi(x) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k r_k \;\middle|\; x_0 = x \right].$$

If the transition probabilities $P_{x,x'}^u$ are known, this can be done by solving the Bellman equation. In a model-free setting, instead, we try to estimate the same quantities from observed episodes.

In Monte Carlo policy evaluation, one lets the system run under a fixed policy π and records an episode

$$(x_0, u_0, r_0),\ (x_1, u_1, r_1),\ \ldots,\ (x_T, u_T, r_T).$$

For each time k, define the empirical return

$$g_k = \sum_{i=k}^{T} \gamma^{i-k} r_i.$$

This is the discounted reward accumulated from time k until the end of the episode.

The quantity g_k can be interpreted as one realization of the return associated with the state-action pair (x_k, u_k). Therefore, a basic Monte Carlo estimate is

$$Q^\pi(x_k, u_k) \approx g_k.$$

If the same state-action pair (x, u) is visited several times, then the natural estimate is the average of all returns observed after visits to that pair:

$$Q^\pi(x, u) = \frac{\sum_{t=0}^{T} \mathbf{1}[x_t = x,\ u_t = u]\, g_t}{\sum_{t=0}^{T} \mathbf{1}[x_t = x,\ u_t = u]}.$$

Thus, Monte Carlo policy evaluation replaces the expectation defining Q^π by an empirical average over complete episodes.

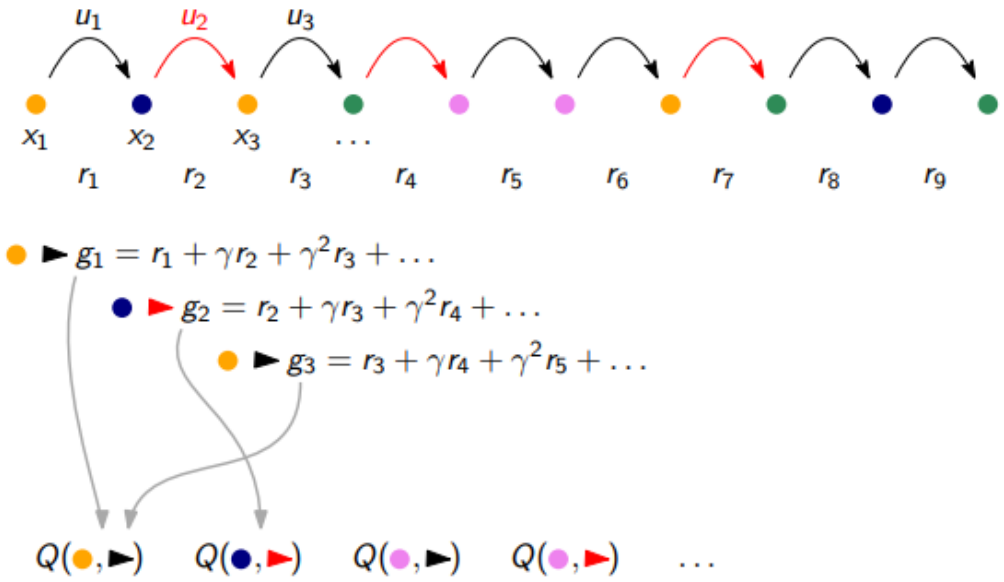

Figure 14: Monte Carlo policy evaluation. Along one episode, each visited state-action pair receives as target the discounted return accumulated from that time onward.

The important point is that this method does not explicitly use the Bellman equation. It simply waits until the future rewards have been observed and then computes the corresponding return. As a consequence, the policy cannot be evaluated until the episode is over, because

$$g_k = r_k + \gamma r_{k+1} + \gamma^2 r_{k+2} + \cdots$$

depends on rewards that occur after time k.

Moreover, for a finite amount of data, the resulting estimate of Q^π does not exactly satisfy the Bellman equation. The Bellman equation is an identity in expectation, whereas Monte Carlo learning is based on sample averages. Only in the limit of infinitely many representative samples does the Monte Carlo estimate converge to the correct expected return.

This also shows a limitation of Monte Carlo learning: it does not exploit the Markov property to make the computation more efficient. Instead of using a one-step recursive consistency relation, it uses the full future tail of the episode.

10.3 Temporal-difference error

The Bellman equation for the Q-function of a fixed policy is

$$Q^\pi(x, u) = R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \sum_{u' \in \mathcal{U}} \pi(x', u') Q^\pi(x', u').$$

Monte Carlo learning estimates Q^π from complete returns. Temporal-difference learning uses a different idea: instead of waiting until the end of the episode, it uses the Bellman equation locally, one transition at a time.

Consider one individual data point

$$x_k,\ u_k,\ x_{k+1},\ u_{k+1},\ r_k.$$

Given a current estimate Q, define the temporal-difference error as

$$e_k = r_k + \gamma Q(x_{k+1}, u_{k+1}) - Q(x_k, u_k).$$

If $Q = Q^\pi$, then the Bellman equation implies that the expected temporal-difference error is zero:

$$\mathbb{E}[e_k] = 0.$$

Thus, the TD error measures the mismatch between the current Q-value estimate and the one-step Bellman target

$$r_k + \gamma Q(x_{k+1}, u_{k+1}).$$

If the TD error is positive, the current estimate $Q(x_k, u_k)$ is too small compared with the observed one-step target. If it is negative, the current estimate is too large.

Temporal-difference error

For one observed transition

$$x_k,\ u_k,\ r_k,\ x_{k+1},\ u_{k+1},$$

the TD error is

$$e_k = r_k + \gamma Q(x_{k+1}, u_{k+1}) - Q(x_k, u_k).$$

At the correct Q-function, this error has zero expectation.

10.4 Stochastic approximation

Temporal-difference learning can be interpreted through stochastic approximation. Suppose we have a random variable $e(q)$, depending on a parameter q, and we want to find a value $q^\star$ satisfying

$$\mathbb{E}[e(q^\star)] = 0.$$

If we had access to the exact expectation, we could solve this equation directly. In learning, however, we only observe samples of $e(q)$. Stochastic approximation provides an iterative method to solve such equations from noisy observations.

The basic iteration is

$$q_{k+1} \leftarrow q_k - \alpha_k e(q_k).$$

Under suitable assumptions, this converges to a solution $q^\star$ of

$$\mathbb{E}[e(q)] = 0.$$

Typical sufficient conditions are:

- the random variable $e(q)$ is bounded;
- the function $\mathbb{E}[e(q)]$ is non-decreasing in q, and increasing at $q^\star$;
- the step sizes satisfy

$$\sum_{k=0}^{\infty} \alpha_k = \infty, \qquad \sum_{k=0}^{\infty} \alpha_k^2 < \infty.$$

The first condition avoids arbitrarily large noisy updates. The second condition ensures that the expected update points toward the root. The last condition is the classical non-summable / square-summable requirement: the steps are large enough to keep learning forever, but small enough for the accumulated noise to remain controlled.

This viewpoint explains why temporal-difference methods update the Q-function directly from sampled TD errors.

10.5 TD-learning and SARSA

TD-learning uses empirical evaluations of the temporal-difference error

$$e_k = r_k + \gamma Q(x_{k+1}, u_{k+1}) - Q(x_k, u_k)$$

to update the Q-function. Since the desired condition is $\mathbb{E}[e_k] = 0$, the Q-value associated with the visited state-action pair is corrected in the direction of the TD error:

$$Q(x_k, u_k) \leftarrow Q(x_k, u_k) + \alpha_k \left(r_k + \gamma Q(x_{k+1}, u_{k+1}) - Q(x_k, u_k) \right).$$

This algorithm is called SARSA because the update uses the quintuple

$$(x_k, u_k, r_k, x_{k+1}, u_{k+1}).$$

The next action u_{k+1} is the action actually selected by the current policy. Therefore, SARSA evaluates and improves the policy that is being followed.

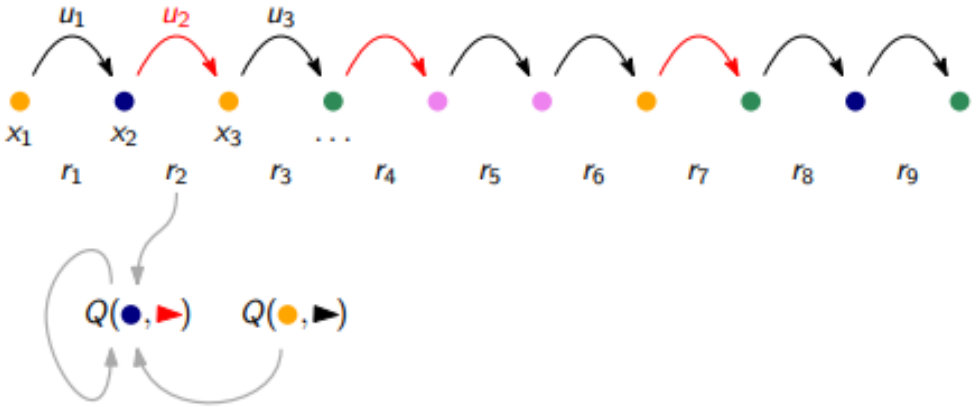

Figure 15: TD-learning with SARSA. Each observed transition immediately produces a one-step update of the Q-value associated with the visited state-action pair.

Compared with Monte Carlo evaluation, the first advantage is that SARSA uses episode data more efficiently. It does not wait until the full return g_k is available. Instead, it updates $Q(x_k, u_k)$ immediately after observing r_k, x_{k+1}, and u_{k+1}.

The second important point is conceptual. SARSA enforces the Bellman equation as a consistency principle during learning, instead of assuming that the Bellman equation will emerge only asymptotically from complete-return averages.

If the step size α_k is kept constant, the algorithm usually does not converge exactly to a deterministic limit. Instead, it fluctuates around the solution with a steady-state nonzero variance. Decreasing step sizes reduce this variance and are used in convergence theory.

10.5.1 Interleaving policy evaluation and improvement

With Monte Carlo policy iteration, one may think of alternating between:

- complete policy evaluation, based on one or more full episodes;
- policy improvement.

Temporal-difference learning makes it possible to interleave these two operations. At every time k, one can:

1. select the action u_k using the latest estimate of $Q(x_k, u)$;
2. observe r_k, x_{k+1}, and u_{k+1};
3. update $Q(x_k, u_k)$ using the TD rule.

A common action-selection rule is ε-greedy:

$$u_k \leftarrow \begin{cases} u \in \mathcal{U} Q(x_k, u) & \text{with probability } 1 - \varepsilon, \\ \text{Uniform}(\mathcal{U}) & \text{with probability } \varepsilon. \end{cases}$$

The parameter ε controls exploration. During learning, one often lets

$$\varepsilon \to 0 \qquad \text{as} \qquad k \to \infty.$$

Thus, the controller explores significantly at the beginning and becomes increasingly greedy as the Q-function estimate improves.

In theory, this removes the need for clearly separated episodes: the policy can be improved online as data are collected. In practice, episodic implementations remain common, but the important conceptual step is that learning can be performed transition by transition.

10.6 Q-learning

SARSA uses the TD error to evaluate the current policy. Q-learning uses a closely related idea, but applies stochastic approximation directly to the Bellman optimality equation.

The optimal Q-function satisfies

$$Q^\star(x, u) = R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \left(\max_{u' \in \mathcal{U}} Q^\star(x', u') \right).$$

For an individual transition, the corresponding sample realization of the Bellman optimality target is

$$r_k + \gamma \max_{u' \in \mathcal{U}} Q(x_{k+1}, u').$$

Therefore, the Q-learning update is

$$Q(x_k, u_k) \leftarrow Q(x_k, u_k) + \alpha_k \left(r_k + \gamma \max_{u' \in \mathcal{U}} Q(x_{k+1}, u') - Q(x_k, u_k) \right).$$

This is the same temporal-difference structure as SARSA, but with a crucial difference: the next action used in the update is not the action actually taken by the policy. Instead, the update uses the greedy value

$$\max_{u' \in \mathcal{U}} Q(x_{k+1}, u').$$

For this reason, Q-learning is an **off-policy** algorithm.

Q-learning update

For an observed transition

$$x_k,\ u_k,\ r_k,\ x_{k+1},$$

the Q-learning update is

$$Q(x_k, u_k) \leftarrow Q(x_k, u_k) + \alpha_k \left(r_k + \gamma \max_{u' \in \mathcal{U}} Q(x_{k+1}, u') - Q(x_k, u_k) \right).$$

There are several important points to notice.

First, the input u_{k+1} is irrelevant for the update. The action actually selected at the next state does not need to be optimal, because the update assumes optimal future play through the maximum over u'.

Second, convergence to the optimal Q-function is guaranteed under classical assumptions such as sufficient exploration of all state-action pairs and suitable step sizes satisfying the non-summable / square-summable conditions.

Third, the policy is typically updated as ε-greedy. Even though the learning update is off-policy, exploration is still needed in order to visit the state-action space. In practice, it is often useful to explore more actions at the beginning and then gradually concentrate exploration around high-reward actions.

Finally, Q-learning can be interpreted as a forward version of dynamic programming. Instead of sweeping over all states using a known model, it updates the Q-function along the trajectory generated by interaction with the system.

10.7 SARSA versus Q-learning

The main difference between SARSA and Q-learning is the following:

$$\text{SARSA learns on-policy,} \qquad \text{Q-learning learns off-policy.}$$

SARSA evaluates the policy that is actually being used. The update contains

$$Q(x_{k+1}, u_{k+1}),$$

where u_{k+1} is the action selected by the current, possibly exploratory, policy. Therefore, SARSA takes into account the fact that future actions may still include exploration.

Q-learning instead uses

$$\max_{u' \in \mathcal{U}} Q(x_{k+1}, u'),$$

independently of which action will actually be taken next. Hence it learns the optimal Q-function associated with greedy future behavior, even while the data are generated by an exploratory policy.

This makes SARSA more conservative. Since it learns the value of the exploratory policy, it accounts for the residual exploration that will occur in the future. Q-learning is more aggressive: it assumes optimal future action selection and therefore pushes the policy toward the optimal greedy strategy more directly.

In reward maximization, once the optimal Q-function has been learned, the optimal policy is obtained by extracting

$$\mu^\star(x) \in_{u \in \mathcal{U}} Q^\star(x, u).$$

10.8 Q-learning as stochastic gradient descent

The Q-learning update can also be interpreted as a stochastic gradient step for minimizing the error in the Bellman optimality principle.

Consider the Bellman optimality residual for a state-action pair (x, u):

$$R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \max_{u' \in \mathcal{U}} Q(x', u') - Q(x, u).$$

A natural loss function is

$$\mathcal{L} = \frac{1}{2}\left(R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \max_{u' \in \mathcal{U}} Q(x', u') - Q(x, u)\right)^2.$$

Here the current estimate of $Q(x', u')$ is used inside the target. This is called **bootstrapping**: the target itself depends on the current value function estimate.

If we differentiate the loss with respect to the decision variable $Q(x, u)$, treating the target as fixed, we obtain

$$\nabla_{Q(x,u)} \mathcal{L} = -\left(R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \max_{u' \in \mathcal{U}} Q(x', u') - Q(x, u)\right).$$

The model-free sample version of this gradient is

$$-\left(r_k + \gamma \max_{u' \in \mathcal{U}} Q(x_{k+1}, u') - Q(x_k, u_k)\right).$$

A stochastic gradient descent step therefore gives exactly the Q-learning update:

$$Q(x_k, u_k) \leftarrow Q(x_k, u_k) + \alpha_k \left(r_k + \gamma \max_{u' \in \mathcal{U}} Q(x_{k+1}, u') - Q(x_k, u_k)\right).$$

Thus, Q-learning can be viewed as stochastic gradient descent on the Bellman optimality error.

10.9 Q-learning with linear approximation

For large state-action spaces, a tabular representation of $Q(x, u)$ is not scalable. As before, we introduce a linear parametrization

$$Q_\theta(x, u) = \sum_{\ell=1}^{d} \phi_\ell(x, u)\theta_\ell = \phi(x, u)^\top \theta.$$

The goal is now to update the parameter vector θ so that Q_θ approximately satisfies the Bellman optimality principle

$$Q^\star(x, u) = R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \left(\max_{u' \in \mathcal{U}} Q^\star(x', u')\right).$$

In terms of the parametrization, this would require

$$\phi(x, u)^\top \theta^\star = R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \left(\max_{u' \in \mathcal{U}} \phi(x', u')^\top \theta^\star\right).$$

In general, this equation does not have an exact solution in the chosen function class. The parametrization may be too restrictive, so no parameter vector θ may satisfy the Bellman optimality equation for all state-action pairs simultaneously.

For a fixed sample, define the target

$$Q^+ = R_x^u + \gamma \sum_{x' \in \mathcal{X}} P_{x,x'}^u \left(\max_{u' \in \mathcal{U}} \phi(x', u')^\top \theta\right).$$

Then a least-squares loss for the current state-action pair is

$$\min_\theta \frac{1}{2}\left(\phi(x, u)^\top \theta - Q^+\right)^2.$$

The corresponding gradient descent iteration is

$$\theta_{k+1} = \theta_k - \alpha \left(\phi(x,u)^\top \theta_k - Q^+ \right) \phi(x,u).$$

In a model-free setting, we do not know the expectation defining Q^+. We replace it by the sampled target

$$r_k + \gamma \max_{u \in \mathcal{U}} \phi(x_{k+1}, u)^\top \theta_k.$$

This gives the stochastic gradient update

$$\theta_{k+1} = \theta_k - \alpha_k \left(\phi(x_k,u_k)^\top \theta_k - \left(r_k + \gamma \max_{u \in \mathcal{U}} \phi(x_{k+1},u)^\top \theta_k \right) \right) \phi(x_k,u_k).$$

Equivalently,

$$\theta_{k+1} = \theta_k + \alpha_k \left(r_k + \gamma \max_{u \in \mathcal{U}} Q_{\theta_k}(x_{k+1},u) - Q_{\theta_k}(x_k,u_k) \right) \phi(x_k,u_k).$$

This is Q-learning with a linear approximator.

10.10 General approximators

Linear parametrizations are useful because they are simple and transparent, but more complex function approximators can also be used. For instance, one can represent the Q-function by a neural network

$$Q_\theta(x,u),$$

where θ collects all the weights of the network.
The requirements are conceptually the same:

- the approximator must provide a parametrized function $Q_\theta(x,u)$;
- it must be possible to compute or approximate

$$\max_{u \in \mathcal{U}} Q_\theta(x,u);$$

- it must be possible to perform a stochastic gradient step in θ to reduce a Bellman-error loss.

For neural networks, the gradient step is computed by backpropagation. The resulting methods are the basis of deep Q-learning and related algorithms.

10.11 Is it really model-free?

Monte Carlo and temporal-difference methods seem to avoid the need for a model of the system. They do not require explicit transition probabilities $P^u_{x,x'}$, and they can update the value or Q-function directly from sampled data.

However, there is still implicit structure. In particular, these methods assume that the process admits a Markovian state x. The Q-function

$$Q(x,u)$$

is meaningful only if x contains all information needed to predict the future distribution of rewards after choosing action u. Thus, even if the transition probabilities are not known, we are still assuming that the chosen state representation is Markovian.

This raises a further question: is it possible to learn the policy directly, without first learning a value function or Q-function? This leads to policy-gradient methods.

10.12 Policy gradient methods

Instead of representing a value function, policy-gradient methods directly parametrize the policy. Consider a trajectory of the system

$$\tau = (x_0, u_0, x_1, u_1, \dots, x_T, u_T, x_{T+1}).$$

For simplicity, assume that the initial state x_0 is fixed across episodes.

The reward associated with the trajectory is

$$R(\tau) = \sum_{t=0}^{T} \gamma^t r_t.$$

We now introduce a stochastic policy parametrized by a vector θ:

$$\pi_\theta(x, u).$$

The parametrization may be:

- based on a linear combination of features $\phi(x, u)^\top \theta$;
- based on a parametric distribution, such as a Gaussian policy;
- based on a neural network with weights θ.

The goal is to maximize the expected reward

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)].$$

Equivalently,

$$J(\theta) = \sum_{\tau} \mathbb{P}_\theta(\tau) R(\tau),$$

where $\mathbb{P}_\theta(\tau)$ is the probability that the trajectory τ occurs when the policy π_θ is used.

There are two sources of complexity:

1. $J(\theta)$ is an expectation, hence it involves a sum over all possible trajectories;
2. $\mathbb{P}_\theta(\tau)$ is unknown, because it depends both on the system transition probabilities and on the policy.

The probability of a trajectory can be factorized as

$$\mathbb{P}_\theta(\tau) = \prod_{t=0}^{T} \underbrace{\mathbb{P}(x_{t+1} \mid x_t, u_t)}_{\text{transition probabilities}} \underbrace{\pi_\theta(x_t, u_t)}_{\text{policy}}.$$

This expression separates the two mechanisms that generate a trajectory. The environment determines the next state through the transition probabilities, while the controller determines the action through the policy.

The central idea of policy-gradient methods is to optimize $J(\theta)$ directly, using sampled trajectories to estimate the gradient with respect to θ. The derivation of such estimators starts from the trajectory-probability factorization above.

A Convex optimization

Convex optimization appears repeatedly in computational control. In unconstrained LQR, the stage optimization is a convex quadratic problem and can be solved in closed form. In constrained MPC, the online problem is typically a convex quadratic program. In feedback optimization, first-order methods, projections, and dual updates are all built on convexity assumptions. For this reason, it is useful to collect in one place the basic facts on convex sets, convex functions, and optimality conditions that make these methods computationally tractable.

At a high level, modern computers are particularly effective at two things: linear algebra and iteration. Linear algebra makes it possible to manipulate vectors and matrices efficiently, solve linear systems, and exploit structure in large-scale problems. Iteration makes it possible to repeat simple update rules until convergence. Numerical optimization sits exactly at the intersection of these two capabilities. This is one of the main reasons why optimization-based control can be implemented in real time when the underlying problem has the right structure, and convexity is precisely the property that provides such structure.

We consider the generic optimization problem

$$\begin{aligned} \min_{x \in X} \quad & f(x) \\ \text{s.t.} \quad & g(x) \leq 0, \\ & h(x) = 0, \end{aligned} \tag{14}$$

where $x \in X \subseteq \mathbb{R}^n$ is the decision variable, $f : \mathbb{R}^n \to \mathbb{R}$ is the cost function, $g : \mathbb{R}^n \to \mathbb{R}^m$ collects inequality constraints, and $h : \mathbb{R}^n \to \mathbb{R}^p$ collects equality constraints.
This formulation is general enough to include many control problems. For example, the finite-horizon MPC problem can be written in this form after stacking all predicted states and inputs into a single vector. The key question is then not only whether a minimizer exists, but whether it can be computed efficiently and reliably online. Convexity provides a positive answer in a large class of relevant cases.

A.1 Convex sets and convex functions

We start from the geometric notion of convexity.
A set $X \subseteq \mathbb{R}^n$ is called *convex* if for every $x', x'' \in X$ and every $t \in [0, 1]$,

$$tx' + (1 - t)x'' \in X.$$

In words, whenever two points belong to the set, the whole line segment joining them also belongs to the set.
A function $\phi : \mathbb{R}^n \to \mathbb{R}$ is called *convex* on a convex domain if for every x', x'' in its domain and every $t \in [0, 1]$,

$$\phi(tx' + (1 - t)x'') \leq t\phi(x') + (1 - t)\phi(x'').$$

This means that the graph of the function lies below the straight line joining the function values at the two endpoints.

These two definitions are closely related. A fundamental fact is that if g is convex, then every sublevel set

$$\{x \mid g(x) \leq c\}$$

is convex. Indeed, if x' and x'' both satisfy $g(x) \leq c$, then by convexity

$$g(tx' + (1 - t)x'') \leq tg(x') + (1 - t)g(x'') \leq tc + (1 - t)c = c,$$

so the convex combination remains in the sublevel set.

The converse, however, is false in general: it is possible for a function to have convex sublevel sets without being convex. Therefore, convexity of the function is a stronger property than convexity of one particular feasible region.

Several sets that appear naturally in control and optimization are convex.
A *halfspace* is a set of the form

$$\{x \mid a^\top x \leq b\},$$

where $a \in \mathbb{R}^n$ and $b \in \mathbb{R}$. Geometrically, a halfspace is one side of a hyperplane.
A *ball* centered at c with radius r is a set of the form

$$\{x \mid \|x - c\| \leq r\}.$$

Because norms are convex, balls are convex sets.
An important class of sets in MPC is given by *polytopes*. These are intersections of finitely many halfspaces:

$$\{x \mid a_i^\top x \leq b_i,\ i = 1, \ldots, r\} = \{x \mid Ax \leq b\}.$$

State and input constraints in linear MPC are usually expressed exactly in this form.
It is useful to distinguish between unions and intersections. The intersection of convex sets is always convex, because any line segment that remains in each set separately also remains in their common intersection. In contrast, the union of convex sets is not necessarily convex. Two disjoint intervals on the real line already provide a counterexample. This simple observation is important in practice: adding constraints tends to preserve convexity, while combining alternative feasible regions often destroys it.

A key geometric property of convex sets is the existence of supporting hyperplanes. Intuitively, a supporting hyperplane touches the boundary of a convex set without cutting through it.
Suppose a set is described as

$$X = \{x \mid g(x) \leq 0\},$$

and let z be a boundary point such that $g(z) = 0$ and g is differentiable at z. Then a local supporting hyperplane is given by

$$\nabla g(z)^\top (x - z) \leq 0.$$

Equivalently,

$$\nabla g(z)^\top x \leq \nabla g(z)^\top z.$$

This is the first-order linear approximation of the boundary at the point z. The result explains why affine constraints are so natural in optimization: they describe exactly the geometry of tangent or supporting hyperplanes.
The picture becomes more delicate when the boundary has a kink and the gradient is not defined. In that case, one has to replace the gradient by a more general notion such as a subgradient. For the purposes of this handout, the differentiable case is the most relevant one.

Several classes of convex functions occur repeatedly in control.

Affine functions. Any function of the form

$$f(x) = a^\top x + b$$

is convex. In fact, affine functions are both convex and concave.

Norms. Functions of the form

$$f(x) = \|x - c\|$$

are convex. This explains why norm-based penalties are common in estimation, robust control, and optimization.

Quadratic functions. A quadratic function

$$f(x) = x^\top A x + b^\top x + c$$

is convex if and only if $A \succeq 0$, that is, if A is positive semidefinite. This is the basic reason why quadratic costs in LQR and MPC lead to convex optimization problems when the constraints are also convex.

A.2 Derivative-based characterizations of convexity

For differentiable and twice differentiable functions, convexity can be tested through derivatives. If f is differentiable on a convex domain, then f is convex if and only if

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) \qquad \forall x, y.$$

This means that the tangent hyperplane at any point underestimates the function globally. Equation (A.2) is one of the most useful characterizations of convexity, both conceptually and algorithmically.
If f is twice differentiable on a convex domain, then f is convex if and only if

$$\nabla^2 f(x) \succeq 0 \qquad \forall x.$$

Thus, convexity can be checked by testing whether the Hessian is positive semidefinite everywhere. In one dimension, this reduces to the familiar condition $f''(x) \geq 0$. In several dimensions, the Hessian plays the same role, but now curvature must be nonnegative in every direction.

A.3 Convex optimization problems

We now return to the constrained problem in (14). It is called a *convex optimization problem* if

- X is a convex set,
- f is convex,
- each inequality constraint function g_i is convex,
- each equality constraint is affine.

Under these assumptions, the feasible set

$$X \cap \{x \mid g(x) \leq 0\} \cap \{x \mid h(x) = 0\}$$

is convex. The reason the equality constraints must be affine is that a general nonlinear equality constraint would typically define a nonconvex set.
Convex optimization problems are much simpler than general nonlinear programs for a fundamental reason: there are no spurious local minima. This is the main structural property that makes them attractive in real-time control.

A point $x^\star$ is a local minimum if there exists $R > 0$ such that

$$f(z) \geq f(x^\star)$$

for all feasible z satisfying $\|z - x^\star\| \le R$.
For convex problems, local optimality automatically implies global optimality. The proof is short and very instructive. Assume that $x^\star$ is locally optimal but not globally optimal. Then there exists a feasible point y such that

$$f(y) < f(x^\star).$$

Because the feasible set is convex, every point on the segment

$$z_t = ty + (1-t)x^\star, \qquad t \in [0,1]$$

is feasible. Choosing $t > 0$ small enough, we can place z_t inside the ball of radius R around $x^\star$. By convexity of f,

$$f(z_t) \le tf(y) + (1-t)f(x^\star) < f(x^\star),$$

which contradicts local optimality. Hence every local minimum is global.
This single fact explains much of the power of convex optimization: once a candidate point is known to be locally optimal, the search is over.

A.4 First-order optimality conditions

Suppose the problem is differentiable. Then a feasible point $x^\star$ is locally optimal if and only if

$$\nabla f(x^\star)^\top (y - x^\star) \ge 0 \qquad \text{for all feasible } y. \tag{15}$$

This condition says that, at an optimum, the gradient cannot point toward any feasible descent direction.
In the unconstrained case, every direction is feasible, so (15) reduces to

$$\nabla f(x^\star) = 0.$$

Thus, unconstrained optimization leads to the familiar linear algebra test of setting the gradient equal to zero.
The constrained case is more interesting. At the optimum, the negative gradient may point outside the feasible set, so it cannot be canceled by free motion of the decision variable. Instead, it must be balanced by the geometry of the active constraints. This leads to the Karush–Kuhn–Tucker conditions.

A.5 KKT conditions

Consider the constrained problem

$$\begin{aligned} \min_{x \in X} \quad & f(x) \\ \text{s.t.} \quad & g(x) \le 0, \\ & h(x) = 0. \end{aligned}$$

Under suitable regularity assumptions, a local optimum $x^\star$ satisfies the KKT conditions: there exist multiplier vectors $\mu \in \mathbb{R}^m$ and $\lambda \in \mathbb{R}^p$ such that

$$\nabla f(x^\star) + \sum_{i=1}^{m} \mu_i \nabla g_i(x^\star) + \nabla h(x^\star)^\top \lambda = 0, \tag{16}$$

$$g_i(x^\star) \le 0, \qquad i = 1, \dots, m, \tag{17}$$

$$h(x^\star) = 0, \tag{18}$$

$$\mu_i \ge 0, \qquad i = 1, \dots, m, \tag{19}$$

$$\mu_i g_i(x^\star) = 0, \qquad i = 1, \dots, m. \tag{20}$$

Each condition has a clear meaning. Equation (16) is the stationarity condition: the gradient of the cost is balanced by a linear combination of the gradients of the active constraints. Conditions (17) and (18) express primal feasibility. Condition (19) says that inequality multipliers must be nonnegative. Finally, (20) is the complementary slackness condition: if a constraint is inactive, then its multiplier is zero; if its multiplier is positive, then the constraint must be active.
For convex optimization problems, these conditions are especially powerful. Under standard constraint qualifications, they are not only necessary but also sufficient. In other words, solving the KKT system is enough to recover the global optimizer.

A.6 Lagrangian viewpoint

The KKT conditions can be organized through the Lagrangian

$$\mathcal{L}(x, \mu, \lambda) := f(x) + \mu^\top g(x) + \lambda^\top h(x).$$

From this viewpoint, constrained optimization can be interpreted as the search for a saddle point of the Lagrangian: one minimizes with respect to the primal variable x and maximizes with respect to the dual variables associated with the inequalities and equalities.
This viewpoint is central in many numerical methods. It is also conceptually useful in control, because dual variables often admit an interpretation as shadow prices, sensitivities, or integral correction signals. This becomes particularly visible in primal–dual algorithms and dual ascent methods.

There are two complementary ways to think about convex optimization.
The first is the *algebraic view*. Since the KKT conditions characterize the optimum, one may try to solve a structured system of equations involving the primal variable and the multipliers. In quadratic programming, this often leads to linear algebraic systems with a very specific sparsity pattern.
The second is the *geometric or iterative view*. Since the gradient provides a descent direction and convexity rules out bad local minima, one can use iterative algorithms such as gradient descent, projected gradient descent, dual ascent, or interior-point methods. These algorithms repeatedly apply simple updates until convergence.

This is exactly where the earlier observation becomes relevant again: computers are good at linear algebra and iteration. Convex optimization exploits both. The geometry guarantees that the iterates move toward the correct solution, while the algebraic structure makes each update computationally efficient. This is the reason convex programs can often be solved fast enough for online control.

A.7 Connection with MPC and control

The relevance of convex optimization for control can now be summarized clearly.
In unconstrained LQR, the optimization problem is quadratic and unconstrained, so the optimum can be computed analytically through the Riccati recursion. In constrained MPC with linear dynamics, quadratic cost, and polyhedral state and input constraints, the online problem is a convex quadratic program. Because the problem is convex, the optimizer is globally meaningful, the value function is well behaved, and the explicit MPC law becomes piecewise affine when it is computed offline.
More generally, many feedback-optimization schemes rely on repeatedly solving simple convex subproblems. Projection steps, local quadratic approximations, and primal–dual iterations are all practical only because the underlying subproblems preserve convexity.